ACADEMIC & TECHNICAL VIVA MASTER GUIDE

TRIPWISE AI

Full-Stack AI Travel Itinerary & City Intelligence System

Complete repository study manual, architecture reference, line-by-line code explanation, database schema guide, system flowcharts, and 150 categorized viva interview questions.

Frontend Architecture

React 19, Vite, Tailwind CSS, React Router v7, Leaflet interactive maps, Recharts budget analytics, and Lucide React icons.

Backend & APIs

Node.js 22, Express 4.21, RESTful endpoints, CORS security, custom logging, and scrypt/crypto authentication.

Gemini AI Integration

Multi-model fallback cascade (Gemini 2.5 Flash), zero-shot structured JSON extraction, and client-side offline heuristics.

Relational Persistence

Native `node:sqlite`, 7 relational tables, Write-Ahead Logging (WAL), foreign key cascading, and ACID transactions.

External Services

Open-Meteo weather API (live 7-day forecast), Nominatim OpenStreetMap debounced geocoding, and Unsplash imagery.

Viva & Interview Master

150 categorized questions & model answers, 10 production bugs explained with root causes, and an 8-day preparation plan.

Candidate Name:
Saumya Pant

Repository:
tripwise-ai (Verified Source)

Document Version:
March 2026 • Final Academic Edition

Table of Contents

00 Guide Index & Recommended Reading Pathways

01 Project Overview, Problem Statement & Viva Pitches

02 Complete Tech Stack & Architectural Decisions

03 Page-by-Page Screen Architecture & Tech Breakdown

04 Complete Repository Structure & File Purposes

05 Frontend Deep-Dive: React 19, Vite, State & Hooks

06 Trip Planning Form & Dual-Tier City Search Engine

07 Backend Architecture, Middleware & REST APIs

08 Google Gemini AI Multi-Model Cascade & Fallbacks

09 Native SQLite Database, Schema & ACID Transactions

10 External Third-Party API Integrations

11 System Flowcharts & Architecture Sequence Diagrams

12 Core Codebase Walkthrough & Line Explanations

13 Technical Glossary (60+ Terms Defined Simply)

14 150 Viva & Technical Interview Questions & Model Answers

15 10 Real-World Bugs Encountered, Root Causes & Fixes

16 Comprehensive Verification & Quality Assurance Guide

17 8-Day Viva Study Schedule & Daily Milestones

18 Presentation Morning 1-Page Quick Revision Sheet

PART 00

TripWise AI — Complete Project Study Guide & Viva Hub

Welcome to the **TripWise AI Project Study Guide**. This directory contains 17 comprehensive, in-depth study guides written directly from the verified source code of the TripWise AI repository. It covers everything from high-level project motivations to line-by-line code explanations, architecture diagrams, database schemas, and 150 categorized viva interview questions.

📚 Table of Contents

Guide	File Name	Topics Covered
01	01_PROJECT_OVERVIEW.md	Problem statement, real-world value, user journeys, and 30s / 1m / 3m viva pitches.
02	02_TECH_STACK.md	Exact versions from package.json, tech stack choices, and alternatives considered.
03	03_PROJECT_FILES_EXPLAINED.md	File-by-file purpose, architecture map, imports, and system connections.
04	04_FRONTEND_GUIDE.md	React 19, Vite, component tree, state management, props vs state, and hooks in depth.
05	05_FORM_AND_CITY_SEARCH.md	Empty form state, input validation, 178-city catalog, and 320ms debounced Nominatim geocoding.
06	06_BACKEND_GUIDE.md	Express 4 architecture, middleware pipeline (CORS, body parsing), status codes, and REST endpoints table.
07	07_GEMINI_INTEGRATION.md	Prompt engineering, multi-model cascade, markdown sanitization, duplicate activity detection, and fallback engine.
08	08_DATABASE_GUIDE.md	Native `node:sqlite`, 7 relational tables, foreign key cascades, WAL mode, and ACID transactions.
09	09_API_INTEGRATIONS.md	Open-Meteo weather API, Nominatim geocoder, Wikipedia REST API, and Unsplash service.
10	10_FLOWCHARTS.md	Visual Mermaid diagrams: Startup, Trip Generation, City Search, Auth, and DB Transactions.
11	11_CODE_EXPLANATION.md	Snippet-by-snippet walkthrough of core frontend and backend files.
12	12_TECHNICAL_GLOSSARY.md	60+ terms defined simply with analogies and technically with project context.
13	13_VIVA_QUESTIONS.md	150 categorized questions and model answers (30 Basic, 30 Int, 30 Adv, 20 Code, 20 Bugs, 20 Decisions).
14	14_BUGS_AND_FIXES.md	10 real bugs encountered, root cause analysis, code diffs, and interview explanation scripts.
15	15_TESTING_GUIDE.md	Manual UI test cases, automated curl scripts, edge case tests, and fallback testing.
16	16_STUDY_PLAN.md	Practical 8-day daily schedule with tasks, files to read, and self-test checkpoints.
17	17_FINAL_CHEAT_SHEET.md	Presentation morning 1-page quick revision sheet, key numbers, top 10 answers, and live demo golden path.
18	TECH_STACK_BY_PAGE.md	Comprehensive page-by-page breakdown of technologies, libraries, hooks, and architectural components across all 7 pages.

🚀 Recommended Reading Order

1. **For an Upcoming Presentation (Under 24 Hours):**
   - Start with 17_FINAL_CHEAT_SHEET.md.
   - Memorize the 30-second and 1-minute pitches in 01_PROJECT_OVERVIEW.md.
   - Skim the top 10 viva questions in 13_VIVA_QUESTIONS.md.
   - Follow the **Live Demo Golden Path** in the cheat sheet.
2. **For Comprehensive Viva & Technical Interview Preparation (1-2 Weeks):**
   - Follow the daily plan in 16_STUDY_PLAN.md.
   - Read guides 01 through 15 sequentially.
   - Practice explaining the bugs in 14_BUGS_AND_FIXES.md.

PART 01

TripWise AI — Project Overview & Pitch Guide

Welcome to the comprehensive study guide for **TripWise AI**. This document is designed to take you from a basic conceptual understanding to a confident, articulate presenter in any college viva, project evaluation, or technical interview.

1. What is TripWise AI?

TripWise AI is a full-stack, AI-powered intelligent travel planning and itinerary generation web application. It automates the complex, time-consuming process of vacation planning by transforming high-level user preferences (origin, destination, budget, duration, dates, and travel styles) into a day-by-day, hour-by-hour realistic travel itinerary.

Beyond static travel itineraries, TripWise AI enriches the plan with:

- **Interactive Geospatial Mapping:** Leaflet-powered maps displaying exact geographic coordinates and key landmarks.
- **Real-Time Weather Forecasting:** 7-day temperature and precipitation projections via the Open-Meteo API without requiring external API keys.
- **Budget Categorization & Visual Analytics:** Breakdown of expenses across flights, accommodation, food, transportation, activities, and shopping with interactive Recharts data visualizations.
- **Curated & Nearby Points of Interest:** Dynamic attractions, local dining spots, and cultural landmarks.
- **Interactive AI Travel Concierge:** A destination-aware chatbot that answers contextual traveler questions.
- **Persistent Relational Storage:** Complete trip data saved into a normalized SQLite database engine (`node:sqlite`), allowing users to revisit, inspect, and manage saved trips on a personal dashboard.

2. The Real-World Problem It Solves

The Problem: Decision Fatigue & Disjointed Planning

Planning a vacation is traditionally fragmented across dozens of browser tabs:

1. Searching blog posts and travel sites for things to do.
2. Checking separate weather websites to decide what to pack.
3. Manually calculating budgets on spreadsheets.
4. Pinning locations on mapping services to see if daily routes make geographic sense.
5. Worrying if attractions are open or require booking.

This disjointed process takes an average of **10 to 20 hours** of research per trip.

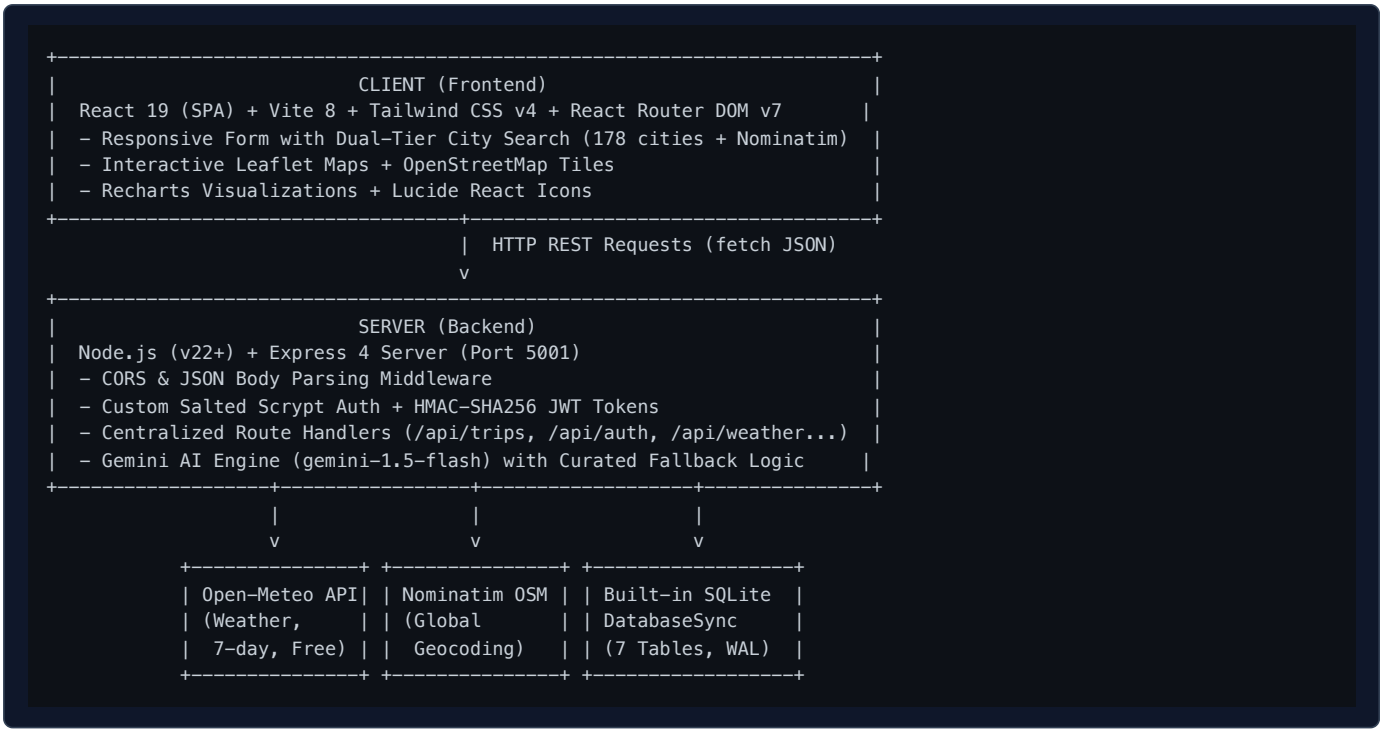
The Solution: TripWise AI

TripWise AI consolidates this entire pipeline into a single workflow:

- The user provides their parameters in a single form.
- The system connects with Google Gemini AI (`@google/generative-ai`) to produce an authentic schedule.
- The backend coordinates geocoding, weather forecasts, destination imagery, and budget calculations in parallel.
- Everything is delivered on an interactive, responsive dashboard and saved in a relational database for instant recall.

3. High-Level System Architecture

TripWise AI follows a clean **Client-Server-Database Architecture**:



4. End-to-End User Journeys

Journey A: Creating a New Trip as a Guest or Authenticated User

1. **Landing Page (`/`)**: The user views the hero section, curated trips, and feature cards. Clicking **"Create Trip"** or **"Start Planning"** navigates to `/generate-trip`.
2. **Form Input (`/generate-trip`)**:
   - The form is strictly blank by default (no hardcoded cities or pre-selected values).
   - The user searches their **Departure City** (e.g., *Chandigarh, India*) and **Destination City** (e.g., *Athens, Greece*).
   - As the user types in the dropdown, the system matches locally across 178 cities in `< 1ms`, or queries OpenStreetMap Nominatim after 320ms debouncing.
   - The user enters budget (e.g., `1800`), duration (e.g., `5` days), dates, and selects travel styles (e.g., *Culture*, *City Exploration*).
3. **Trip Generation & Persistence**:
   - The frontend sends a `POST /api/trips/generate` request.
   - Express queries the Gemini AI service while fetching destination imagery and geocodes concurrently via `Promise.all`.
   - The backend runs an atomic SQLite transaction inserting the trip, days, activities, budget breakdown, and highlights.
4. **Trip Details Dashboard (`/trip-details?id=...`)**:
   - The user is redirected to the generated trip view.
   - Left column displays the day-by-day itinerary with timings, activity tags, costs, and AI tips.
   - Right column presents weather widgets, the interactive Leaflet map, budget pie/bar charts, and the AI concierge chat button.

Journey B: User Authentication & Saved Trips Dashboard

1. **Sign Up / Login (`/signup`, `/login`)**:
   - Passwords are encrypted server-side using salted `crypto.scryptSync(password, salt, 64)`.
   - The backend issues a signed HMAC-SHA256 JWT token.
   - User identity and token are stored in browser `localStorage`.
2. **Personal Dashboard (`/dashboard`)**:
   - The frontend queries `GET /api/trips?userId=<id>`.
   - Displays all historical trips created by this user with cover photos, duration, and budget.
   - Clicking any card opens that trip's full details.

5. Viva Pitches (Memorize These!)

30-Second Elevator Pitch

> *"TripWise AI is an intelligent travel planning web application built with React 19, Vite, Express, and Node.js SQLite. It solves vacation planning fatigue by taking a user's destination, budget, duration, and style preferences, and generating a realistic, day-by-day itinerary powered by Google Gemini AI. The application coordinates live weather forecasts via Open-Meteo, interactive Leaflet maps, visual budget charts, and persistent storage across 7 relational database tables, complete with an offline fallback engine that guarantees reliable operation even without internet or API keys."*

1-Minute Project Overview

> *"Good morning/afternoon. My project is TripWise AI, a full-stack automated travel planning application. Traditional vacation planning is tedious because users have to cross-reference travel guides, weather sites, map services, and budget spreadsheets across dozens of tabs.
> TripWise AI unifies this workflow. On the frontend, built with React 19 and Tailwind CSS, we created an intuitive form featuring a dual-tier city search: an instant 178-city catalog combined with a debounced OpenStreetMap Nominatim API for worldwide coverage.
> When the user submits the form, our Node.js and Express backend contacts Google's Gemini generative AI model with strict prompt engineering to produce structured, non-repeating daily itineraries. The backend simultaneously fetches live weather from Open-Meteo and authentic imagery. Everything is stored in an embedded SQLite database using Node's native DatabaseSync with full ACID transactions. Even if the Gemini API key is absent or exhausted, our custom curated fallback engine ensures the user still receives an authentic, realistic travel itinerary."*

3-Minute Comprehensive Technical Viva Speech

> *"Respected examiners, I would like to present TripWise AI, an AI-powered travel itinerary and trip management platform.
> **Problem Statement & Motivation:** Travelers spend an average of 10 to 20 hours planning vacations because data is fragmented. Most existing solutions either offer generic static lists or hallucinate impossible schedules. My objective was to build a cohesive full-stack web application that combines generative AI reasoning with real-world geospatial and meteorological data.
> **Architecture & Tech Stack:** The frontend is developed with React 19 and Vite 8, utilizing React Router DOM v7 for declarative routing, Tailwind CSS v4 for modern responsive styling, React-Leaflet for interactive OpenStreetMap rendering, and Recharts for budget data visualization.
> The backend is built with Node.js v22 and Express 4. Instead of adding heavy third-party ORMs or external database dependencies, I utilized Node's native `node:sqlite` module via `DatabaseSync`. The schema is fully normalized into 7 relational tables: `users`, `trips`, `itinerary_days`, `activities`, `budget_breakdown`, `trip_highlights`, and `saved_places`, with foreign keys, cascade deletes, and WAL (Write-Ahead Logging) mode enabled for concurrency.
> **Key Innovations & Engineering Challenges:**
>
> 1. **Dual-Tier Geocoding & City Search:** We integrated a local dataset of 178 global and Indian destinations that responds in under 1 millisecond. If a user types a lesser-known city, an asynchronous 320-millisecond debounced request queries OpenStreetMap Nominatim to resolve coordinates safely.
> 2. **Resilient AI Pipeline:** We integrated `@google/generative-ai` with a multi-model fallback cascade: `gemini-1.5-flash` followed by `gemini-2.0-flash` and `gemini-1.5-pro`. To ensure robust operations during evaluations or offline demos, we engineered a deterministic fallback engine with authentic schedules for major hubs and dynamic procedural scheduling for any custom city.
> 3. **Security & Cryptography:** User authentication is built from the ground up using Node's native `node:crypto`. Passwords use cryptographic salts and `scryptSync` key derivation, compared via `crypto.timingSafeEqual` to eliminate timing attacks. Authentication tokens use a stateless HMAC-SHA256 signed structure.
>
> In conclusion, TripWise AI demonstrates how generative AI can be securely and reliably married with relational databases and external open-source APIs to solve a practical real-world problem."*

PART 02

TripWise AI — Technology Stack Deep Dive

This document provides a detailed breakdown of every technology, library, framework, and tool used in the TripWise AI project. Every version listed is verified directly against `package.json` and `server/package.json`.

1. Complete Technology Inventory

Layer / Domain	Technology	Exact Version	Primary Role in Project
Frontend Framework	**React**	`^19.2.6`	Component-based User Interface (SPA) with declarative rendering
Frontend DOM	**React DOM**	`^19.2.6`	Mounting and updating the React Virtual DOM onto browser DOM
Build Tool & Bundler	**Vite**	`^8.0.12`	Next-generation ESM dev server with lightning-fast HMR and Rollup bundling
CSS Utility Framework	**Tailwind CSS**	`^4.3.0`	Utility-first responsive CSS styling with zero runtime CSS-in-JS overhead
Tailwind Vite Plugin	**@tailwindcss/vite**	`^4.3.0`	Native Vite integration for Tailwind v4 engine
Client-Side Routing	**React Router DOM**	`^7.16.0`	Client-side routing, URL navigation, query string parsing (`/` , `/generate-trip` , `/trip-details` , `/dashboard`)
Interactive Maps	**Leaflet**	`^1.9.4`	Open-source interactive JavaScript mapping engine
React Map Wrapper	**React-Leaflet**	`^5.0.0`	React component bindings for Leaflet map containers, tiles, and markers
Data Visualization	**Recharts**	`^3.8.1`	SVG-based responsive charting library for budget breakdown visualizer
Icon Library	**Lucide React**	`^1.3.0`	Clean, customizable SVG icons throughout UI (pins, calendar, dollar, weather)
Backend Runtime	**Node.js**	`v22.5.0+`	Server runtime executing JavaScript on Google V8 engine with native ESM support
Backend Framework	**Express**	`^4.21.2`	Minimalist HTTP web server handling REST routing, middleware, and controllers
Database Engine	**node:sqlite**	*Built-in*	Native Node.js embedded relational database (`DatabaseSync`) with WAL mode
Generative AI SDK	**@google/generative-ai**	`^0.24.1`	Official Google SDK for Gemini LLM prompting and structured inference
CORS Middleware	**cors**	`^2.8.5`	Cross-Origin Resource Sharing middleware enabling frontend-backend communication
Environment Config	**dotenv**	`^16.4.7`	Loads environment variables from `.env` file into `process.env`
Cryptography	**node:crypto**	*Built-in*	Native password hashing via `scryptSync` , timing defense via `timingSafeEqual` , and HMAC-SHA256 JWTs

2. Why Each Technology Was Chosen & Alternatives Evaluated

A. Frontend: React 19 vs Vanilla JS / Vue / Angular

- **Why React 19?**
  - **Component Reusability:** Complex UI elements like `CityDropdown` , `BudgetCard` , `WeatherCard` , and `InteractiveMap` are self-contained, testable, and reusable.
  - **Virtual DOM Reconciliation:** Frequent updates (such as debounced city typing, toggling style chips, switching itinerary days) re-render with minimal browser DOM repaints.
  - **Ecosystem Maturity:** Robust support for libraries like React-Leaflet and Recharts.

- **Alternatives Considered:**
  - *Vanilla JavaScript:* Requires tedious manual DOM manipulation (`document.createElement` , manual state sync), making complex nested itinerary views unmaintainable.
  - *Angular:* High boilerplate, steep learning curve, and heavyweight framework overhead for a travel planner.
  - *Vue.js:* Excellent reactivity, but React's widespread industry adoption makes it superior for technical viva and portfolio defense.

B. Bundler: Vite 8 vs Create React App (Webpack)

- **Why Vite 8?**
  - **Native ESM:** Vite serves source code over native ES modules during development, starting up in under 300ms compared to Webpack's 30-second cold starts.
  - **Hot Module Replacement (HMR):** Edits to React components reflect instantly without losing form state.
  - **Lightweight Production Builds:** Uses Rollup under the hood with aggressive tree-shaking.
- **Alternatives Considered:**
  - *Create React App (CRA):* Deprecated by the React team; slow Webpack bundling, outdated dependencies, and sluggish build pipelines.

C. Styling: Tailwind CSS v4 vs Bootstrap vs Plain CSS

- **Why Tailwind CSS v4?**
  - **Utility-First Speed:** Eliminates context switching between JSX and separate CSS files.
  - **Zero Unused CSS in Production:** Purges unused utility classes, keeping stylesheet payload under 30 KB.
  - **Tailwind v4 Engine:** Powered by a modern Lightning CSS engine built into Vite (`@tailwindcss/vite`), eliminating legacy PostCSS config files.
- **Alternatives Considered:**
  - *Bootstrap:* Generic "cookie-cutter" look, difficult to customize modern travel cards and dark-mode gradients.
  - *Plain CSS / BEM:* Difficult to maintain across 30+ components without class collision.

D. Backend: Express.js 4 vs Fastify vs NestJS

- **Why Express 4?**
  - **Simplicity & Transparency:** Express is the de facto standard for Node.js REST APIs. Its unopinionated middleware chain (`req` , `res` , `next`) is universally understood and easy to explain in an interview.
  - **Lightweight Footprint:** Does not impose enterprise-level decorators or complex dependency injection frameworks.
- **Alternatives Considered:**
  - *NestJS:* Imposes heavy TypeScript decorator patterns and steep complexity unnecessary for a college project.
  - *Fastify:* Slightly faster request parsing, but smaller ecosystem and harder to demonstrate middleware mechanics simply in a viva.

E. Database: Native `node:sqlite` vs PostgreSQL / MongoDB / MySQL

- **Why Node's Native `node:sqlite` ?**
  - **Zero External Server Dependency:** SQLite is a serverless, self-contained database engine. It stores data in a single file (`server/src/data/tripwise.db`). Anyone who clones the repository can immediately run the project without installing PostgreSQL, Docker, or MySQL servers.
  - **Native Node.js 22+ Integration:** `DatabaseSync` is built directly into Node's standard library (`node:sqlite`). It requires **zero npm packages** to install or compile (no `sqlite3` or `better-sqlite3` native node-gyp build failures on Windows/Mac/Linux).
  - **Full Relational Power:** Supports real relational joins, foreign key constraints (`ON DELETE CASCADE`), indexes, and atomic ACID transactions (`BEGIN TRANSACTION` , `COMMIT` , `ROLLBACK`).
  - **Write-Ahead Logging (WAL):** Enabled via `PRAGMA journal_mode = WAL;` , allowing concurrent readers and writers without database locking errors.
- **Alternatives Considered:**
  - *MongoDB / Mongoose:* NoSQL document stores lack strict relational integrity and require a running MongoDB daemon or Atlas cloud credentials, complicating local examiner evaluation.
  - *PostgreSQL:* Industrial standard, but requires examiners to configure local Postgres services, role permissions, and connection strings.

F. Generative AI: Google Gemini (`@google/generative-ai`) vs OpenAI GPT-4

- **Why Google Gemini?**
  - **Cost & Free Tier Accessibility:** Gemini offers a generous free tier for developers with high RPM (Requests Per Minute), avoiding paywall lockouts during project grading.
  - **Structured Reasoning Speed:** `gemini-1.5-flash` features sub-second response times and high adherence to JSON output schemas.
  - **Large Context Window:** Capable of ingesting comprehensive city details, hotel preferences, and style constraints without token exhaustion.

---

3. Package Scripts Reference

Frontend Scripts (`package.json`)

- `npm run dev`: Boots Vite development server on port 5173 with instant HMR.
- `npm run build`: Compiles JSX and assets into optimized static bundle inside `/dist`.
- `npm run preview`: Locally previews the production build from `/dist`.
- `npm run lint`: Runs ESLint 10 to check React code quality and hook rules.

Server Scripts (`server/package.json`)

- `npm run start`: Starts the production Express server on port 5001 via `node src/index.js`.
- `npm run dev` / `npm run server:dev`: Starts Express with Node's native file watcher (`node --watch src/index.js`), auto-restarting whenever backend files change.
- `npm run db:seed`: Executes the database seed script to populate sample trips and destinations.

PART 03

TripWise AI — Tech Stack by Webpage

This guide breaks down the exact technologies used on every screen of TripWise AI. Every statement is based strictly on the actual implementation in the repository.

01 — Home / Landing Page

- **Route:** `/`
- **File Path:** `src/pages/Home.jsx`
- **Components:** `Navbar.jsx`, `Section1.jsx` (`HeroBanner.jsx`, `HeroImage.jsx`, `WhyChooseUs.jsx`), `Section2.jsx`, `Section3.jsx`, `Footer.jsx`

Technologies & Role on This Page:

- **React 19 & JSX** — Renders the landing page as modular components (`Navbar`, `HeroBanner`, `WhyChooseUs`, `Footer`).
  - *Why needed:* Separates marketing content into organized, reusable blocks instead of one massive HTML file.
  - *How it supports functionality:* Keeps component rendering clean and enables reactive state for the mobile navbar toggle.
- **Tailwind CSS v4** — Provides all styling, responsive grids (`grid md:grid-cols-3`), hero text gradients, and interactive hover animations (`hover:scale-105`).
  - *Why needed:* Delivers a modern travel landing page layout without writing separate CSS files.
  - *How it supports functionality:* Ensures fast loading speeds and adapts seamlessly across mobile phones, tablets, and desktops.
- **React Router DOM (`Link`)** — Powers the primary call-to-action button (`"Generate My Trip ✨"`) and trending city shortcuts (`Tokyo`, `Seoul`, `Paris`, `Kerala`, `New York`).
  - *Why needed:* Enables client-side navigation without full browser page reloads.
  - *How it supports functionality:* Passes destination names directly into the URL query string (e.g. `/generate-trip?destination=Tokyo`) so the form can pre-fill automatically.
- **Lucide React** — Renders clean SVG vector icons (`Plane`, `Compass`, `Sparkles`, `Star`).
  - *Why needed:* Provides lightweight visual icons that scale crisply on any display.
  - *How it supports functionality:* Enhances visual engagement in feature cards without downloading heavy icon fonts.
- **Backend & Database on this page:** *Not directly involved.* The landing page is completely static on the client side for instant initial loading.
- **APIs & External Services on this page:** *None used directly on this page.*

02 — Generate Trip Form

- **Route:** `/generate-trip`
- **File Path:** `src/pages/GenerateTrip.jsx`
- **Components:** `Gnavbar.jsx`, `Tripgenerator.jsx`, `GeneratorHeader.jsx`, `Form.jsx`, `CityDropdown.jsx`

Technologies & Role on This Page:

- **React `useState` Hook** — Manages input states for origin city, destination, budget, duration, calendar dates, travel style chips, loading spinners, and validation error banners.
  - *Why needed:* Maintains an **empty initial state** (no hardcoded defaults) so users always start with a clean form.
  - *How it supports functionality:* Updates values as the user types and triggers validation banners if mandatory fields are missing.
- **React `useMemo` Hook** — Filters our local catalog of 178 cities in RAM based on user keystrokes and selected country.
  - *Why needed:* Filters hundreds of city records in under 1 millisecond.
  - *How it supports functionality:* Prevents expensive recalculations on unrelated component re-renders.
- **React `useEffect` & `useRef` Hooks** — Powers the 320ms debounced remote city search, auto-focuses the search input, and closes the dropdown when clicking outside.

- *Why needed:* Prevents firing API requests on every single keystroke and provides intuitive dropdown behavior.
- *How it supports functionality:* Cancels pending timers (`clearTimeout`) during rapid typing and checks click boundaries via `dropdownRef.current` .

- **In-Memory City Catalog (`src/data/destinations.js`)** — Contains 178 pre-verified global and Indian cities with pre-computed GPS coordinates.
  - *Why needed:* Provides instant offline city recommendations with verified coordinates.
  - *How it supports functionality:* Automatically attaches latitude and longitude to the destination without waiting for external geocoders.
- **OpenStreetMap Nominatim API (`/api/places/search-cities`)** — Fallback geocoding service called via our Express backend when a user searches for an uncataloged city (e.g. *Gwalior*, *Bhopal*).
  - *Why needed:* Delivers global coverage for any town or city worldwide.
  - *How it supports functionality:* Resolves city names to coordinates with a 2.8s safety timeout.
- **Google Gemini AI API (`POST /api/trips/generate`)** — Invoked when the user submits the form.
  - *Why needed:* Generates an authentic, structured, day-by-day itinerary according to user budget and style.
  - *How it supports functionality:* Operates behind our Express proxy with a multi-model cascade (`1.5-flash` , `2.0-flash` , `1.5-pro`) and curated offline fallback.
- **Node.js, Express & SQLite Database** — Backend server receives the form data and runs an atomic transaction (`BEGIN TRANSACTION ... COMMIT`) saving the trip, days, activities, and budget into `server/src/data/tripwise.db` .
  - *Why needed:* Permanently persists the generated trip in a normalized relational database.
  - *How it supports functionality:* Generates a unique trip ID and returns `HTTP 201 Created` .
- **React Router DOM (`useNavigate` , `useLocation`)** — Reads incoming `?destination=` query params and redirects to `/trip-details?id=trip_...` upon successful generation.
  - *Why needed:* Seamlessly transitions the user from form submission to the generated itinerary.

---

03 — Trip Details & Itinerary Dashboard

- **Route:** `/trip-details`
- **File Path:** `src/pages/TripDetails.jsx`
- **Components:** `Tnavbar.jsx` , `Hero.jsx` , `StatsCards.jsx` , `Itinerary.jsx` , `ExploreMapWeather.jsx` , `InteractiveMap.jsx` , `WeatherCard.jsx` , `BudgetCard.jsx` , `TravelTips.jsx` , `DailySpending.jsx` , `ViewFullItinerary.jsx` , `ChatButton.jsx` , `CuratedTrips.jsx`

Technologies & Role on This Page:

- **React Router DOM (`useLocation` , `useSearchParams`)** — Extracts the `?id=trip_...` parameter from the URL.
  - *Why needed:* Tells the page which specific trip to fetch and display.
  - *How it supports functionality:* Enables bookmarking, sharing, and reloading the exact trip URL.
- **React State & Hooks (`useState` , `useEffect` , `useCallback`)** — Manages active tabs (Overview vs. Map/Weather), currency switching, day accordion collapse/expansion, and modal visibility.
  - *Why needed:* Handles heavy user interactions across multiple analytical widgets.
  - *How it supports functionality:* Renders live data updates without page reloads.
- **Leaflet & React-Leaflet (`MapContainer` , `TileLayer` , `Marker` , `Popup`)** — Displays an interactive map centered over the destination coordinates (`lat` , `lon`).
  - *Why needed:* Visualizes the geographical location of the trip without expensive paid map services.
  - *How it supports functionality:* Renders OpenStreetMap tiles, custom marker pins, and smooth pan/zoom controls.
- **Recharts (`PieChart` , `Pie` , `Cell` , `Tooltip` , `ResponsiveContainer`)** — Renders an interactive SVG donut/pie chart visualizing the budget breakdown (flights, hotels, food, transport, activities, shopping).
  - *Why needed:* Converts raw numbers into an immediate, easy-to-read visual financial breakdown.
  - *How it supports functionality:* Shows animated slices with percentage tooltips when hovering.
- **Open-Meteo Weather API (`/api/weather`)** — Fetches live 7-day meteorological weather projections using the destination coordinates.
  - *Why needed:* Informs travelers about expected temperatures and rain without requiring an API key.
  - *How it supports functionality:* Populates the 7-day weather card with high/low temperatures and WMO weather icons.

- **Google Gemini AI (`POST /api/chat` via `ChatButton.jsx`)** — Powers the floating AI Travel Concierge chatbot modal.
  - *Why needed:* Answers custom, conversational traveler questions in real time.
  - *How it supports functionality:* Receives destination and trip context, returning concise advice on transit, safety, and dining.
- **Wikipedia REST API / Unsplash Image Service** — Resolves dynamic, high-resolution destination cover images on the backend.
  - *Why needed:* Displays authentic imagery for the destination hero banner.
  - *How it supports functionality:* Queries Wikipedia summaries first, then curated fallbacks if API keys are absent.
- **SQLite Database (`GET /api/trips/:id` , `PUT /api/trips/:id` , `DELETE /api/trips/:id`)** — The Express backend queries the database, executing joins across `trips` , `itinerary_days` , `activities` , `budget_breakdown` , and `trip_highlights` .
  - *Why needed:* Retrieves and updates the complete multi-table trip structure.
  - *How it supports functionality:* Populates the entire page with verified relational data.

---

04 — Saved Trips Dashboard

- **Route:** `/dashboard`
- **File Path:** `src/pages/Dashboard.jsx`

Technologies & Role on This Page:

- **Express & SQLite Database (`GET /api/trips` , `DELETE /api/trips/:id`)** — Queries the backend for all saved trips created by the user and executes cascading deletes when a trip is removed.
  - *Why needed:* Provides persistent access to past trips across browser restarts.
  - *How it supports functionality:* Loads trip cards with titles, dates, budgets, and cover images from the database.
- **Web Storage API (`localStorage`)** — Reads the logged-in user profile (`tripwise_user`) and acts as a local fallback if the backend is offline.
  - *Why needed:* Identifies who is currently signed in on the client side.
  - *How it supports functionality:* Filters trips belonging to the logged-in user.
- **React State (`useState` , `useEffect`)** — Manages the trips list, search query filtering, loading skeletons, and trip deletion.
  - *Why needed:* Allows instant in-browser filtering as the user types in the dashboard search bar.
  - *How it supports functionality:* Dynamically filters trips by destination name in real time.
- **React Router DOM (`Link` , `useNavigate`)** — Navigates to `/trip-details?id=...` when clicking any trip card, or to `/generate-trip` if no trips exist.
  - *Why needed:* Seamless transition between the overview dashboard and specific trip details.
- **Lucide React & Tailwind CSS** — Renders responsive trip cards, trash deletion buttons, calendar badges, and empty-state illustrations.
  - *Why needed:* Provides a clean, organized user dashboard layout.

---

05 — User Login

- **Route:** `/login`
- **File Path:** `src/pages/Login.jsx`
- **Components:** `Lsection1.jsx` , `Lsection2.jsx` (`Login.jsx` , `Placeholders.jsx` , `Extra.jsx` , `Continue.jsx`)

Technologies & Role on This Page:

- **React `useState` Hook** — Tracks user email and password inputs, error alerts, and button loading state.
  - *Why needed:* Provides responsive two-way data binding for authentication credentials.
  - *How it supports functionality:* Displays inline error messages if credentials are invalid.
- **Express Backend API (`POST /api/auth/login`)** — Receives credentials and verifies identity on the server.
  - *Why needed:* Keeps sensitive authentication logic off the client browser.
  - *How it supports functionality:* Queries the `users` table in SQLite by email.
- **Node.js `node:crypto` (`scryptSync` & `timingSafeEqual`)** — Server-side cryptography verifying password hashes.
  - *Why needed:* Protects against brute-force attacks and cryptographic timing attacks.

- *How it supports functionality:* Hashes the entered password with the stored salt and compares memory buffers in constant time.

- **Stateless HMAC-SHA256 Token (Custom JWT Implementation)** — Server creates a signed authentication token upon successful login.
  - *Why needed:* Provides secure, stateless session verification without server session memory.
  - *How it supports functionality:* Returned to the client and stored in `localStorage` for verified requests.
- **Web Storage API (`localStorage`)** — Stores `tripwise_user` and `tripwise_token` upon successful login.
  - *Why needed:* Persists the login session across browser tabs and page reloads.
- **React Router DOM (`useNavigate`, `Link`)** — Redirects to `/dashboard` upon successful login, or links to `/signup` if the user needs an account.

---

06 — User Registration (Sign Up)

- **Route:** `/signup`
- **File Path:** `src/pages/Signup.jsx`
- **Components:** `Lsection1.jsx`, Registration Form

Technologies & Role on This Page:

- **Client-Side Form Validation** — Checks that name, email, and password are non-empty, verifies password length (≥ 6 characters), and confirms that `password === confirmPassword`.
  - *Why needed:* Catches common typing mistakes before making network requests.
  - *How it supports functionality:* Prevents invalid registration requests from reaching the server.
- **Express Backend API (`POST /api/auth/signup`)** — Sends registration data to the backend.
  - *Why needed:* Safely creates user accounts on the server.
  - *How it supports functionality:* Checks for duplicate emails (`HTTP 409 Conflict`) and inserts new user rows.
- **Node.js `crypto.randomBytes(16)` & `crypto.scryptSync()`** — Generates a unique 16-byte cryptographic salt and derives a memory-hard password hash.
  - *Why needed:* Ensures no plain-text passwords or vulnerable unsalted hashes are ever stored.
  - *How it supports functionality:* Saves `${salt}:${derivedKey}` in the SQLite `users` table.
- **SQLite Database (`users` Table)** — Stores user records (`id`, `name`, `email`, `password_hash`, `created_at`).
  - *Why needed:* Persistent relational storage for user accounts.
  - *How it supports functionality:* Enforces email uniqueness via `UNIQUE` constraints.
- **Web Storage API (`localStorage`)** — Automatically saves user session data upon signup.
  - *Why needed:* Instantly signs the user in after registration without forcing a second login step.
- **React Router DOM (`useNavigate`)** — Redirects directly to `/dashboard`.

---

07 — 404 Not Found Page

- **Route:** `*` (Catch-All Route)
- **File Path:** `src/pages/NotFound.jsx`
- **Components:** `Navbar.jsx`, `Footer.jsx`

Technologies & Role on This Page:

- **React Router DOM (`path="*"`)** — Catch-all route declaration in `App.jsx`.
  - *Why needed:* Intercepts invalid, broken, or mistyped URLs.
  - *How it supports functionality:* Prevents empty white screens and handles routing errors gracefully.
- **Tailwind CSS & Lucide React (`Compass`, `Home`, `LayoutDashboard`)** — Displays a centered travel-themed error card with icon illustrations.
  - *Why needed:* Maintains professional visual branding even during error states.
  - *How it supports functionality:* Creates an aesthetically pleasing, clear error message.

- **React Router DOM (`Link`)** — Provides two recovery buttons: `"Return Home"` (`/`) and `"My Trips"` (`/dashboard`).
  - *Why needed:* Guides lost users back into active application workflows.
  - *How it supports functionality:* Restores navigation without browser back-button confusion.
- **Backend & Database on this page:** *Not used on this page.*

📊 Summary Table: What Each Technology Powers

Technology	Primarily Powers
React.js 19	Component-based UI rendering, state-driven updates, and Virtual DOM diffing across all 7 pages
Vite 8	Development server, sub-second Hot Module Replacement (HMR), and optimized production Rollup bundling
Express.js / Node.js	Backend REST API (port 5001), request routing, CORS headers, error handling, and static file serving
Google Gemini API	Multi-day travel plan generation, structured JSON itinerary schemas, and AI concierge chatbot
SQLite (`node:sqlite`)	Embedded database persistence across 7 relational tables with ACID transactions, foreign keys, and WAL mode
Leaflet / React-Leaflet	Interactive OpenStreetMap rendering, zoom/pan controls, and destination GPS pin drops on `/trip-details`
Open-Meteo API	Free, keyless 7-day meteorological temperature and precipitation forecasts on `/trip-details`
OpenStreetMap Nominatim	Worldwide debounced city geocoding and coordinate resolution on `/generate-trip`
Recharts	SVG-based responsive budget breakdown donut/pie charts on `/trip-details`
Tailwind CSS v4	Utility-first responsive design, layout styling, and CSS variables across all components
React Router DOM v7	Client-side SPA navigation, query param parsing (`?destination=`, `?id=`), and catch-all 404 routing
Node.js `node:crypto`	Salted `scrypt` password hashing, `timingSafeEqual` comparison, and stateless HMAC-SHA256 JWT tokens
Lucide React	Scalable vector SVG icon components throughout the interface
Web Storage (`localStorage`)	Client-side user session persistence (`tripwise_user`, `tripwise_token`)

🚫 Technologies NOT Used in the Current Implementation

To prevent any confusion or inaccurate claims in a viva, the following technologies are **not used** in TripWise AI:

- **MongoDB / Mongoose:** *Not used.* (TripWise AI uses native relational SQLite for structured data).
- **Third-Party ORMs (Prisma, Sequelize, TypeORM):** *Not used.* (Native prepared statements via `DatabaseSync` are used for zero-dependency simplicity).
- **bcrypt:** *Not used.* (Node's native `node:crypto` with memory-hard `scrypt` is used instead).
- **Third-Party JWT Packages (`jsonwebtoken`):** *Not used.* (A custom, native HMAC-SHA256 implementation is used for zero external package footprint).
- **Google Maps API:** *Not used.* (Leaflet and open-source OpenStreetMap tiles are used to avoid paid API keys).
- **Socket.IO / WebSockets:** *Not used.* (Communication uses standard stateless HTTP REST fetch requests).
- **Cloudinary / AWS S3:** *Not used.* (Images are dynamically resolved via Unsplash and Wikipedia REST APIs).

🔮 Future Scope Technologies (Recommended for Viva Discussion)

If examiners ask what technologies you would introduce next:

1. **WebSockets / Socket.IO:** For real-time group collaboration so multiple friends can edit a trip simultaneously.
2. **Amadeus / Skyscanner APIs:** For real-time live flight price checking and hotel room booking integration.
3. **PWA (Progressive Web App) Service Workers:** For caching itineraries offline so travelers can read plans in flight without internet.
4. **Redis Cache:** For high-speed caching of frequent Gemini trip plans in a multi-server production environment.

PART 04

TripWise AI — Project Architecture & File-by-File Guide

This document explains the purpose, dependencies, internal mechanics, and connections of every key file in the TripWise AI repository.

1. High-Level Folder Structure

```
TRIPWISE/
└── tripwise-ai/
    ├── docs/                      # Project documentation, study guides, and screenshots
    │   ├── screenshots/           # Verified application screenshots (Home, Form, Details, Dashboard)
    │   └── study-guide/           # 17 comprehensive viva and technical study guides
    ├── public/                    # Static assets served directly by Vite
    ├── src/                       # Frontend React source code
    │   ├── assets/                # Local images and branding (hero.png)
    │   ├── components/            # Reusable React UI components
    │   │   ├── Details/           # Trip details page components (Itinerary, Budget, Weather, Map)
    │   │   │   ├── Explore/       # Interactive Leaflet Map component
    │   │   │   ├── LSection/      # Left column (Day-by-day Itinerary accordion)
    │   │   │   └── RSection/      # Right column (BudgetCard, WeatherCard, Highlights, Tips)
    │   │   ├── Tripgenerator/     # Form & CityDropdown components
    │   │   ├── Section1..3/       # Home landing page sections (Hero, Cards, WhyChooseUs)
    │   │   ├── Lsection1..2/      # Alternate landing layout components
    │   │   ├── ErrorBoundary.jsx  # React error boundary catching render exceptions
    │   │   ├── Footer.jsx         # App-wide footer with links
    │   │   ├── Navbar.jsx         # Main navigation bar with auth session awareness
    │   │   └── CuratedTrips.jsx   # Showcase cards for pre-set destinations
    │   ├── data/
    │   │   └── destinations.js    # 178 curated cities across 56 countries dataset & helper functions
    │   ├── pages/                 # React Router page-level route components
    │   │   ├── Home.jsx           # Landing page (/)
    │   │   ├── GenerateTrip.jsx   # Create trip form page (/generate-trip)
    │   │   ├── TripDetails.jsx    # Full itinerary & analytics view (/trip-details)
    │   │   ├── Dashboard.jsx      # User saved trips dashboard (/dashboard)
    │   │   ├── Login.jsx          # Sign in page (/login)
    │   │   ├── Signup.jsx         # Register account page (/signup)
    │   │   └── NotFound.jsx       # 404 catch-all page
    │   ├── services/
    │   │   ├── api.js             # Centralized backend HTTP fetch client (15 REST methods)
    │   │   ├── gemini.js          # Client-side AI fallback helper
    │   │   └── unsplash.js        # Client-side image fallback helper
    │   ├── App.jsx                # Route declarations via React Router DOM v7
    │   ├── App.css                # Component-level styling overrides
    │   ├── index.css              # Global Tailwind CSS v4 imports
    │   └── main.jsx               # Application entrypoint (ReactDOM.createRoot)
    ├── server/                    # Backend Node.js & Express source code
    │   ├── src/
    │   │   ├── data/
    │   │   │   ├── destinations.js # Server copy of the 178-city geocoded dataset
    │   │   │   └── tripwise.db     # SQLite database file (WAL mode)
    │   │   ├── db/
    │   │   │   ├── database.js     # Node native DatabaseSync instance & PRAGMA configurations
    │   │   │   ├── schema.js       # Table definitions (7 tables), indices, and migrations
    │   │   │   ├── repositories.js # CRUD query methods, transactions, and user repos
    │   │   │   └── seed.js         # Initial mock data seeder
    │   │   ├── routes/             # Express REST router modules
    │   │   │   ├── auth.js         # Authentication (/signup, /login, /verify)
    │   │   │   ├── trips.js        # Trip generation, retrieval, update, delete (/api/trips)
    │   │   │   ├── weather.js      # Weather endpoints (/api/weather)
    │   │   │   ├── places.js       # City search, curated & nearby POIs, geocoding (/api/places)
    │   │   │   ├── savedPlaces.js  # Bookmark places endpoints (/api/places/saved)
    │   │   │   └── chat.js         # AI concierge chatbot endpoint (/api/chat)
    │   │   ├── services/           # External API adapters and domain logic
    │   │   │   ├── geminiService.js# Google Generative AI integration + fallback itinerary generator
    │   │   │   ├── weatherService.js# Open-Meteo weather API client + Nominatim geocoder
    │   │   │   ├── placesService.js# Curated attractions & POI catalog
    │   │   │   └── unsplashService.js# Dynamic Wikipedia & Unsplash destination photo fetcher
    │   │   └── index.js            # Express server entry point, middleware, routes, and error handlers
    │   └── package.json            # Backend dependency specifications
    ├── package.json                # Root / Frontend dependency specifications
    ├── vite.config.js              # Vite bundler configuration
    ├── eslint.config.js            # ESLint code quality rules
    ├── .env.example                # Safe environment variable template
    └── .gitignore                  # Git exclusion rules (excluding .env, node_modules, tripwise.db)
```

2. Detailed Breakdown: Frontend Core Files

`src/main.jsx`

- **What it does:** The JavaScript entry point. It finds the `<div id="root">` element in `index.html` and creates a React 19 root using `ReactDOM.createRoot()`, rendering the `<App />` component wrapped in `<React.StrictMode>`.
- **Key imports:** `react`, `react-dom/client`, `./App.jsx`, `./index.css`.

`src/App.jsx`

- **What it does:** Declares the client-side routing structure using `react-router-dom` v7 (`BrowserRouter`, `Routes`, `Route`).
- **Registered Routes:**
  - `/` -> `<Home />`
  - `/login` -> `<Login />`
  - `/signup` -> `<Signup />`
  - `/dashboard` -> `<Dashboard />`
  - `/generate-trip` -> `<GenerateTrip />`
  - `/trip-details` -> `<TripDetails />`
  - `*` -> `<NotFound />` (Graceful 404 handling)

`src/services/api.js`

- **What it does:** Centralized HTTP API client for all frontend communication with the backend.
- **Key Mechanics:**
  - Base URL defaults to `import.meta.env.VITE_API_BASE_URL || "/api"`.
  - Reusable `request(endpoint, options)` wrapper with automatic `Content-Type: application/json` headers, response parsing, and error propagation.
  - **Exported Methods (15 total):**
    - `generateTrip(params)`: `POST /trips/generate`
    - `getTrips(userId)`: `GET /trips?userId=...`
    - `getTripById(id)`: `GET /trips/:id`
    - `saveTrip(data)`: `POST /trips`
    - `updateTrip(id, data)`: `PUT /trips/:id`
    - `deleteTrip(id)`: `DELETE /trips/:id`
    - `signup(data)`: `POST /auth/signup`
    - `login(credentials)`: `POST /auth/login`
    - `getWeather(dest, lat, lon)`: `GET /weather`
    - `getCuratedPlaces(dest, type)`: `GET /places/curated`
    - `getNearbyPlaces(dest)`: `GET /places/nearby`
    - `getGeocode(query)`: `GET /places/geocode`
    - `chatWithAI(payload)`: `POST /chat`
    - `savePlace(data)`, `getSavedPlaces()`, `deleteSavedPlace(id)`: Bookmark operations
    - `getDestinationImage(dest)`: `GET /places/destination-image`
    - `searchCities(q, country)`: `GET /places/search-cities`

`src/data/destinations.js`

- **What it does:** The primary frontend geospatial catalog containing **exactly 178 destinations across 56 countries**.
- **Data Structure:** Each entry contains `city`, `country`, `region`, `countryCode`, and `coordinates: { lat, lon }`.
- **Exported Utilities:**
  - `DESTINATIONS_DATA`: The 178-element array.
  - `COUNTRIES_LIST`: Sorted list of 56 unique countries with total city counts.
  - `filterDestinations({ country, search, region })`: Case-insensitive instant multi-criteria filter.
  - `findDestinationByCity(cityName)`: Normalizes city names and returns exact coordinate matches.

`src/components/Tripgenerator/CityDropdown.jsx`

- **What it does:** The searchable dropdown component providing dual-tier geocoding.
- **Key Mechanics:**
  - **Local Tier:** Filters `DESTINATIONS_DATA` instantly (`< 1ms`) via `useMemo` .
  - **Remote Tier:** A 320ms debounced `useEffect` triggers `searchCities()` against OpenStreetMap Nominatim when the user types a city outside the 178 catalog.
  - **Deduplication:** Merges local and remote results using a JavaScript `Map` keyed by `${city}_${country}` .
  - **Accessibility:** Auto-focuses the search input upon opening and closes on clicks outside via a `useRef` listener.

`src/components/Tripgenerator/Form.jsx`

- **What it does:** The core trip generation form.
- **Key Mechanics:**
  - **Empty by Default:** Every input state (`city` , `destination` , `budget` , `duration` , `selectedStyles`) initializes as empty/blank—no hardcoded default city or values.
  - **Validation:** Enforces non-empty strings, positive numeric budget, duration clamped between 1 and 30 days, and at least one selected travel style chip.
  - **Coordinate Resolution:** Automatically identifies whether the entered destination matches the 178-city catalog or has remote geocodes before submitting to `generateTrip()` .

`src/pages/TripDetails.jsx`

- **What it does:** The comprehensive dashboard rendering a generated or saved trip.
- **Key Mechanics:**
  - Reads `?id=<tripId>` from URL query parameters.
  - Fetches the full trip object from `GET /api/trips/:id` (which joins all 7 database tables).
  - Fetches live 7-day weather via `getWeather(destination, lat, lon)` .
  - Splits UI into a two-column responsive layout:
    - **Left Column:** Day-by-day itinerary accordion with timing, activity cards, and tips.
    - **Right Column:** Stats cards, interactive Leaflet map, weather forecast, budget breakdown charts, and AI travel concierge chat.

---

3. Detailed Breakdown: Backend Core Files

`server/src/index.js`

- **What it does:** Express application entry point.
- **Execution Lifecycle:**
  1. Calls `initSchema()` from `db/schema.js` to create or verify SQLite tables on startup.
  2. Sets up middleware: `cors()` , `express.json()` , and a custom request logger.
  3. Registers REST routers: `/api/auth` , `/api/trips` , `/api/weather` , `/api/places` , `/api/chat` .
  4. Checks if the Vite production build exists in `../../dist` . If present, serves static files and routes all non-API GET requests to `index.html` (SPA fallback).
  5. 404 handlers for missing API endpoints and global error handling middleware.
  6. Starts listening on `PORT` (default `5001`).

`server/src/db/database.js`

- **What it does:** Establishes the connection to the SQLite database.
- **Key Code:**

```
import { DatabaseSync } from "node:sqlite";
export const db = new DatabaseSync(dbPath);
db.exec("PRAGMA foreign_keys = ON;");
db.exec("PRAGMA journal_mode = WAL;");
```

- **Why it matters:** Uses Node 22's native SQLite API. Enables relational foreign key enforcement and Write-Ahead Logging for high concurrency without locks.

`server/src/db/schema.js`

- **What it does:** Creates the relational schema across 7 tables:
  1. `users`: Stores user credentials (`id`, `name`, `email`, `password_hash`, `created_at`).
  2. `trips`: Main trip metadata (`id`, `user_id`, `city`, `destination`, `country`, `coordinates`, `budget`, `duration`, `dates`, `travel_style`, `image_url`).
  3. `itinerary_days`: Individual trip days (`id`, `trip_id`, `day_number`, `title`).
  4. `activities`: Specific scheduled events (`id`, `day_id`, `trip_id`, `time`, `title`, `type`, `location`, `cost`, `rating`, `booking_required`, `ai_recommendation`).
  5. `budget_breakdown`: Itemized expense allocations (`flights`, `hotels`, `food`, `transport`, `activities`, `shopping`).
  6. `trip_highlights`: Summary cards and JSON string arrays for `nearby_places` and `travel_tips`.
  7. `saved_places`: Bookmarked attractions per user and trip.
- Includes B-tree indexes (`idx_trips_user_id`, `idx_itinerary_days_trip_id`, etc.) for fast joins.

`server/src/db/repositories.js`

- **What it does:** The data access layer (DAO / Repository pattern).
- **Core Methods:**
  - `tripsRepo.saveTrip(tripRecord)`: Executes an **atomic transaction** (`BEGIN TRANSACTION ... COMMIT / ROLLBACK`) that inserts data across `trips`, `budget_breakdown`, `trip_highlights`, `itinerary_days`, and `activities`.
  - `tripsRepo.getTripById(id)`: Queries and joins all related records into a single structured JavaScript object.
  - `tripsRepo.getAllTrips(userId)`: Fetches all trips or filters by authenticated user.
  - `usersRepo.createUser()`, `findUserByEmail()`, `findUserById()`: User account management.

`server/src/services/geminiService.js`

- **What it does:** The AI orchestration service.
- **Key Mechanics:**
  - Instantiates `GoogleGenerativeAI` with `process.env.GEMINI_API_KEY`.
  - Executes a multi-model cascade: tries `gemini-1.5-flash`, then `gemini-2.0-flash`, then `gemini-1.5-pro`.
  - Enforces strict JSON output schemas via prompt instructions.
  - **Duplicate Activity Detection:** Iterates through activities with a `Set()`; if the AI repeats activities across days, it rejects the hallucinated schedule and activates the curated fallback engine.
  - **Curated Fallback Engine:** Features authentic multi-day itineraries for major hubs (Seoul, Busan, Tokyo, Kyoto, Paris, London, Athens, New York, Bali, Kerala, Rajasthan, Kashmir, etc.) and procedural generation for custom destinations.

`server/src/services/weatherService.js`

- **What it does:** Weather forecasts and geocoding.
- **Key Mechanics:**
  - Queries the free, open-source **Open-Meteo API** (`https://api.open-meteo.com/v1/forecast`).
  - Returns current temperature, apparent temperature, humidity, wind speed, UV index, and 7-day daily high/low forecasts with WMO weather codes.
  - Includes an internal `KNOWN_COORDINATES` dictionary and an OpenStreetMap Nominatim resolver with 4-second timeout abort controllers.

`server/src/routes/auth.js`

- **What it does:** User registration, login, and token verification.
- **Key Mechanics:**
  - **Password Hashing:** Uses `crypto.randomBytes(16)` for salt and `crypto.scryptSync(password, salt, 64)`.
  - **Timing Defense:** Uses `crypto.timingSafeEqual()` to verify hashes, preventing timing attack vulnerabilities.
  - **Stateless Tokens:** Generates custom signed HMAC-SHA256 tokens with expiration timestamps (`exp`).

PART 05

TripWise AI — Frontend Architecture & React Guide

This guide explains the frontend implementation of TripWise AI. It covers React 19 concepts, Vite tooling, component hierarchies, state management, and React hooks with code references from the actual codebase.

1. Core React Concepts Explained Simply

What is React?

React is a declarative, component-based JavaScript library for building user interfaces. Instead of manually manipulating the browser's DOM (e.g., `document.getElementById('budget').innerText = ...`), you declare what the UI should look like for a given state, and React automatically updates the browser efficiently.

The Virtual DOM & Reconciliation

1. **Virtual DOM (VDOM):** React keeps an in-memory lightweight JavaScript representation of the real DOM elements.
2. **Reconciliation (Diffing):** When component state changes (e.g., the user selects a new travel style), React generates a new Virtual DOM tree, compares it to the previous tree (diffing algorithm), and applies *only the minimal necessary changes* to the actual browser DOM (batching repaints).
3. **Why this matters in TripWise AI:** When a user types into the city search field, the autocomplete list updates dozens of times per second. Because of the Virtual DOM, only the dropdown list items re-render—the rest of the form stays completely intact without flickering.

2. Component Hierarchy Tree

The following diagram illustrates how components are structured and nested throughout the application:

```
[index.html]
   └── [main.jsx] (ReactDOM.createRoot)
         └── <App /> (App.jsx)
               └── <BrowserRouter>
                     └── <Routes>
                           |
                           ├── <Route path="/" element={<Home />} />
                           |     ├── <Navbar />
                           |     ├── <HeroBanner />
                           |     ├── <Cards />
                           |     ├── <WhyChooseUs />
                           |     ├── <CuratedTrips />
                           |     └── <Footer />
                           |
                           ├── <Route path="/generate-trip" element={<GenerateTrip />} />
                           |     ├── <Navbar />
                           |     └── <Tripgenerator />
                           |           ├── <GeneratorHeader />
                           |           └── <Form />
                           |                 └── <CityDropdown /> (x2 for Origin & Destination)
                           |
                           ├── <Route path="/trip-details" element={<TripDetails />} />
                           |     ├── <Navbar />
                           |     ├── <Hero /> (Cover image, title, dates, budget tags)
                           |     ├── Left Column (<LSection />)
                           |     |     └── <Itinerary /> (Day cards, timeline, activity details)
                           |     ├── Right Column (<RSection />)
                           |     |     ├── <StatsCards /> (Duration, activities count, dining count)
                           |     |     ├── <ExploreMapWeather />
                           |     |     |     ├── <InteractiveMap /> (Leaflet map + markers)
                           |     |     |     └── <WeatherCard /> (7-day forecast tabs)
                           |     |     ├── <BudgetCard /> (Recharts visual breakdown)
                           |     |     ├── <TripHighlights /> (Flight, hotel, top-rated tags)
                           |     |     ├── <TravelTips /> (AI concierge bullet recommendations)
                           |     |     └── <NearbyPoints /> (Bookmarked POIs)
                           |     └── <ChatButton /> (Interactive floating AI travel concierge)
                           |
                           ├── <Route path="/dashboard" element={<Dashboard />} />
                           |     ├── <Navbar />
                           |     └── Saved Trips Grid (Card mapping over trips array)
                           |
                           ├── <Route path="/login" element={<Login />} />
                           ├── <Route path="/signup" element={<Signup />} />
                           └── <Route path="*" element={<NotFound />} />
```

3. Props vs State in TripWise AI

Understanding the distinction between **State** and **Props** is a mandatory viva question:

Feature	State (`useState`)	Props (`properties`)
Ownership	Managed internally within the component	Passed down from a parent component
Mutability	Mutable via its dedicated setter (`setX`)	**Immutable / Read-only** for the child
Purpose	Holds dynamic data that changes over time	Configures the child or shares parent data
Project Example	`const [budget, setBudget] = useState("")` inside `Form.jsx`	`<CityDropdown id="dest" label="Destination" onChange={...} />`

Code Example: Parent-to-Child Communication

In `src/components/Tripgenerator/Form.jsx` (Parent):

```
// Form owns the state:
const [destination, setDestination] = useState("");

// Form passes state and updater down to CityDropdown as props:
<CityDropdown
  id="destination-city"
  label="📍 Destination City"
  selectedCity={destination}
  onChange={(val) => setDestination(val)}
  error={!!errorBanner && !destination}
/>
```

In `src/components/Tripgenerator/CityDropdown.jsx` (Child):

```
// CityDropdown receives props and renders according to parent instructions:
const CityDropdown = ({ id, label, selectedCity, onChange, error }) => {
  // Child cannot directly modify selectedCity; it invokes onChange(newCity)
  // to notify the parent to update its state!
};
```

---

4. React Hooks Used in TripWise AI

A. `useState` (State Management)

Declares state variables that trigger a re-render when modified.

Example from `src/components/Tripgenerator/Form.jsx`:

```
const [city, setCity] = useState("");
const [budget, setBudget] = useState("");
const [duration, setDuration] = useState("");
const [selectedStyles, setSelectedStyles] = useState([]);
const [loading, setLoading] = useState(false);
const [errorBanner, setErrorBanner] = useState(null);
```

- When `setLoading(true)` is called, React re-renders the button into a spinning animation.
- When `setErrorBanner("Please enter a budget")` is called, the alert banner appears above the inputs.

B. `useEffect` (Side Effects & Lifecycle)

Performs side effects (data fetching, DOM listeners, timers) in response to component mounting, updating, or unmounting.

Example 1: Outside Click Listener in `CityDropdown.jsx`

```
useEffect(() => {
  const handleClickOutside = (event) => {
    if (dropdownRef.current && !dropdownRef.current.contains(event.target)) {
      setIsOpen(false); // Close dropdown if user clicked outside
    }
  };
  document.addEventListener("mousedown", handleClickOutside);
  // Cleanup function: runs when component unmounts to prevent memory leaks!
  return () => document.removeEventListener("mousedown", handleClickOutside);
}, []); // Empty dependency array = runs ONCE on mount
```

Example 2: Debounced Remote Geocoding Search in `CityDropdown.jsx`

```
useEffect(() => {
  const query = searchQuery.trim();
  if (query.length < 2) return;

  // Set a 320ms debounce timer
  const timer = setTimeout(() => {
    setIsSearchingRemote(true);
    searchCities(query, activeCountry)
      .then((res) => {
        if (res?.success && Array.isArray(res.data)) {
          setRemoteResults(res.data);
        }
      })
      .finally(() => setIsSearchingRemote(false));
  }, 320);

  // If the user types another character before 320ms, cancel the previous timer!
  return () => clearTimeout(timer);
}, [searchQuery, activeCountry]); // Runs whenever search query or country filter changes
```

C. `useRef` (Direct DOM References & Mutable Values)

Holds a persistent reference across renders without triggering a re-render when changed.

Example from ***`CityDropdown.jsx`*** *:*

```
const dropdownRef = useRef(null);
const searchInputRef = useRef(null);

// Focusing the search input box immediately when dropdown opens:
useEffect(() => {
  if (isOpen && searchInputRef.current) {
    setTimeout(() => searchInputRef.current?.focus(), 50);
  }
}, [isOpen]);

// JSX binding:
<div ref={dropdownRef}>
  <input ref={searchInputRef} ... />
</div>
```

D. `useMemo` (Performance Optimization)

Memoizes the calculated return value of an expensive operation so it only recalculates when its dependencies change.

Example from ***`CityDropdown.jsx`*** *:*

```
// Instant local search filter over 178 destinations
const localCities = useMemo(() => {
  return filterDestinations({
    country: activeCountry,
    search: searchQuery,
  });
}, [activeCountry, searchQuery]); // Only recomputes if country or search text changes!
```

Without **`useMemo`**, filtering 178 objects would run on *every single render* of the component (e.g., when other states toggle). **`useMemo`** caches the filtered array.

5. Client-Side Routing with React Router DOM v7

TripWise AI is a **Single Page Application (SPA)**. When navigating from `/generate-trip` to `/trip-details`, the browser does **not** reload the entire HTML page from the server. Instead, React Router intercepts the URL change and swaps the visible page components in place.

Key Router Hooks Used:

1. `useNavigate()`: Programmatic redirection.

```
const navigate = useNavigate();
// Inside Form.jsx after trip generation succeeds:
navigate(`/trip-details?id=${response.data.id}`);
```

2. `useLocation()`: Accessing current route information and query parameters.

```
const location = useLocation();
// Inside TripDetails.jsx to extract the trip ID:
const params = new URLSearchParams(location.search);
const tripId = params.get("id");
```

6. How Data Flows From Form to Details Page

```
1. User enters data in Form.jsx
       |
2. User clicks "Generate Trip" (handleGenerateTrip)
       |
3. Validation checks passed (budget > 0, duration 1–30, styles selected)
       |
4. POST request sent via api.js: generateTrip(params)
       |
5. Backend Express server generates itinerary via Gemini & saves to SQLite
       |
6. Backend responds with: { success: true, data: { id: "trip_1741...", ... } }
       |
7. Frontend receives response: navigate(`/trip-details?id=${res.data.id}`)
       |
8. TripDetails.jsx mounts, reads ?id=trip_1741...
       |
9. TripDetails.jsx executes useEffect:
       ├── getTripById(tripId) -> loads full joined itinerary from SQLite
       └── getWeather(dest, lat, lon) -> loads 7-day forecast from Open-Meteo
       |
10. State populated -> UI renders Itinerary, Leaflet Map, Recharts, Weather Cards
```

PART 06

TripWise AI — Trip Form & Dual-Tier City Search Engine

This document details the architecture, validation mechanics, and dual-tier geocoding search engine powering the "Create Your Perfect Trip" form in TripWise AI.

1. Form Inputs & State Architecture

The trip generation form resides in `src/components/Tripgenerator/Form.jsx`. It manages the following state variables:

Form Field	State Variable	Type	Validation Constraint	Description
Origin City	`city`	`string`	Mandatory, non-empty	Where the traveler is departing from (e.g., *Chandigarh, India*)
Destination	`destination`	`string`	Mandatory, non-empty	Where the traveler wants to visit (e.g., *Athens, Greece*)
Budget	`budget`	`string -> number`	Mandatory, number > 0	Total spending budget in USD
Duration	`duration`	`string -> number`	Mandatory, 1 to 30 days	Length of itinerary
Departure Date	`fromDate`	`string` (YYYY-MM-DD)	Optional	Desired start date
Return Date	`toDate`	`string` (YYYY-MM-DD)	Optional	Desired conclusion date
Travel Styles	`selectedStyles`	`array of strings`	At least 1 selected	6 lifestyle categories
Special Notes	`preferences`	`string`	Optional	Custom user requests (e.g. *vegetarian, kid-friendly*)

The 6 Travel Style Chips

The form includes 6 interactive style cards:

1. 🏖️ **Beach & Relaxation**
2. ⛰️ **Adventure**
3. 🏛️ **Culture**
4. 🍜 **Food & Culinary**
5. 🏙️ **City Exploration**
6. 🦁 **Wildlife**

Users can select multiple styles simultaneously. The selection is toggled via:

```
const handleToggleStyle = (styleName) => {
  setSelectedStyles((prev) =>
    prev.includes(styleName)
      ? prev.filter((s) => s !== styleName)
      : [...prev, styleName]
  );
  setErrorBanner(null);
};
```

2. Empty Initial State Design Principle

In earlier development builds, the form contained hardcoded defaults (such as *Seoul*, *San Francisco*, `budget: 2500`, `duration: 5`). This caused severe usability issues because users accidentally created trips for default cities without noticing.

The Fix: Strict Zero-Default Initialization

In `src/components/Tripgenerator/Form.jsx`, every state is initialized completely blank:

```
const [city, setCity] = useState("");
const [destination, setDestination] = useState(() => {
  // Only pre-populates IF explicitly passed via URL query (e.g. ?destination=Kyoto)
  const params = new URLSearchParams(location.search || window.location.search);
  return params.get("destination") || "";
});
const [budget, setBudget] = useState("");
const [duration, setDuration] = useState("");
const [fromDate, setFromDate] = useState("");
const [toDate, setToDate] = useState("");
const [selectedStyles, setSelectedStyles] = useState([]);
const [preferences, setPreferences] = useState("");
```

- When opening `/generate-trip`, the inputs are 100% clean.
- The user must explicitly choose their origin, destination, budget, duration, and styles.

---

3. Strict Validation & Error Handling

Before dispatching an API request to the backend, `handleGenerateTrip` executes synchronous client-side validation:

```
// 1. Mandatory Text Validations
if (!city.trim()) {
  setErrorBanner("Please enter a Starting City.");
  return;
}
if (!destination.trim()) {
  setErrorBanner("Please enter or select a Destination.");
  return;
}
if (!budget || !budget.trim()) {
  setErrorBanner("Please enter a Total Budget.");
  return;
}
if (!duration || !duration.trim()) {
  setErrorBanner("Please enter a Trip Duration.");
  return;
}
if (selectedStyles.length === 0) {
  setErrorBanner("Please select at least one Travel Style.");
  return;
}

// 2. Numeric Range Validations
const parsedBudget = Number(budget);
const parsedDuration = Number(duration);

if (isNaN(parsedBudget) || parsedBudget <= 0) {
  setErrorBanner("Budget must be a valid positive number.");
  return;
}
if (isNaN(parsedDuration) || parsedDuration <= 0 || parsedDuration > 30) {
  setErrorBanner("Trip duration must be between 1 and 30 days.");
  return;
}
```

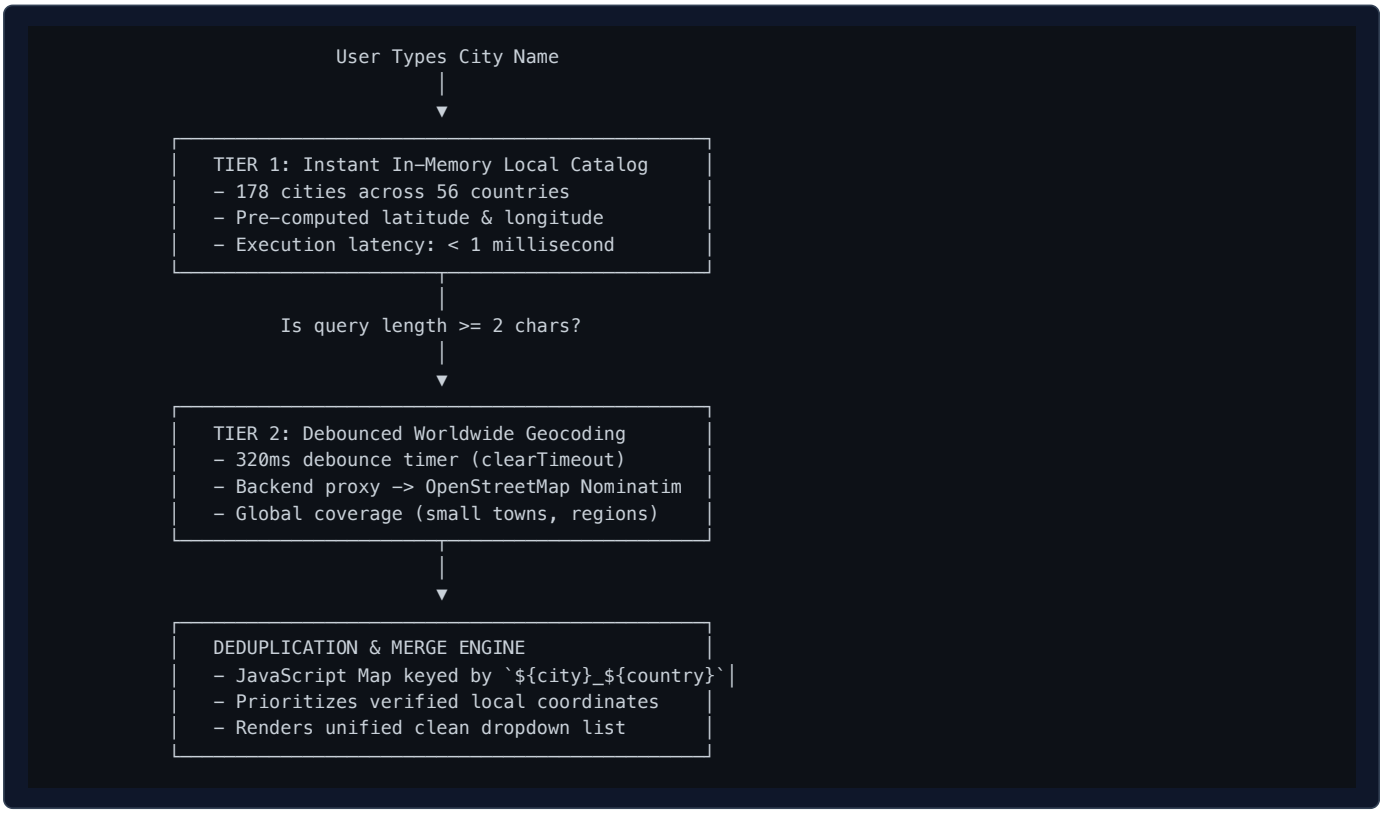
Why Clamp Duration Between 1 and 30 Days?

- Generative AI models have token output limits (e.g., 4096 or 8192 tokens).
- A 30-day itinerary with 4 detailed activities per day produces approximately 6,000 words of structured JSON. Exceeding 30 days would risk token truncation or JSON syntax parsing errors mid-stream.

---

4. The Dual-Tier City Search & Geocoding Engine

TripWise AI solves the classic city search problem (spelling mistakes, unknown coordinates, slow search latency) by combining two complementary systems:



Tier 1: Instant Local Catalog (`src/data/destinations.js`)

- **Size:** Exactly **178 cities across 56 countries**.
- **Execution:** Zero network latency. Searches client memory using JavaScript regex and string matching:

```
export function filterDestinations({ country = "ALL", search = "", region = "ALL" }) {
  const q = search.trim().toLowerCase();
  return DESTINATIONS_DATA.filter((dest) => {
    const matchCountry =
      country === "ALL" ||
      dest.country.toLowerCase() === country.toLowerCase() ||
      dest.countryCode.toLowerCase() === country.toLowerCase();

    const matchSearch =
      !q ||
      dest.city.toLowerCase().includes(q) ||
      dest.country.toLowerCase().includes(q) ||
      dest.region.toLowerCase().includes(q);

    return matchCountry && matchSearch;
  });
}
```

- **Coordinate Guarantee:** Every city has pre-verified GPS coordinates (e.g., Chandigarh: `30.7333, 76.7794`, Athens: `37.9838, 23.7275`).

Tier 2: Debounced Remote Search (OpenStreetMap Nominatim)

If a user searches for a destination not present in the 178-city catalog (e.g., *Gwalior*, *Reykjavik*, *Bhopal*), the second tier activates.

What is Debouncing?

Debouncing is a programming pattern that limits the rate at which a function gets executed. When a user types *"C-h-a-n-d-i-g-a-r-h"*, without debouncing, the app would fire 10 simultaneous HTTP requests to the geocoding server.

With a **320ms debounce**:

1. When "C" is typed, a timer starts for 320ms.
2. If "h" is typed after 100ms, the previous timer is canceled (`clearTimeout`) and a new 320ms timer starts.
3. Only when the user *stops typing for 320ms* does the single HTTP request get sent.

Implementation in `src/components/Tripgenerator/CityDropdown.jsx`:

```
useEffect(() => {
  const query = searchQuery.trim();
  if (query.length < 2) return;

  const timer = setTimeout(() => {
    setIsSearchingRemote(true);
    searchCities(query, activeCountry)
      .then((res) => {
        if (res?.success && Array.isArray(res.data)) {
          setRemoteResults(res.data);
        }
      })
      .catch(() => {
        // Silent catch: Retain local catalog matches if network fails
      })
      .finally(() => setIsSearchingRemote(false));
  }, 320);

  return () => clearTimeout(timer); // Cleanup function resets the timer!
}, [searchQuery, activeCountry]);
```

Deduplication Engine

When merging local results with remote results, duplicate entries could easily appear (for example, if both the local catalog and Nominatim return *"Paris, France"*).

To prevent duplicates, `CityDropdown.jsx` uses an ES6 `Map`:

```
const displayedCities = useMemo(() => {
  const map = new Map();

  // 1. Add instant local matches first
  for (const item of localCities) {
    const key = `${item.city.toLowerCase()}_${item.country.toLowerCase()}`;
    map.set(key, item);
  }

  // 2. Add remote results only if key doesn't already exist
  for (const item of remoteResults) {
    const key = `${item.city.toLowerCase()}_${item.country.toLowerCase()}`;
    if (!map.has(key)) {
      map.set(key, item);
    }
  }

  return Array.from(map.values()).slice(0, 20);
}, [localCities, remoteResults]);
```

---

5. How Coordinates Flow to the Map & Weather

When the user selects a city:

1. `CityDropdown` passes the full item object:

```
{
  city: "Athens",
  country: "Greece",
  coordinates: { lat: 37.9838, lon: 23.7275 }
}
```

2. `Form.jsx` includes these coordinates in the payload to `POST /api/trips/generate`:

```
{
  city: "London",
  destination: "Athens, Greece",
  coordinates: { lat: 37.9838, lon: 23.7275 },
  budget: 1800,
  duration: 5,
  travelStyle: "Culture, City Exploration"
}
```

3. The backend stores `coordinates` in the SQLite `trips` table as a JSON string: `{"lat":37.9838,"lon":23.7275}`.
4. When `TripDetails.jsx` loads, it passes these exact coordinates to:
   - `<InteractiveMap coordinates={trip.coordinates} />` to center the Leaflet map and drop the destination marker.
   - `getWeather(trip.destination, trip.coordinates.lat, trip.coordinates.lon)` to fetch accurate Open-Meteo forecasts.

PART 07

TripWise AI — Backend Architecture & Express Guide

This document covers the backend architecture of TripWise AI. It explains Express.js 4, the middleware pipeline, request-response lifecycles, HTTP status codes, and provides a complete REST API reference table.

1. Backend Architecture Overview

TripWise AI's backend is a modular Node.js REST API service built with **Express 4.21.2** running on port `5001`.



2. Express Middleware Pipeline Explained

What is Middleware?

Middleware functions are functions that have access to the request object (`req`), the response object (`res`), and the `next` middleware function in the application's request-response cycle.

In `server/src/index.js`, middleware executes in the exact order declared:

1. `app.use(cors());`

- **What it does:** Enables Cross-Origin Resource Sharing.
- **Why it is necessary:** The frontend runs on `http://localhost:5173` (Vite) while the backend runs on `http://localhost:5001` (Express). Web browsers enforce the **Same-Origin Policy (SOP)** by default, blocking scripts on port 5173 from reading responses from port 5001.

`cors()` injects headers like `Access-Control-Allow-Origin: *` to allow safe cross-origin communication.

2. `app.use(express.json());`

- **What it does:** Built-in body parser.
- **Why it is necessary:** When the client sends `POST /api/trips/generate` with a JSON payload, the HTTP request delivers the data as a raw data stream. `express.json()` reads the stream, parses the JSON string, and assigns the resulting JavaScript object to `req.body`. Without this, `req.body` would be `undefined`.

3. Request Logger

```
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`);
  next(); // Hands control to the next middleware or route handler!
});
```

- Every incoming request outputs a clean log (e.g. `[2026-09-08T05:00:00.000Z] POST /api/trips/generate`). Calling `next()` is critical; omitting `next()` would leave the request hanging indefinitely.

4. Global Error Handling Middleware

```
app.use((err, req, res, next) => {
  void next;
  console.error("Unhandled Error:", err.stack || err.message);
  res.status(err.status || 500).json({
    success: false,
    error: err.message || "Internal Server Error",
  });
});
```

- Defined with **four parameters**: `(err, req, res, next)`.
- If any asynchronous route handler catches an exception and calls `next(error)`, Express bypasses all remaining route handlers and jumps straight to this error handler, returning a consistent JSON error structure.

---

3. HTTP Status Codes Used in TripWise AI

Examiners frequently ask candidates to explain which HTTP status codes their API returns and why:

Status Code	Name	Where Used in TripWise AI	Reason
200	**OK**	`GET /api/trips`, `GET /api/weather`, `POST /api/auth/login`	Request succeeded and data is returned
201	**Created**	`POST /api/trips/generate`, `POST /api/auth/signup`	A new resource was successfully created in the SQLite database
400	**Bad Request**	Missing destination, negative budget, duration > 30	Client failed to provide valid required parameters
401	**Unauthorized**	Wrong password on login, invalid or expired JWT token	Authentication credentials missing or invalid
404	**Not Found**	`GET /api/trips/:id` for non-existent ID, unknown route	The requested entity or API route does not exist
409	**Conflict**	`POST /api/auth/signup` with an already registered email	Attempting to create duplicate unique entity
500	**Internal Server Error**	Database disk failure, unhandled runtime exception	Server-side malfunction

---

4. Complete REST API Reference Table

The following table documents every API endpoint available in the TripWise AI backend:

Method	Endpoint Path	Query / Body Parameters	Response Structure	Status Code
`GET`	`/api/health`	None	`{ status: "ok", service: "TripWise AI Backend", uptime, timestamp }`	`200`
`POST`	`/api/auth/signup`	Body: `{ name, email, password }`	`{ success: true, token, data: { id, name, email } }`	`201` / `400` / `409`
`POST`	`/api/auth/login`	Body: `{ email, password }`	`{ success: true, token, data: { id, name, email } }`	`200` / `400` / `401`
`GET`	`/api/auth/verify`	Header: `Authorization: Bearer <token>`	`{ success: true, data: { id, name, email } }`	`200` / `401`
`POST`	`/api/trips/generate`	Body: `{ city, destination, budget, duration, travelStyle, ... }`	`{ success: true, data: <full_joined_trip_record> }`	`201` / `400` / `500`
`GET`	`/api/trips`	Query: `?userId=<id>` (optional)	`{ success: true, count: N, data: [ ...trips ] }`	`200`
`GET`	`/api/trips/:id`	Route Param: `:id`	`{ success: true, data: <trip_with_days_and_activities> }`	`200` / `404`
`PUT`	`/api/trips/:id`	Route Param: `:id`, Body: `{ tripUpdates }`	`{ success: true, data: <updated_trip> }`	`200` / `404`
`DELETE`	`/api/trips/:id`	Route Param: `:id`	`{ success: true, message: "Trip deleted successfully." }`	`200` / `404`
`GET`	`/api/weather`	Query: `?destination=Paris&lat=48.85&lon=2.35`	`{ destination, coordinates, current: {...}, forecast: [7 days] }`	`200`
`GET`	`/api/places/search-cities`	Query: `?q=Athens&country=Greece`	`{ success: true, source: "local" | "remote", data: [ ...cities ] }`	
`GET`	`/api/places/curated`	Query: `?destination=Kyoto&type=tourist_attraction`	`{ success: true, data: [ ...curatedAttractions ] }`	`200`
`GET`	`/api/places/nearby`	Query: `?destination=Kyoto`	`{ success: true, data: [ ...nearbyPOIs ] }`	`200`
`GET`	`/api/places/geocode`	Query: `?query=Kyoto`	`{ lat: 35.0116, lon: 135.7681, displayName: "Kyoto, Japan" }`	`200` / `404`
`GET`	`/api/places/destination-image`	Query: `?destination=Kyoto`	`{ success: true, imageUrl: "https://images.unsplash..." }`	`200`
`POST`	`/api/places/saved`	Body: `{ tripId, userId, name, category, rating, address, imageUrl }`	`{ success: true, data: <saved_place_record> }`	`201`
`GET`	`/api/places/saved`	Query: `?tripId=<id>&userId=<id>`	`{ success: true, data: [ ...bookmarkedPlaces ] }`	`200`
`DELETE`	`/api/places/saved/:id`	Route Param: `:id`	`{ success: true, message: "Place removed from saved." }`	`200`
`POST`	`/api/chat`	Body: `{ destination, tripContext, question }`	`{ success: true, answer: "concierge response text" }`	`200`

PART 08

TripWise AI — Google Gemini AI Integration & Fallback Engine

This document details how Google Gemini Generative AI is integrated into TripWise AI. It explores prompt engineering, schema enforcement, duplicate detection, model cascades, and the curated fallback engine.

1. Integration Architecture & Model Cascade

TripWise AI uses the official `@google/generative-ai` SDK (`GoogleGenerativeAI`) on the backend in `server/src/services/geminiService.js`.

Why Call Gemini from the Backend Instead of the Frontend?

1. **API Key Security:** If Gemini were called directly from React in the browser, the secret `GEMINI_API_KEY` would be exposed in the browser's Network inspector or client bundle. Calling it from Express keeps the key completely safe on the server.
2. **Server-Side Validation & Sanitization:** The backend can inspect the AI's response, validate its schema, deduplicate activities, and run the fallback engine before persisting to the database.

The Multi-Model Fallback Cascade

In `generateItinerary()`, the service does not rely on a single model endpoint. It iterates through a prioritized candidate list:

```
const candidateModels = [
  "gemini-1.5-flash",  // Tier 1: Fastest response (<1s), lowest latency, high JSON fidelity
  "gemini-2.0-flash",  // Tier 2: Next-gen flash model for fallback
  "gemini-1.5-pro"     // Tier 3: High reasoning capability for complex prompts
];

for (const modelName of candidateModels) {
  try {
    const model = genAI.getGenerativeModel({ model: modelName });
    const result = await model.generateContent(prompt);
    // ... parse and validate ...
  } catch (err) {
    console.warn(`Gemini generation with ${modelName} failed:`, err.message);
  }
}
```

If one model experiences temporary rate limits or regional maintenance, the loop automatically attempts the next candidate.

2. Prompt Engineering & Strict Schema Enforcement

LLMs naturally tend to produce conversational text (e.g. *"Here is your 5-day itinerary for Athens: ..."*). However, our React application requires structured data to render cards, charts, and maps.

The System Prompt Architecture

The prompt in `geminiService.js` incorporates strict prompt engineering techniques:

```
You are an expert, realistic, and highly knowledgeable travel planner.
Generate a realistic, non-repeating day-by-day travel itinerary for:
Destination: ${destination}
Duration: ${duration} Days
Total Budget: $${budget} USD
Travel Style: ${travelStyle}
Preferences: ${preferences.join(", ")}

STRICT REQUIREMENTS:
1. OUTPUT FORMAT: Return ONLY a valid, parseable JSON object.
   DO NOT include any markdown fences (like ```json), DO NOT include any explanatory text before or after the JSON.
2. NON-REPETITION: Every day MUST feature completely unique, realistic landmarks, activities, and dining spots. DO
NOT repeat the same attraction across different days.
3. BUDGET CONSISTENCY: The sum of numbers in budgetBreakdown must equal approximately ${budget}.
4. DAYS COUNT: You MUST return exactly ${duration} day objects in the "days" array.

JSON SCHEMA:
{
  "tripHighlights": {
    "flight": "Flight from origin to destination",
    "hotel": "Recommended accommodation name & area",
    "topRated": "Most iconic landmark in destination",
    "activities": number,
    "restaurants": number,
    "nearbyPlaces": ["Place 1", "Place 2", "Place 3"]
  },
  "budgetBreakdown": {
    "flights": number,
    "hotels": number,
    "food": number,
    "transport": number,
    "activities": number,
    "shopping": number
  },
  "travelTips": [
    { "category": "Timing" | "Food" | "Transport" | "Photography", "title": "...", "description": "..." }
  ],
  "days": [
    {
      "day": 1,
      "title": "Day 1 Theme Title",
      "activities": [
        {
          "time": "09:00 AM",
          "title": "Unique Activity Name",
          "type": "Sightseeing" | "Dining" | "Hotel" | "Transport" | "Activity",
          "location": "Specific Location Name",
          "description": "Short description (1-2 sentences)",
          "duration": "2 Hours",
          "cost": "$25",
          "rating": 4.8,
          "bookingRequired": false,
          "bestTime": "Morning",
          "additionalDetails": "Helpful notes",
          "aiRecommendation": "Pro travel tip"
        }
      ]
    }
  ]
}
```

3. Parsing & Markdown Sanitization

Even when instructed not to output markdown, LLMs often wrap responses in markdown code fences (```` ```json ... ``` ````).

To ensure bulletproof parsing, `geminiService.js` performs a two-step sanitization:

```
const text = await result.response.text();

// Strip markdown fences and whitespace
const cleaned = text
  .replace(/```json/gi, "")
  .replace(/```/g, "")
  .trim();

const parsed = JSON.parse(cleaned);
```

4. Hallucination & Duplicate Activity Detection

A common flaw in AI-generated travel itineraries is **activity repetition** (e.g., suggesting the Eiffel Tower on Day 1, Day 3, and Day 5).

TripWise AI implements an automated **Duplicate Activity Detector** using a JavaScript `Set`:

```
const seenTitles = new Set();
let hasDuplicates = false;

for (const day of parsed.days) {
  if (!day.activities) continue;
  for (const act of day.activities) {
    if (seenTitles.has(act.title)) {
      hasDuplicates = true;
      break;
    }
    seenTitles.add(act.title);
  }
  if (hasDuplicates) break;
}

if (!hasDuplicates) {
  console.log(`Gemini successfully generated non-repeating itinerary using ${modelName}`);
  return parsed;
} else {
  console.log(`Gemini generated repeating activities across days; falling back to curated non-repeating engine.`);
  return generateFallbackItinerary(destination, duration, budget, travelStyle, preferences);
}
```

If the AI hallucinates repeating activities, the response is discarded and the reliable curated engine steps in.

5. The Curated Fallback Itinerary Engine

A vital architectural strength of TripWise AI is its **Zero-Failure Guarantee**. Even if:

- No `GEMINI_API_KEY` is provided in `.env`
- The user runs out of API quota
- The internet goes offline during an examiner demo
- Gemini returns malformed JSON or repeating activities

The application **never crashes**. It transparently generates a 100% realistic itinerary via `generateFallbackItinerary()`.

A. Authentic Curated Hubs (`CURATED_DESTINATION_ITINERARIES`)

The backend maintains pre-computed, verified multi-day itineraries for major destinations:

- **Seoul & Busan (South Korea):** Gyeongbokgung Palace, Tosokchon Samgyetang, Bukchon Hanok Village, Haeundae Beach, Gamcheon Culture Village.
- **Tokyo & Kyoto (Japan):** Senso-ji, Fushimi Inari-taisha, Tsukiji Market, Arashiyama Bamboo Grove.
- **Athens (Greece):** Acropolis, Parthenon, Plaka District, Ancient Agora, Temple of Olympian Zeus.

- **Paris, London, New York, Bali, Rome, Dubai, Sydney, Switzerland, and 15+ Indian states** (Kerala, Kashmir, Rajasthan, Himachal Pradesh, etc.).

B. Procedural Engine for Custom Cities

If a user creates a trip for an uncurated city (e.g., *Chandigarh* or *Gwalior*), the fallback engine dynamically synthesizes a realistic plan using category templates (Heritage morning walk, iconic regional dining, botanical gardens, sunset panorama viewpoints) ensuring no duplicate titles and adjusting budget allocations mathematically:

```
budgetBreakdown: {
  flights: Math.round(budget * 0.35),
  hotels: Math.round(budget * 0.30),
  food: Math.round(budget * 0.15),
  transport: Math.round(budget * 0.08),
  activities: Math.round(budget * 0.08),
  shopping: Math.round(budget * 0.04),
}
```

---

6. The Interactive AI Travel Concierge (`/api/chat`)

TripWise AI includes a floating interactive concierge on the trip details page (`src/components/Details/ChatButton.jsx`).

- **Endpoint:** `POST /api/chat`
- **Request Payload:**

```
{
  "destination": "Athens, Greece",
  "tripContext": "5-day trip, budget $1800, Culture & City Exploration",
  "question": "What is the best way to get from Athens airport to Syntagma Square?"
}
```

- **Backend Function (`chatAboutTrip`):** Uses `gemini-1.5-flash` with a focused conversational prompt.
- **Fallback Response:** If the API key is unavailable, it provides practical local travel tips (e.g. metro advice, comfortable walking shoes, currency recommendations) instead of failing.

PART 09

TripWise AI — Database Architecture & SQLite Guide

This document details the database engine, relational schema, ACID transactions, performance indexing, and configuration pragma settings powering TripWise AI.

---

1. Database Technology: Native `node:sqlite`

TripWise AI uses Node.js's built-in `node:sqlite` module via the `DatabaseSync` class (available in Node.js v22.5.0+).

```
import { DatabaseSync } from "node:sqlite";
export const db = new DatabaseSync(dbPath);
```

Why Native `node:sqlite` Over External Packages?

1. **Zero npm Dependencies:** Does not require third-party packages like `sqlite3` or `better-sqlite3`.
2. **Zero Native Build Failures:** Compiling native C/C++ bindings via `node-gyp` often fails on different operating systems (Windows Visual Studio Build Tools, macOS Xcode Command Line Tools, Linux python dependencies). Native `node:sqlite` works instantly across all platforms.
3. **Embedded & Serverless:** The database is stored in a single binary file on disk (`server/src/data/tripwise.db`). No database servers, background services, or network authentication ports are required.
4. **Synchronous Prepared Statements:** Prepared statements run in-process with nanosecond execution speed.

---

2. Essential SQLite PRAGMA Configurations

When the database initializes in `server/src/db/database.js`, two critical PRAGMA commands are executed:

A. `PRAGMA foreign_keys = ON;`

- **What it does:** Enables foreign key constraint checking.
- **Why it matters:** In SQLite, foreign key enforcement is disabled by default for backwards compatibility. Enabling this ensures that:
  - Records cannot reference non-existent parent IDs.
  - Cascading deletes (`ON DELETE CASCADE`) work automatically. If a trip is deleted, all its days, activities, budget items, and highlights are purged instantly.

B. `PRAGMA journal_mode = WAL;`

- **What it does:** Switches the database journaling mode to **Write-Ahead Logging (WAL)**.
- **How WAL Works:**
  - In traditional rollback journaling, when a write operation occurs, the entire database file is locked. Readers must wait until the write finishes.
  - In **WAL mode**, changes are appended to a separate log file (`tripwise.db-wal`). Readers can continue reading uncommitted snapshots from the main database file simultaneously!
- **Result:** Drastically higher concurrency and zero "Database is locked" errors.

---

3. The 7 Relational Tables & Schema Details

The schema is created in `server/src/db/schema.js` :

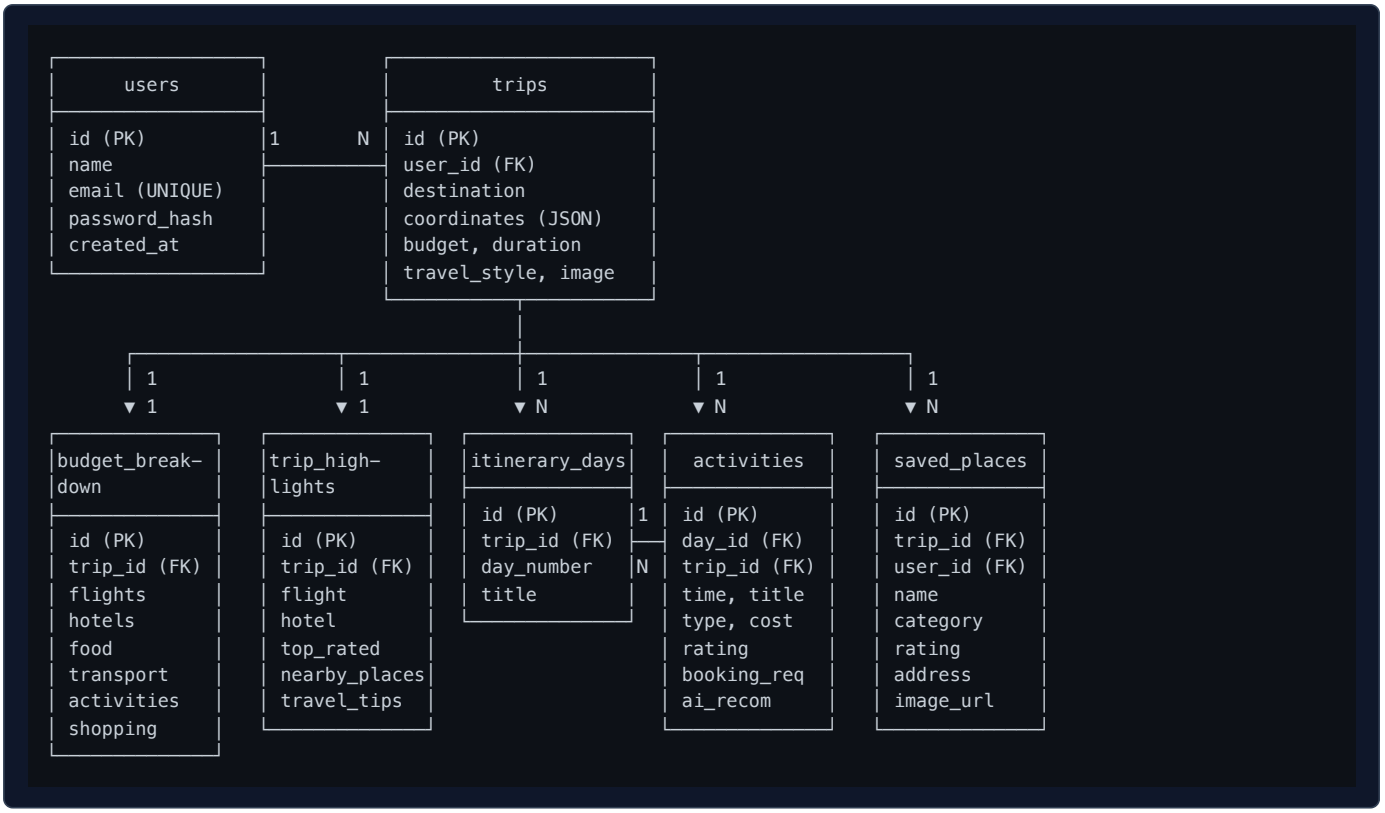


Table Field Descriptions

1. `users`

- `id` (TEXT, PRIMARY KEY): Unique identifier (e.g. `usr_1741...`).
- `name` (TEXT NOT NULL): User's full display name.
- `email` (TEXT UNIQUE NOT NULL): Login email.
- `password_hash` (TEXT NOT NULL): Salted scrypt key (`<salt>:<derivedKey>`).
- `created_at` (DATETIME): Registration timestamp.

2. `trips`

- `id` (TEXT, PRIMARY KEY): Trip identifier (e.g. `trip_1741...`).
- `user_id` (TEXT, FK -> `users(id)` ON DELETE SET NULL): Owner user ID. Allows trips to exist for guest users.
- `city` (TEXT NOT NULL): Origin city.
- `destination` (TEXT NOT NULL): Target destination.
- `country` (TEXT): Country name.
- `coordinates` (TEXT): JSON string of latitude and longitude (`{"lat":37.98,"lon":23.72}`).
- `budget` (REAL NOT NULL): Total trip budget in USD.
- `duration` (INTEGER NOT NULL): Duration in days.
- `from_date`, `to_date` (TEXT): Scheduled calendar travel dates.
- `travel_style` (TEXT NOT NULL): Selected style chips (e.g. *"Culture, Adventure"*).
- `preferences` (TEXT): Custom notes.
- `image_url` (TEXT): Destination cover photo URL.
- `created_at`, `updated_at` (DATETIME).

3. `itinerary_days`

- `id` (TEXT, PRIMARY KEY): Day ID (`day_1741...`).
- `trip_id` (TEXT NOT NULL, FK -> `trips(id)` ON DELETE CASCADE).
- `day_number` (INTEGER NOT NULL): 1, 2, 3, etc.
- `title` (TEXT NOT NULL): Theme title (e.g. *"Historic Palaces & Bukchon Enclave"*).

4. `activities`

- `id` (TEXT, PRIMARY KEY): Activity ID (`act_1741...`).
- `day_id` (TEXT NOT NULL, FK -> `itinerary_days(id)` ON DELETE CASCADE).
- `trip_id` (TEXT NOT NULL, FK -> `trips(id)` ON DELETE CASCADE).
- `time` (TEXT): Scheduled time (e.g. *"09:00 AM"*).
- `title` (TEXT NOT NULL): Landmark or activity name.
- `type` (TEXT): *"Sightseeing"*, *"Dining"*, *"Hotel"*, *"Transport"*.
- `location` (TEXT): Specific neighborhood or address.
- `description` (TEXT): 1-2 sentence overview.
- `duration` (TEXT): e.g. *"2.5 Hours"*.
- `cost` (TEXT): Estimated cost (e.g. *"$15"* or *"Free"*).
- `rating` (REAL): Visitor rating (e.g. `4.8`).
- `booking_required` (INTEGER): `1` if reservation required, `0` if walk-in.
- `best_time` (TEXT): *"Morning"*, *"Afternoon"*, *"Sunset"*, *"Evening"*.
- `additional_details` (TEXT): Practical tips (e.g. *"Free entry if wearing Hanbok"*).
- `ai_recommendation` (TEXT): Expert AI tip.

5. `budget_breakdown`

- Stores exact dollar allocations for `flights`, `hotels`, `food`, `transport`, `activities`, and `shopping`. Linked 1-to-1 to `trips(id)` with a UNIQUE constraint.

6. `trip_highlights`

- Stores summary tags (`flight`, `hotel`, `top_rated`, `activities_count`, `restaurants_count`) plus JSON strings for `nearby_places` and `travel_tips`.

7. `saved_places`

- Stores user-bookmarked attractions and restaurants per trip.

---

4. B-Tree Indexes for Performance

To ensure millisecond response times when querying trips with hundreds of activities, `schema.js` defines indexes on all foreign key columns:

```
CREATE INDEX IF NOT EXISTS idx_trips_user_id ON trips(user_id);
CREATE INDEX IF NOT EXISTS idx_itinerary_days_trip_id ON itinerary_days(trip_id);
CREATE INDEX IF NOT EXISTS idx_activities_day_id ON activities(day_id);
CREATE INDEX IF NOT EXISTS idx_activities_trip_id ON activities(trip_id);
CREATE INDEX IF NOT EXISTS idx_saved_places_trip_id ON saved_places(trip_id);
CREATE INDEX IF NOT EXISTS idx_saved_places_user_id ON saved_places(user_id);
```

Without indexes, SQLite would have to perform full table scans across all activities for every single day lookup. Indexes reduce lookups to logarithmic time complexity **O(log N)**.

---

5. ACID Transactions in Action

In `server/src/db/repositories.js`, saving a generated trip involves writing to **5 separate tables**. If an error occurred halfway through (e.g., the server lost power on Day 3's activities), the database would end up with orphaned, corrupted trip data.

To prevent this, `saveTrip()` wraps all operations in an **atomic ACID transaction**:

```
// 1. Begin atomic transaction
db.exec("BEGIN TRANSACTION;");

try {
  // Clear existing records if updating/overwriting
  db.prepare("DELETE FROM trips WHERE id = ?").run(tripId);

  // Insert into trips
  insertTripStmt.run(...);

  // Insert budget breakdown
  insertBudgetStmt.run(...);

  // Insert highlights and travel tips
  insertHighlightsStmt.run(...);

  // Insert days and nested activities
  for (const day of days) {
    insertDayStmt.run(dayId, tripId, day.day, day.title);
    for (const act of day.activities) {
      insertActStmt.run(actId, dayId, tripId, ...);
    }
  }

  // 2. Commit transaction: all changes permanently written together!
  db.exec("COMMIT;");
  return this.getTripById(tripId);

} catch (err) {
  // 3. Rollback: if ANY step failed, revert everything!
  db.exec("ROLLBACK;");
  console.error("[DB] Transaction failed while saving trip:", err);
  throw err;
}
```

Explaining ACID to Examiners:

- **Atomicity:** All-or-nothing. Either all 5 tables are updated, or none are.
- **Consistency:** Database transitions only between valid states conforming to schema constraints.
- **Isolation:** Concurrent reads see only fully committed trips, never partial states.
- **Durability:** Once committed, records survive server crashes because changes are safely persisted to disk.

PART 10

TripWise AI — External API Integrations Guide

This document provides a comprehensive technical analysis of every third-party API used in TripWise AI. It details request parameters, response structures, authentication mechanisms, and fallback safeguards.

1. External APIs Summary Table

Service / API	Provider	Purpose	Authentication	Cost / Quota	Offline / Failure Safeguard
Google Gemini API	Google AI Studio	Generates structured multi-day travel itineraries and concierge chat	API Key (`GEMINI_API_KEY`) in `.env`	Generous free tier (15 RPM)	Multi-model cascade (`1.5-flash` , `2.0-flash` , `1.5-pro`) + Curated Fallback Engine
Open-Meteo API	Open-Meteo GmbH	7-day meteorological weather forecast & current conditions	**None (No API key needed!)**	100% Free & Open-Source (up to 10,000 calls/day)	`getFallbackWeather()` mock generator
Nominatim API	OpenStreetMap Foundation	Worldwide city geocoding & coordinate resolution	**None** (Requires custom `User-Agent` header)	Free (Max 1 req/sec policy)	In-memory `Map` cache + `KNOWN_COORDINATES` dictionary
Wikipedia REST API	Wikimedia Foundation	Dynamic destination cover photo retrieval	**None** (Requires custom `User-Agent` header)	100% Free & Open	Curated `CITY_FALLBACKS` dictionary with Unsplash photos
Unsplash API *(Optional)*	Unsplash Inc.	High-resolution photography search	Client ID (`UNSPLASH_ACCESS_KEY`)	50 reqs/hr (Free tier)	Automatically falls back to Wikipedia & curated photo bank

2. Open-Meteo Weather API Integration

Why Open-Meteo?

Most commercial weather providers (OpenWeatherMap, WeatherAPI) require paid subscriptions or credit-card registration. **Open-Meteo** is an open-source weather API providing global forecasts using high-resolution weather models (ECMWF, GFS, DWD ICON) without requiring any API keys.

Implementation in `server/src/services/weatherService.js`:

```
const weatherUrl = `https://api.open-meteo.com/v1/forecast?latitude=${lat}&longitude=${lon}&current=temperature_2m,relative_humidity_2m,apparent_temperature,wind_speed_10m,weather_code,uv_index&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max&forecast_days=7&timezone=auto`;

const response = await fetch(weatherUrl);
const weatherData = await response.json();
```

Data Transformations:

Open-Meteo returns parallel arrays for daily metrics (e.g. `time: ["2026-09-08", ...], temperature_2m_max: [28.4, ...]`). The service reshapes this into a clean array of day objects for the React frontend:

```
const days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
const formattedForecast = weatherData.daily.time.map((dateStr, index) => ({
  date: dateStr,
  day: days[new Date(dateStr).getDay()],
  high: Math.round(weatherData.daily.temperature_2m_max[index]),
  low: Math.round(weatherData.daily.temperature_2m_min[index]),
  weatherCode: weatherData.daily.weather_code[index],
  precipitation: weatherData.daily.precipitation_probability_max[index] || 0,
}));
```

Understanding WMO Weather Codes:

The `weatherCode` integer maps to standard World Meteorological Organization weather types:

- `0` : Clear Sky ☀️
- `1, 2, 3` : Mainly Clear / Partly Cloudy ⛅
- `45, 48` : Fog 🌫️
- `51, 53, 55` : Drizzle 🌦️
- `61, 63, 65` : Rain 🌧️
- `71, 73, 75` : Snow ❄️
- `95, 96` : Thunderstorm ⛈️

---

3. OpenStreetMap Nominatim Geocoding API

What is Geocoding?

Geocoding is the computational process of converting human-readable address names (e.g., *"Athens, Greece"*) into geographic coordinates (Latitude: `37.9838`, Longitude: `23.7275`).

Implementation in `server/src/routes/places.js`:

```
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 2800); // 2.8s strict timeout

const response = await fetch(
  `https://nominatim.openstreetmap.org/search?format=json&addressdetails=1&limit=8&q=${encodeURIComponent(searchSt
r)}`,
  {
    headers: {
      "User-Agent": "TripWiseAI/1.0 (city-search-service)",
      Accept: "application/json",
    },
    signal: controller.signal,
  }
);
clearTimeout(timeoutId);
```

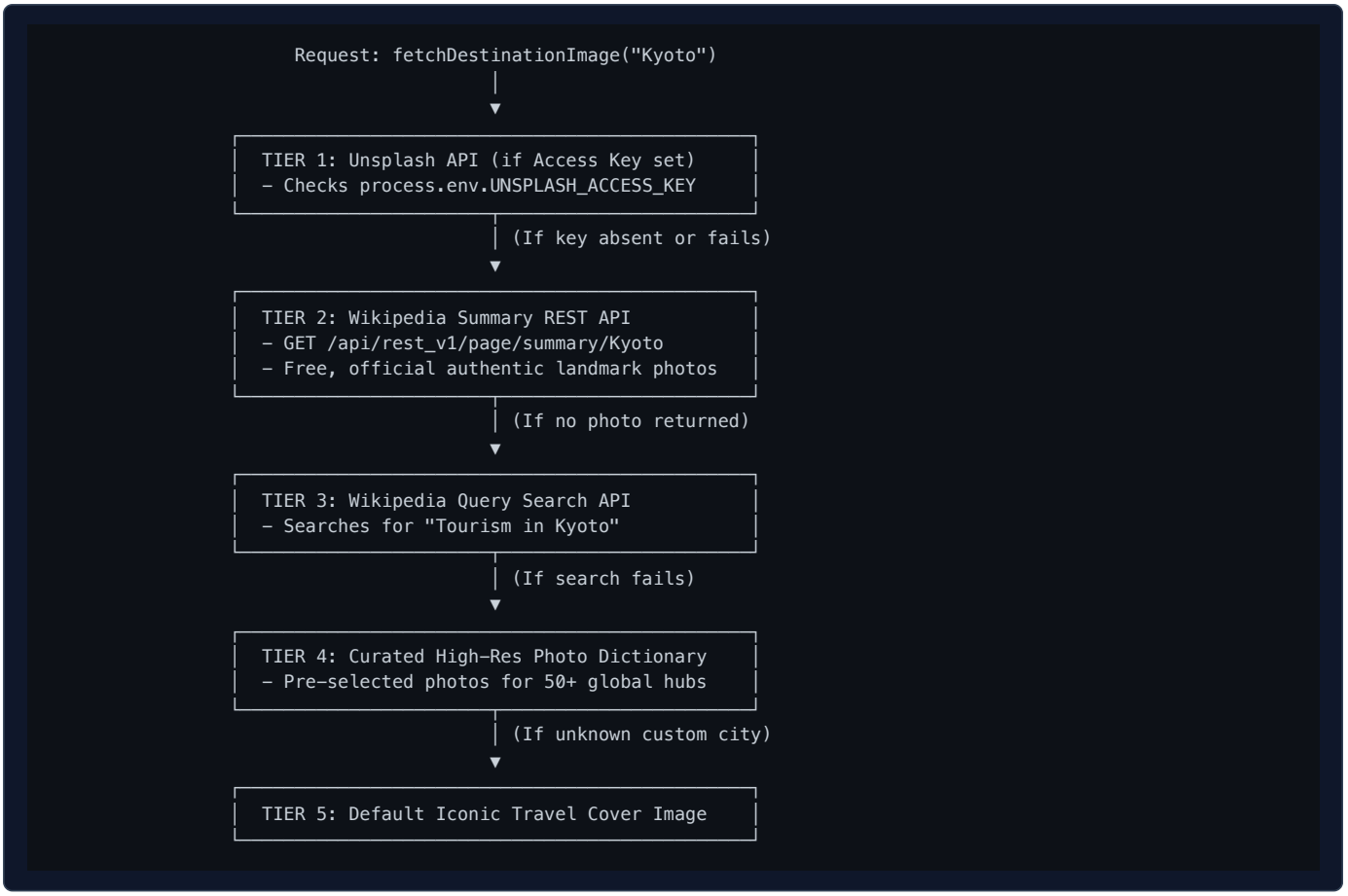
Critical Implementation Details:

1. **User-Agent Requirement:** The OpenStreetMap Nominatim Acceptable Use Policy strictly forbids requests without a valid, descriptive `User-Agent`. Requests sent with generic curl/fetch headers receive an HTTP 403 Forbidden. TripWise AI supplies `"TripWiseAI/1.0"`.
2. **AbortController Timeout:** If Nominatim's public server experiences latency, the `AbortController` terminates the connection after 2.8 seconds, preventing the user's dropdown from freezing.
3. **In-Memory Caching:** Both `weatherService.js` and `places.js` store past search queries in an in-memory `Map` (`citySearchCache`). Repeated queries for *"Athens"* return in 0 milliseconds without querying Nominatim again.

---

4. Destination Cover Image Resolution Pipeline

Destination cover photos are dynamically resolved through a 5-tier fallback cascade in `server/src/services/unsplashService.js`:



Why Query Wikipedia Before Generic Stock Photos?

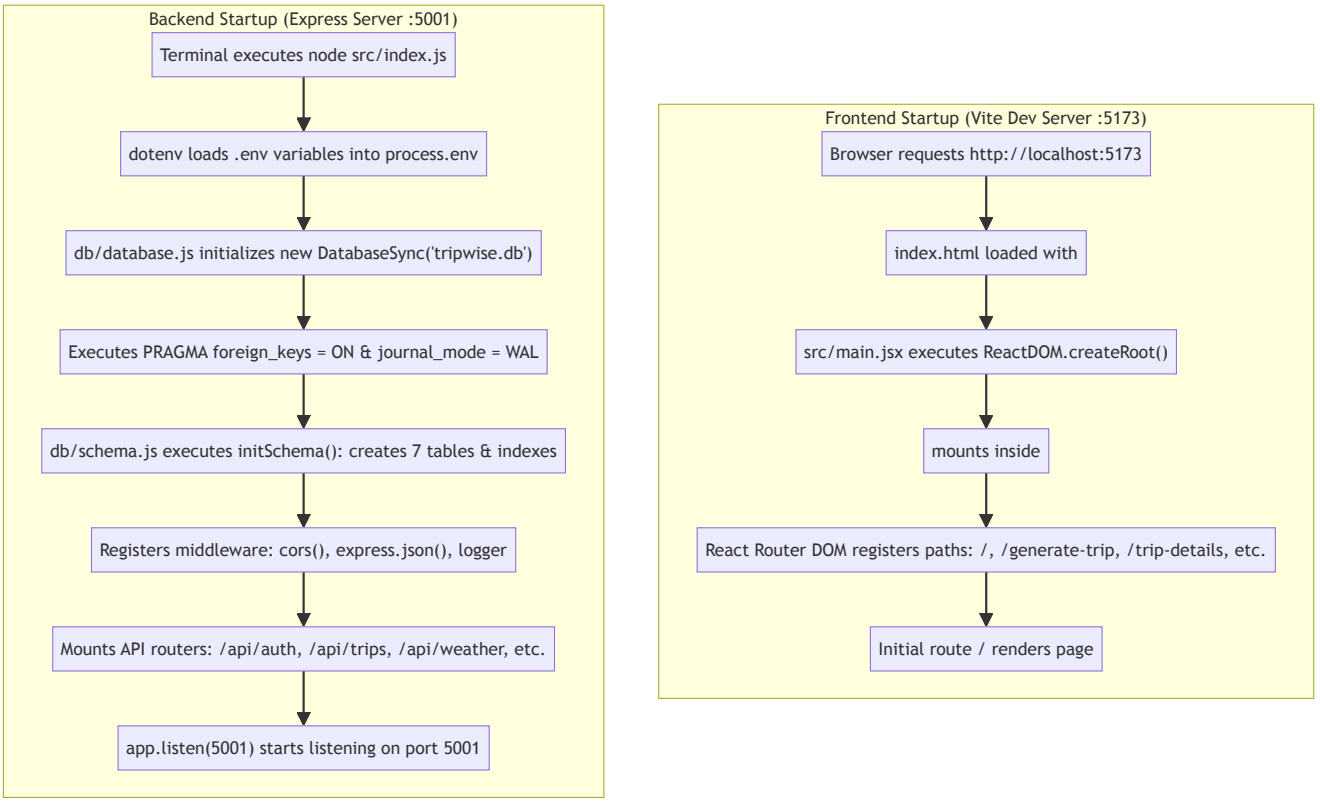
Wikipedia's REST API (`https://en.wikipedia.org/api/rest_v1/page/summary/`) provides authentic, copyright-cleared, high-resolution photographs directly representing the actual destination (not generic unrelated stock photos). It requires no API keys, has high availability, and supports virtually every city on Earth.

PART 11

TripWise AI — System Flowcharts & Sequence Diagrams

This document visualizes the complete architectural flows of TripWise AI using Mermaid diagrams. Each diagram is accompanied by a step-by-step technical explanation.

1. Application Startup & Initialization Flow



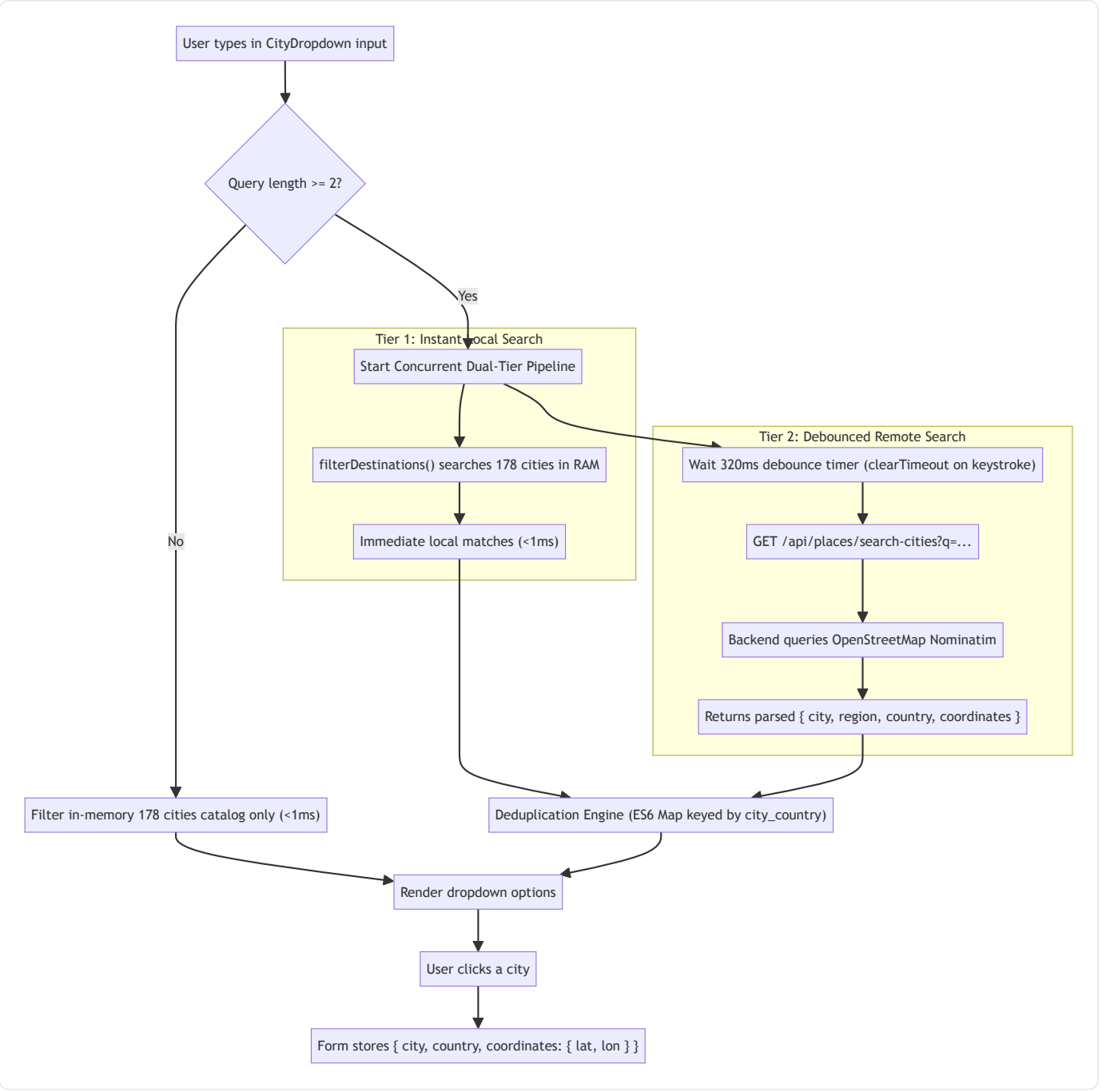
Explanation:

- **Frontend:** Vite serves ES modules directly to the browser. React mounts into the `#root` container, initializes routing, and renders the `<Home />` page.
- **Backend:** Express imports `node:sqlite`, connects to `tripwise.db`, enables Write-Ahead Logging (`WAL`), and verifies table schemas before listening on port 5001.

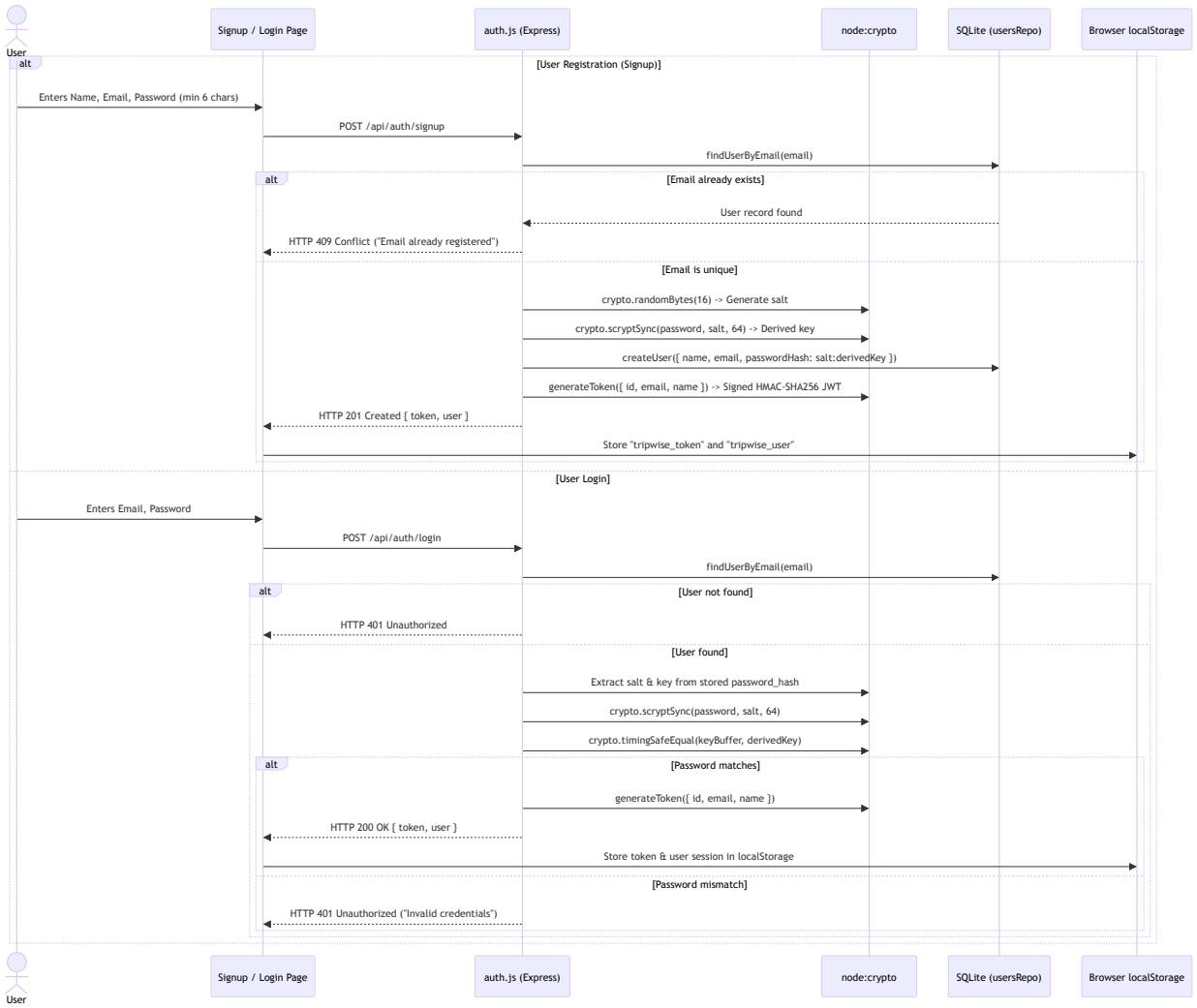
2. End-to-End Trip Generation Sequence



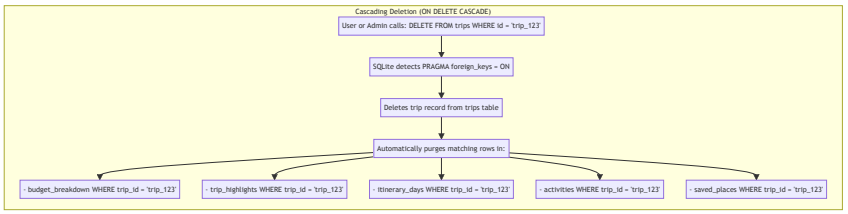
3. Dual-Tier City Search & Geocoding Flow

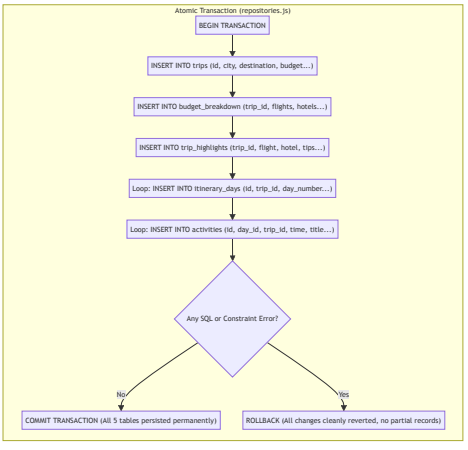


4. User Authentication & Session Security Flow



5. Database Persistence & Foreign Key Cascade Flow





PART 12

TripWise AI — Core Code Walkthrough & Line-by-Line Guide

This document breaks down the most important code snippets across the frontend and backend of TripWise AI. Every function, parameter, and algorithmic design choice is explained line-by-line so you can confidently explain the code during an interview.

1. Frontend: `src/components/Tripgenerator/Form.jsx`

Snippet 1: Initial State (Zero Hardcoded Defaults)

```jsx
// Lines 11–30 in Form.jsx
const [city, setCity] = useState("");
const [destination, setDestination] = useState(() => {
  const params = new URLSearchParams(location.search || window.location.search);
  return params.get("destination") || "";
});
const [budget, setBudget] = useState("");
const [duration, setDuration] = useState("");
const [fromDate, setFromDate] = useState("");
const [toDate, setToDate] = useState("");
const [selectedStyles, setSelectedStyles] = useState([]);
const [preferences, setPreferences] = useState("");
const [loading, setLoading] = useState(false);
const [errorBanner, setErrorBanner] = useState(null);
```

- **Line-by-line explanation:**
  - `const [city, setCity] = useState("")`: Departure city defaults to an empty string.
  - `const [destination, setDestination] = useState(() => ...)`: Uses a **lazy initializer function**. It checks if the URL has a `?destination=` query parameter (e.g. if the user clicked a destination card from the landing page). If present, it pre-fills that destination; otherwise, it initializes as blank.
  - `budget`, `duration`, `fromDate`, `toDate`: All initialized as empty strings so input placeholders are clearly visible.
  - `selectedStyles`: Initialized as an empty array `[]` requiring the user to choose at least one category chip.
  - `loading`: Boolean state controlling the spinning loader on the submit button.
  - `errorBanner`: Holds user-facing validation error strings or `null` if the form is valid.

Snippet 2: Validation & Submission Handler

```
// Lines 118–160 in Form.jsx
const handleGenerateTrip = async () => {
  setErrorBanner(null);

  if (!city.trim()) {
    setErrorBanner("Please enter a Starting City.");
    return;
  }
  if (!destination.trim()) {
    setErrorBanner("Please enter or select a Destination.");
    return;
  }
  if (!budget || !budget.trim()) {
    setErrorBanner("Please enter a Total Budget.");
    return;
  }
  if (!duration || !duration.trim()) {
    setErrorBanner("Please enter a Trip Duration.");
    return;
  }
  if (selectedStyles.length === 0) {
    setErrorBanner("Please select at least one Travel Style.");
    return;
  }

  const parsedBudget = Number(budget);
  const parsedDuration = Number(duration);

  if (isNaN(parsedBudget) || parsedBudget <= 0) {
    setErrorBanner("Budget must be a valid positive number.");
    return;
  }
  if (isNaN(parsedDuration) || parsedDuration <= 0 || parsedDuration > 30) {
    setErrorBanner("Trip duration must be between 1 and 30 days.");
    return;
  }

  setLoading(true);
  // ... proceed to API request
```

- **Line-by-line explanation:**
  - `setErrorBanner(null)` : Clears any previous error messages before re-evaluating.
  - `!city.trim()` , `!destination.trim()` : Strips whitespace to ensure the user didn't just enter spaces.
  - `selectedStyles.length === 0` : Ensures the user picked at least one category chip.
  - `Number(budget)` / `Number(duration)` : Converts strings to numbers.
  - `isNaN(...) || parsedBudget <= 0` : Guards against strings like `"abc"` or negative budgets like `"-500"` .
  - `parsedDuration > 30` : Enforces a 30-day upper bound to prevent LLM token limits and JSON truncation.
  - `setLoading(true)` : Disables the form button and shows a progress spinner.

---

2. Frontend: `src/components/Tripgenerator/CityDropdown.jsx`

Snippet: Debounced Geocoding & Local In-Memory Filtering

```jsx
// Lines 65–96 in CityDropdown.jsx

// 1. Instant local search (<1ms)
const localCities = useMemo(() => {
  return filterDestinations({
    country: activeCountry,
    search: searchQuery,
  });
}, [activeCountry, searchQuery]);

// 2. Debounced remote geocoding search for worldwide coverage
useEffect(() => {
  const query = searchQuery.trim();
  if (query.length < 2) {
    return;
  }

  const timer = setTimeout(() => {
    setIsSearchingRemote(true);
    searchCities(query, activeCountry)
      .then((res) => {
        if (res && res.success && Array.isArray(res.data)) {
          setRemoteResults(res.data);
        }
      })
      .catch(() => {
        // Retain local matches if remote search fails
      })
      .finally(() => {
        setIsSearchingRemote(false);
      });
  }, 320);

  return () => clearTimeout(timer);
}, [searchQuery, activeCountry]);
```

- **Line-by-line explanation:**
  - `useMemo(() => ..., [activeCountry, searchQuery])`: Filters the 178-city catalog in RAM. Because it is wrapped in `useMemo`, it only re-runs when the search text or selected country changes.
  - `if (query.length < 2) return`: Prevents firing network requests for single characters like "a".
  - `setTimeout(..., 320)`: Delays the remote API call by 320 milliseconds.
  - `return () => clearTimeout(timer)`: **Cleanup function**. If the user types another letter within 320ms, React cancels the previous timer and starts a fresh one. This eliminates unnecessary network traffic.
  - `setIsSearchingRemote(true/false)`: Toggles the subtle loading spinner inside the search input.

3. Backend: `server/src/services/geminiService.js`

Snippet: AI Model Cascade, Markdown Stripping & Duplicate Detection

```
// Lines 580–628 in geminiService.js
const candidateModels = ["gemini-1.5-flash", "gemini-2.0-flash", "gemini-1.5-pro"];

for (const modelName of candidateModels) {
  try {
    console.log(`Calling Gemini with model: ${modelName}...`);
    const model = genAI.getGenerativeModel({ model: modelName });
    const result = await model.generateContent(prompt);
    const text = await result.response.text();

    const cleaned = text
      .replace(/```json/gi, "")
      .replace(/```/g, "")
      .trim();

    const parsed = JSON.parse(cleaned);

    if (parsed.days && Array.isArray(parsed.days) && parsed.days.length > 0 && parsed.budgetBreakdown) {
      // Validate non-repetition across days
      const seenTitles = new Set();
      let hasDuplicates = false;

      for (const day of parsed.days) {
        if (!day.activities) continue;
        for (const act of day.activities) {
          if (seenTitles.has(act.title)) {
            hasDuplicates = true;
            break;
          }
          seenTitles.add(act.title);
        }
        if (hasDuplicates) break;
      }

      if (!hasDuplicates) {
        console.log(`Gemini successfully generated non-repeating itinerary using ${modelName}`);
        return parsed;
      } else {
        console.log(`Gemini generated repeating activities across days; falling back to curated non-repeating engin
e.`);
        return generateFallbackItinerary(destination, duration, budget, travelStyle, preferences);
      }
    }
  } catch (err) {
    console.warn(`Gemini generation with ${modelName} failed:`, err.message);
  }
}

console.log("All Gemini attempts failed; generating realistic fallback itinerary.");
return generateFallbackItinerary(destination, duration, budget, travelStyle, preferences);
```

- **Line-by-line explanation:**
  - `candidateModels`: Prioritized array of Gemini model names to try in sequence.
  - `cleaned = text.replace(...)`: Uses regex to strip markdown code blocks like ```` ```json ```` and ```` ``` ```` which LLMs frequently append despite prompt instructions.
  - `const parsed = JSON.parse(cleaned)`: Parses the cleaned string into a JavaScript object.
  - `const seenTitles = new Set()`: Creates an ES6 Set to track activity titles across the entire trip.
  - `if (seenTitles.has(act.title))`: If an activity name was already used on an earlier day, `hasDuplicates` is set to `true`.
  - `if (!hasDuplicates) return parsed`: Returns the validated AI plan.
  - `else ... return generateFallbackItinerary(...)`: Rejects repetitive hallucinations and uses the curated engine.
  - `catch (err)`: Logs errors (e.g. quota limit, network timeout) and gracefully falls through to the next model or fallback generator.

4. Backend: `server/src/db/repositories.js`

Snippet: Atomic Transaction for Multi-Table Persistence

```
// Lines 127–250 in repositories.js
db.exec("BEGIN TRANSACTION;");
try {
  // 1. Delete if exists (for upsert or overwrite)
  db.prepare("DELETE FROM trips WHERE id = ?").run(tripId);

  // 2. Insert into trips
  const insertTripStmt = db.prepare(`
    INSERT INTO trips (
      id, user_id, city, destination, country, coordinates, budget, duration,
      from_date, to_date, travel_style, preferences, image_url,
      created_at, updated_at
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)
  `);
  insertTripStmt.run(tripId, userId, city, destination, country, coordinatesStr, budget, duration, fromDate, toDat
e, travelStyle, preferencesStr, imageUrl);

  // 3. Insert budget breakdown
  const insertBudgetStmt = db.prepare(`
    INSERT INTO budget_breakdown (id, trip_id, flights, hotels, food, transport, activities, shopping)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?)
  `);
  insertBudgetStmt.run(generateId("bb"), tripId, budgetBreakdown.flights, budgetBreakdown.hotels, budgetBreakdown.f
ood, budgetBreakdown.transport, budgetBreakdown.activities, budgetBreakdown.shopping);

  // 4. Insert trip highlights
  const insertHighlightsStmt = db.prepare(`
    INSERT INTO trip_highlights (id, trip_id, flight, hotel, top_rated, activities_count, restaurants_count, nearby
_places, travel_tips)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
  `);
  insertHighlightsStmt.run(generateId("th"), tripId, tripHighlights.flight, tripHighlights.hotel, tripHighlights.to
pRated, tripHighlights.activities, tripHighlights.restaurants, JSON.stringify(tripHighlights.nearbyPlaces), JSON.st
ringify(travelTipsToSave));

  // 5. Insert days and activities
  for (let i = 0; i < days.length; i++) {
    const day = days[i];
    const dayId = generateId("day");
    insertDayStmt.run(dayId, tripId, day.day || i + 1, day.title || `Day ${i + 1}`);

    for (const act of day.activities) {
      insertActStmt.run(generateId("act"), dayId, tripId, act.time, act.title, act.type, act.location, act.descript
ion, act.duration, act.cost, act.rating, act.bookingRequired ? 1 : 0, act.bestTime, act.additionalDetails, act.aiRe
commendation);
    }
  }

  // 6. Commit transaction
  db.exec("COMMIT;");
  return this.getTripById(tripId);
} catch (err) {
  // 7. Rollback on failure
  db.exec("ROLLBACK;");
  console.error("[DB] Transaction failed while saving trip:", err);
  throw err;
}
```

- **Line-by-line explanation:**
  - `db.exec("BEGIN TRANSACTION;")`: Begins a SQLite transaction block.
  - `db.prepare(...).run(...)`: Uses **parameterized queries** (`?` placeholders). This guarantees protection against **SQL Injection attacks** because parameters are treated strictly as data literals, never executable SQL commands.
  - `JSON.stringify(...)`: Encodes arrays (`nearbyPlaces`, `travelTips`) into JSON strings before storing in SQLite TEXT columns.

- `db.exec("COMMIT;")`: Persists all 5 tables atomically.
- `catch (err) { db.exec("ROLLBACK;"); throw err; }`: If any error occurs anywhere in the process, every single write is completely undone.

---

5. Backend: `server/src/routes/auth.js`

Snippet: Password Hashing, Scrypt, and Timing Attack Defense

```
// Lines 67–84 in auth.js
function hashPassword(password) {
  const salt = crypto.randomBytes(16).toString("hex");
  const derivedKey = crypto.scryptSync(password, salt, 64).toString("hex");
  return `${salt}:${derivedKey}`;
}

function verifyPassword(password, storedHash) {
  if (!storedHash) return false;
  if (storedHash.includes(":")) {
    const [salt, key] = storedHash.split(":");
    const keyBuffer = Buffer.from(key, "hex");
    const derivedKey = crypto.scryptSync(password, salt, 64);
    return crypto.timingSafeEqual(keyBuffer, derivedKey);
  }
  const sha = crypto.createHash("sha256").update(password).digest("hex");
  return storedHash === sha || storedHash === password;
}
```

- **Line-by-line explanation:**
  - `crypto.randomBytes(16)`: Generates 16 cryptographically secure random bytes to serve as a unique **salt**. The salt prevents attackers from using precomputed **Rainbow Tables** to crack passwords.
  - `crypto.scryptSync(password, salt, 64)`: Uses the **scrypt** key derivation function. Scrypt is memory-hard and computationally intensive, making brute-force attacks via ASICs or GPUs computationally unfeasible.
  - `${salt}:${derivedKey}`: Combines the unique salt and hash with a colon separator for clean storage.
  - `crypto.timingSafeEqual(keyBuffer, derivedKey)`: Standard string comparison (`===`) terminates as soon as the first mismatched byte is encountered, allowing attackers to measure execution time down to nanoseconds to deduce password bytes (a **timing attack**). `timingSafeEqual` takes constant time regardless of where or if a difference occurs, eliminating timing vulnerabilities.

PART 13

TripWise AI — Technical Glossary & Conceptual Guide

This glossary defines over 60 essential web engineering, React, backend, database, AI, and cybersecurity terms used in TripWise AI. Each term includes a simple real-world analogy, an exact technical definition, and where it is used in the project.

Part 1: Frontend & React Architecture

1. SPA (Single Page Application)

- **Simple Analogy:** A dynamic TV screen where the channel picture changes smoothly without turning the physical TV off and on again.
- **Technical Definition:** A web app that loads a single HTML file (`index.html`) on the initial request and dynamically updates the DOM via JavaScript as the user navigates, eliminating full page reloads.
- **In TripWise AI:** Navigation between `/generate-trip`, `/trip-details`, and `/dashboard` happens instantly via React Router without page refreshes.

2. Virtual DOM (VDOM)

- **Simple Analogy:** A rough architect's sketch. The architect experiments and modifies the sketch quickly, only calling the construction crew when final changes are ready.
- **Technical Definition:** An in-memory JavaScript tree representing the real DOM nodes. React computes differences between VDOM snapshots and applies batched updates to the real browser DOM.
- **In TripWise AI:** Allows smooth keystroke typing in `CityDropdown` while filtering 178 cities without UI stutter.

3. Reconciliation & Diffing

- **Simple Analogy:** Spot-the-difference game between two identical photos.
- **Technical Definition:** React's heuristic **O(N)** diffing algorithm that compares an older Virtual DOM snapshot with a newer one to determine the minimal set of DOM operations required.
- **In TripWise AI:** When a user checks a travel style chip, only that specific chip re-renders; the rest of the form is untouched.

4. JSX (JavaScript XML)

- **Simple Analogy:** Writing HTML inside JavaScript with superpower brackets `{}`.
- **Technical Definition:** A syntactic extension for JavaScript that allows developers to write HTML-like markup directly inside JS files, transpiled into `React.createElement()` calls.
- **In TripWise AI:** All components (`Form.jsx`, `Itinerary.jsx`) use JSX.

5. Props (Properties)

- **Simple Analogy:** Instructions or ingredients passed from a chef (parent) to an assistant (child).
- **Technical Definition:** Read-only inputs passed from a parent component to a child component to configure its behavior and appearance.
- **In TripWise AI:** `<CityDropdown id="dest" selectedCity={destination} onChange={setDestination} />`.

6. State

- **Simple Analogy:** A person's memory of their current mood or current task, which changes over time.
- **Technical Definition:** An internal, mutable JavaScript object managed within a component that stores data that can change and trigger a UI re-render when updated.
- **In TripWise AI:** `const [budget, setBudget] = useState("")` in `Form.jsx`.

7. Prop Drilling

- **Simple Analogy:** Passing a bucket of water down a human chain of 10 people just to deliver it to the last person.
- **Technical Definition:** The practice of passing props through several intermediate components that do not need the data themselves, solely to reach a deeply nested child.
- **In TripWise AI:** Handled cleanly in `TripDetails.jsx` by grouping related child components into `<LSection />` and `<RSection />`.

8. React Hooks

- **Simple Analogy:** Modular plug-in power tools that give functional components superpowers.
- **Technical Definition:** Special functions starting with `use` (e.g., `useState`, `useEffect`) that let functional components hook into React state and lifecycle features.
- **In TripWise AI:** Used throughout: `useState`, `useEffect`, `useRef`, `useMemo`, `useNavigate`, `useLocation`.

9. `useState`

- **Technical Definition:** A hook that declares a state variable and a setter function, returning a tuple `[state, setState]`.
- **In TripWise AI:** Manages form inputs (`city`, `budget`, `duration`, `loading`, `errorBanner`).

10. `useEffect`

- **Technical Definition:** A hook that executes side-effects (data fetching, DOM listeners, timers) after rendering. Accepts a callback and a dependency array.
- **In TripWise AI:** Handles outside click listeners to close `CityDropdown` and the 320ms debounced search timer.

11. `useRef`

- **Technical Definition:** Returns a mutable ref object with a `.current` property that persists across renders without causing a re-render when modified. Also used for direct DOM access.
- **In TripWise AI:** `searchInputRef.current.focus()` auto-focuses the input when opening the city dropdown.

12. `useMemo`

- **Technical Definition:** Caches the result of an expensive calculation between renders, only recalculating when designated dependencies change.
- **In TripWise AI:** Caches the instant local filtering of 178 cities in `CityDropdown.jsx`.

13. React StrictMode

- **Technical Definition:** A development-only wrapper (`<React.StrictMode>`) that intentionally double-invokes component renders and effects to surface side-effect bugs and deprecated API usage.
- **In TripWise AI:** Configured in `src/main.jsx`.

14. Vite & Hot Module Replacement (HMR)

- **Simple Analogy:** Swapping a burnt-out lightbulb in a house while the party inside keeps going without interruption.
- **Technical Definition:** Vite replaces, adds, or removes individual modules at runtime without reloading the entire browser page or losing application state.
- **In TripWise AI:** Changes saved to `Form.jsx` appear on screen in milliseconds without clearing already-typed form inputs.

15. Tailwind CSS Utility-First Styling

- **Technical Definition:** A CSS framework offering low-level utility classes (`flex`, `p-4`, `bg-blue-600`, `rounded-xl`) directly inside JSX instead of writing custom CSS rules.
- **In TripWise AI:** Used across all components with zero external CSS files needed.

Part 2: Backend & Networking

16. REST (Representational State Transfer)

- **Technical Definition:** An architectural style for network communication that uses stateless HTTP requests, standard URIs (`/api/trips`), and standard HTTP methods (`GET`, `POST`, `PUT`, `DELETE`).
- **In TripWise AI:** All backend routes in `server/src/routes/` follow REST conventions.

17. HTTP Status Codes

- **Technical Definition:** Standardized 3-digit integer responses issued by a server to indicate the outcome of a client request:
  - `200 OK`: Request succeeded.
  - `201 Created`: Resource successfully created (used in `POST /api/trips/generate`).
  - `400 Bad Request`: Validation failure (missing parameters, negative budget).

- `401 Unauthorized`: Invalid credentials or missing JWT.
- `404 Not Found`: Entity does not exist.
- `409 Conflict`: Attempting to register an email that already exists.
- `500 Internal Server Error`: Unhandled server exception.

18. Middleware

- **Simple Analogy:** Security guards and baggage scanners at an airport checkpoint that inspect everyone before they board the plane.
- **Technical Definition:** Functions in Express that sit between the raw incoming HTTP request and the final route handler, having access to `req`, `res`, and `next()`.
- **In TripWise AI:** `cors()`, `express.json()`, and the global error handler in `server/src/index.js`.

19. CORS (Cross-Origin Resource Sharing)

- **Simple Analogy:** An apartment gatekeeper checking if a visitor from a different street has permission to enter.
- **Technical Definition:** A browser security mechanism that uses HTTP headers (`Access-Control-Allow-Origin`) to tell browsers whether requests originating from a different origin (e.g. port 5173 to port 5001) are permitted.
- **In TripWise AI:** Enabled via `app.use(cors())` so Vite frontend can talk to Express backend.

20. Same-Origin Policy (SOP)

- **Technical Definition:** A fundamental web security policy implemented in browsers that restricts scripts on one origin (scheme + domain + port) from interacting with resources on a different origin.

21. Debouncing

- **Simple Analogy:** An elevator door sensor that waits 5 seconds after the last person steps inside before closing the door.
- **Technical Definition:** A programming practice that postpones function execution until a predetermined duration has elapsed since the last event invocation.
- **In TripWise AI:** 320ms debounce in `CityDropdown.jsx` to prevent firing HTTP requests to Nominatim on every keystroke.

22. AbortController

- **Technical Definition:** A built-in JavaScript object that allows you to abort one or more Web fetch requests on demand or via a timer timeout.
- **In TripWise AI:** Sets a 2.8s - 4.0s timeout on external geocoding requests to prevent frozen server threads.

Part 3: Database & Relational Architecture

23. Embedded / Serverless Database

- **Simple Analogy:** Carrying your notebook in your backpack versus having to phone a central library in another town every time you need to read a page.
- **Technical Definition:** A database engine that runs in-process inside the host application and stores data directly in a local file, without running a separate database server process.
- **In TripWise AI:** Native `node:sqlite` stores everything in `server/src/data/tripwise.db`.

24. Database Normalization

- **Technical Definition:** Organizing a database into structured tables to reduce data redundancy and improve data integrity (1NF, 2NF, 3NF).
- **In TripWise AI:** 7 separate tables (`users`, `trips`, `itinerary_days`, `activities`, `budget_breakdown`, `trip_highlights`, `saved_places`) rather than dumping one huge unstructured blob.

25. Primary Key (PK) & Foreign Key (FK)

- **Technical Definition:**
  - **PK:** A unique identifier for every row in a table (`id`).
  - **FK:** A column in one table that points to the Primary Key of another table, creating a relational link (`trip_id` in `activities` points to `trips.id`).

26. `ON DELETE CASCADE`

- **Technical Definition:** A foreign key constraint rule that automatically deletes all child rows whenever the referenced parent row is deleted.
- **In TripWise AI:** Deleting a trip from `trips` automatically deletes its rows in `activities`, `itinerary_days`, `budget_breakdown`, and `trip_highlights`.

27. B-Tree Index

- **Simple Analogy:** The index at the back of a textbook that lets you jump straight to page 142 instead of reading every page from page 1.
- **Technical Definition:** A self-balancing search tree data structure that allows database queries to find matching records in **O(log N)** time rather than scanning every row (**O(N)**).
- **In TripWise AI:** Indexes on `idx_trips_user_id`, `idx_activities_day_id`, etc.

28. ACID Properties

- **Technical Definition:** The four core guarantees of reliable database transactions:
  - **Atomicity:** All operations in the transaction succeed, or all are rolled back.
  - **Consistency:** Data must conform to all declared constraints and rules.
  - **Isolation:** Concurrent transactions do not interfere with each other.
  - **Durability:** Committed data is permanently written to non-volatile storage.

29. Database Transaction (`BEGIN`, `COMMIT`, `ROLLBACK`)

- **In TripWise AI:** `saveTrip()` executes `BEGIN TRANSACTION;`, writes to 5 tables, and calls `COMMIT;`. If any step fails, `ROLLBACK;` undoes everything cleanly.

30. WAL (Write-Ahead Logging)

- **Technical Definition:** A database journaling technique where modifications are recorded sequentially in an auxiliary WAL file before writing to the main database file. Allows readers to read while writers write.
- **In TripWise AI:** Enabled via `PRAGMA journal_mode = WAL;`.

31. SQL Injection Prevention & Prepared Statements

- **Technical Definition:** An attack where malicious SQL commands are entered into form fields. Prevented using prepared statements (`db.prepare("INSERT ... VALUES (?, ?)")`) where values are bound strictly as data literals.

Part 4: Artificial Intelligence & Gemini

32. LLM (Large Language Model)

- **Technical Definition:** A deep learning transformer model trained on massive text corpora to predict tokens and generate human-like natural language.
- **In TripWise AI:** Google's `gemini-1.5-flash` model.

33. Prompt Engineering

- **Technical Definition:** The discipline of designing, structuring, and optimizing textual inputs (prompts) to guide LLMs toward accurate, consistent, and structured responses.
- **In TripWise AI:** Enforces strict JSON schema output, non-repetition of activities, and budget consistency.

34. Hallucination

- **Technical Definition:** A phenomenon where an LLM generates plausible-sounding but factually incorrect, illogical, or repetitive statements.
- **In TripWise AI:** Detected via a `Set()` title duplicate checker in `geminiService.js`.

35. Multi-Model Cascade

- **Technical Definition:** Sequentially falling back across multiple model endpoints (`gemini-1.5-flash` -> `gemini-2.0-flash` -> `gemini-1.5-pro`) if the primary model fails or experiences rate limits.

36. Zero-Failure Fallback Engine

- **Technical Definition:** A resilient software pattern where if external AI services fail completely, deterministic pre-computed or procedural logic generates an authentic response so the user experience never crashes.
- **In TripWise AI:** `generateFallbackItinerary()` delivers authentic schedules for hubs like Seoul, Paris, and Athens.

Part 5: Security & Cryptography

37. Cryptographic Salt

- **Technical Definition:** Random binary data added as an additional input to a one-way password hashing function to defend against dictionary attacks and pre-computed rainbow table attacks.
- **In TripWise AI:** Generated via `crypto.randomBytes(16)`.

38. Scrypt Key Derivation

- **Technical Definition:** A password-based key derivation function designed to be memory-hard, making hardware-accelerated attacks (ASICs, GPUs) prohibitively expensive.
- **In TripWise AI:** Computed via `crypto.scryptSync(password, salt, 64)`.

39. Timing Attack & `timingSafeEqual`

- **Technical Definition:** A side-channel attack where an attacker infers secrets by analyzing the execution duration of string comparisons.
- **In TripWise AI:** Fixed using `crypto.timingSafeEqual()` which compares buffers in constant time.

40. JWT (JSON Web Token) & HMAC-SHA256

- **Technical Definition:** A compact, URL-safe standard (RFC 7519) for transmitting claims between parties. Signed with a cryptographic hash (`HMAC-SHA256`) to ensure tamper-proof authentication without database session lookups.
- **In TripWise AI:** Issued in `auth.js` upon signup and login.

PART 14

TripWise AI — Comprehensive Viva & Technical Interview Q&A (150 Questions)

This document contains 150 comprehensive, categorized questions and model answers tailored specifically to TripWise AI. It is organized into 6 distinct tiers to prepare you for any examiner or interviewer.

Category 1: Basic & Conceptual Questions (Q1 – Q30)

Q1: What is TripWise AI?

Answer: TripWise AI is a full-stack automated travel planning web application. It takes user travel preferences—origin, destination, budget, duration, dates, and travel styles—and uses Google's Gemini AI and open-source APIs to generate a realistic, day-by-day itinerary with weather forecasts, interactive maps, expense breakdowns, and persistent storage.

Q2: What are the main technologies used in this project?

Answer: The frontend is built with React 19, Vite 8, Tailwind CSS v4, React Router DOM v7, Leaflet, and Recharts. The backend is built with Node.js v22+ and Express 4, with native `node:sqlite` for database persistence, Google Gemini AI (`@google/generative-ai`), Open-Meteo for weather, and OpenStreetMap Nominatim for geocoding.

Q3: What is a Single Page Application (SPA)?

Answer: A Single Page Application loads a single HTML file on initial load. As the user navigates between pages, JavaScript dynamically updates the DOM without reloading or re-fetching the entire HTML page from the server. TripWise AI uses React Router DOM to achieve this.

Q4: What is Vite and why did you choose it over Create React App?

Answer: Vite is a modern frontend build tool that leverages native ES Modules (ESM) in the browser. It offers instantaneous server start times (<300ms) and lightning-fast Hot Module Replacement (HMR). Create React App is deprecated, slow to bundle, and relies on outdated Webpack configurations.

Q5: What is JSX?

Answer: JSX stands for JavaScript XML. It is a syntax extension for JavaScript that allows us to write HTML-like markup inside React component files. It compiles down to standard `React.createElement()` function calls.

Q6: What is the Virtual DOM and how does it work?

Answer: The Virtual DOM is a lightweight, in-memory representation of the real DOM. When state changes, React creates a new Virtual DOM tree, compares it with the previous tree using a diffing algorithm, and updates only the changed elements in the real browser DOM.

Q7: What is the difference between State and Props?

Answer: State is an internal, mutable data store managed inside a component via `useState`. Props are read-only properties passed from a parent component down to a child component to configure it or share data.

Q8: What does `useState` do?

Answer: `useState` is a React Hook that declares a state variable and an updater function. When the updater function is called with a new value, React schedules a re-render of the component with the updated state.

Q9: What does `useEffect` do?

Answer: `useEffect` is a React Hook for executing side-effects in functional components, such as data fetching, setting up event listeners, or timers. It accepts a function and an optional dependency array that controls when the effect executes.

Q10: What is the purpose of the dependency array in `useEffect`?

Answer: The dependency array tells React when to re-run the effect. An empty array `[]` means the effect runs once when the component mounts. Passing variables `[searchQuery, activeCountry]` means the effect re-runs only when any of those values change. Omitting the array causes the effect to run on every single render.

Q11: What is `useRef` used for in this project?

Answer: `useRef` creates a mutable reference object whose `.current` property persists across renders without triggering a re-render when changed. In `CityDropdown.jsx`, we use `dropdownRef` to detect clicks outside the dropdown and `searchInputRef` to auto-focus the search field when opened.

Q12: What does `useMemo` do and where is it used?

Answer: `useMemo` memoizes the result of an expensive calculation so it is only recomputed when specified dependencies change. In `CityDropdown.jsx`, we use `useMemo` to filter our in-memory catalog of 178 cities so it doesn't recalculate on unrelated renders.

Q13: What is Tailwind CSS?

Answer: Tailwind CSS is a utility-first CSS framework that provides single-purpose utility classes (e.g. `flex`, `p-6`, `text-blue-600`, `rounded-xl`). It eliminates the need to write custom CSS stylesheets and purges unused styles during production builds.

Q14: What is client-side routing?

Answer: In client-side routing, the browser's URL changes, but the browser does not make a GET request to the server for a new HTML page. Instead, JavaScript (React Router) listens to the URL change and renders the matching component in place.

Q15: What HTTP port does the frontend run on? What about the backend?

Answer: The Vite frontend runs on port `5173`. The Express backend server runs on port `5001`.

Q16: What is Node.js?

Answer: Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 JavaScript engine that executes JavaScript code outside a web browser, enabling server-side development.

Q17: What is Express.js?

Answer: Express.js is a minimalist, flexible web application framework for Node.js that provides routing, middleware support, and HTTP utility methods for building RESTful APIs.

Q18: What is REST?

Answer: REST stands for Representational State Transfer. It is an architectural style for networked applications that uses standard HTTP methods (`GET`, `POST`, `PUT`, `DELETE`) and stateless request-response communication over unique resource URLs.

Q19: What is the difference between `GET` and `POST` requests?

Answer: `GET` is used to retrieve data from a server and should not modify server state; parameters are passed in the URL query string. `POST` is used to send data to the server to create or process a resource; parameters are passed in the HTTP request body.

Q20: What is CORS and why is it needed?

Answer: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts web pages from making requests to a different domain, protocol, or port than the one that served the page. Because our frontend is on port 5173 and our backend is on port 5001, CORS middleware is required on Express to permit requests.

Q21: What does `express.json()` do?

Answer: `express.json()` is built-in middleware that parses incoming requests with JSON payloads and populates `req.body` with the parsed JavaScript object.

Q22: What is SQLite?

Answer: SQLite is a C-language library that implements a self-contained, serverless, zero-configuration, transactional SQL database engine. It stores the entire database in a single disk file.

Q23: Where is the SQLite database file located in your project?

Answer: In `server/src/data/tripwise.db`.

Q24: What is Google Gemini?

Answer: Google Gemini is a family of multimodal large language models developed by Google AI. In our project, we use the `@google/generative-ai` SDK to prompt the model to generate structured travel itineraries.

Q25: What is Open-Meteo?

Answer: Open-Meteo is an open-source, free weather API that provides 7-day weather forecasts without requiring an API key. We use it to fetch temperature, weather codes, and precipitation probabilities.

Q26: What is OpenStreetMap Nominatim?

Answer: Nominatim is a search engine for OpenStreetMap data that provides geocoding (converting city names to GPS coordinates) and reverse geocoding.

Q27: What is Leaflet?

Answer: Leaflet is an open-source JavaScript library for mobile-friendly interactive maps. We use `react-leaflet` to render OpenStreetMap tiles and destination pins.

Q28: What is Recharts?

Answer: Recharts is a composable, SVG-based charting library built specifically for React components. We use it to render the visual budget breakdown on the trip details page.

Q29: What happens if you submit the trip form with an empty city?

Answer: The form validation in `Form.jsx` intercepts the submission, displays an inline red error banner saying *"Please enter a Starting City"*, and halts execution without sending an API request.

Q30: What is `.env` and why is it excluded from GitHub?

Answer: `.env` contains environment variables, including secret API keys (`GEMINI_API_KEY` , `JWT_SECRET`). It is excluded via `.gitignore` to prevent leaking private credentials to public version control. We provide `.env.example` as a template.

Category 2: Intermediate Questions (Q31 – Q60)

Q31: How do components communicate in React?

Answer: Components communicate downwards (parent to child) via props. Child components communicate upwards (child to parent) by calling callback functions passed down as props (e.g., `onChange={(val) => setDestination(val)}`). For sibling communication, state is lifted to their common parent.

Q32: What is "lifting state up"?

Answer: Lifting state up is the practice of moving shared state to the closest common ancestor component so that multiple child components can reflect the same data or trigger updates.

Q33: How does the dual-tier city search work in `CityDropdown.jsx`?

Answer: When the user types, Tier 1 immediately filters an in-memory catalog of 178 cities in RAM with `<1ms` latency. Concurrently, if the query has at least 2 characters, Tier 2 waits for a 320ms debounce and calls `/api/places/search-cities` , querying OpenStreetMap Nominatim for worldwide locations. Both result sets are merged and deduplicated using an ES6 `Map` .

Q34: What is debouncing and why is it important in search inputs?

Answer: Debouncing ensures that a time-consuming task is not executed until a specified delay has elapsed since it was last called. Without debouncing, typing "Paris" would fire 5 separate network requests. With a 320ms debounce, only 1 request is fired when the user finishes typing.

Q35: How does the cleanup function in `useEffect` work?

Answer: When `useEffect` returns a function, React executes that function when the component unmounts or before re-running the effect on dependency change. In `CityDropdown` , `return () => clearTimeout(timer)` cancels the pending timeout if the user types another letter before 320ms expires.

Q36: Why did you use `useRef` instead of `useState` for the search input focus?

Answer: `useRef` directly references the underlying HTML `<input>` DOM node without causing a re-render. Calling `.focus()` on a ref is an imperative DOM action that does not require re-rendering the component.

Q37: How does React Router DOM pass the trip ID to the trip details page?

Answer: When a trip is generated, `Form.jsx` calls `navigate('/trip-details?id=' + res.data.id)`. On the details page, `useLocation()` is paired with `new URLSearchParams(location.search)` to extract the `id` parameter and fetch the complete trip data.

Q38: What are the 7 relational tables in your SQLite database?

Answer: `users`, `trips`, `itinerary_days`, `activities`, `budget_breakdown`, `trip_highlights`, and `saved_places`.

Q39: What is the relationship between `trips` and `itinerary_days`?

Answer: It is a One-to-Many relationship. One trip has multiple itinerary days, linked by `itinerary_days.trip_id` referencing `trips.id`.

Q40: What is the relationship between `itinerary_days` and `activities`?

Answer: It is a One-to-Many relationship. One day has multiple activities, linked by `activities.day_id` referencing `itinerary_days.id`.

Q41: What is the relationship between `trips` and `budget_breakdown`?

Answer: It is a One-to-One relationship. Each trip has exactly one budget breakdown, enforced by a `UNIQUE` constraint on `budget_breakdown.trip_id`.

Q42: What is `PRAGMA foreign_keys = ON;` and why is it required?

Answer: By default, SQLite disables foreign key constraint checking for backwards compatibility. Executing this PRAGMA on startup forces SQLite to reject invalid foreign key inserts and enables `ON DELETE CASCADE`.

Q43: What is `ON DELETE CASCADE`?

Answer: It is a referential action that automatically deletes all child records when their parent record is deleted. In TripWise AI, deleting a row in `trips` automatically deletes all its days, activities, budget allocations, and highlights.

Q44: What is `PRAGMA journal_mode = WAL;`?

Answer: WAL stands for Write-Ahead Logging. Instead of locking the entire database during write transactions, SQLite appends changes to a `-wal` log file. This allows concurrent read operations to proceed without blocking writers, drastically improving concurrency.

Q45: What is an atomic database transaction?

Answer: A transaction is a unit of work that must satisfy ACID properties. In `saveTrip()`, writes across 5 tables are wrapped in `BEGIN TRANSACTION;` and `COMMIT;`. If an error occurs, `ROLLBACK;` reverts all changes so no partial or orphaned data remains.

Q46: How does password hashing work in your authentication route?

Answer: When a user registers, `crypto.randomBytes(16)` generates a 16-byte random salt. `crypto.scryptSync(password, salt, 64)` derives a 64-byte cryptographic hash. The salt and hash are stored together as `${salt}:${derivedKey}` in the `password_hash` column.

Q47: Why do you store a salt with each password?

Answer: A salt ensures that two users with identical passwords have completely different password hashes. It prevents attackers from using pre-computed lookup tables (Rainbow Tables) to crack passwords in bulk.

Q48: What is a timing attack and how did you prevent it?

Answer: A timing attack measures the exact time it takes to compare strings character-by-character. Standard comparison `===` exits on the first mismatched byte, leaking information about password characters. We use `crypto.timingSafeEqual()`, which takes constant time regardless of where mismatches occur.

Q49: How do your JWT tokens work?

Answer: Our `generateToken()` function constructs a header (`HS256`) and payload (`id`, `email`, `exp`), base64url encodes them, and generates an HMAC-SHA256 signature using `JWT_SECRET`. The client transmits this token in the `Authorization: Bearer <token>` header, and `verifyToken()` recalculates the HMAC signature to verify authenticity without querying the database.

Q50: How does the backend validate trip generation inputs?

Answer: In `server/src/routes/trips.js`, the route validates that `city` and `destination` are non-empty, `budget` is a positive number, and `duration` is an integer between 1 and 30 days. If any check fails, it responds with `HTTP 400 Bad Request`.

Q51: Why clamp trip duration between 1 and 30 days?

Answer: LLM output tokens are limited. Generating 30+ days of detailed activities with descriptions, tips, and costs would exceed Gemini's output token window, causing the JSON stream to cut off mid-syntax and fail `JSON.parse()`.

Q52: What is the purpose of `Promise.all` in `POST /api/trips/generate`?

Answer: It executes the itinerary generation (`generateItinerary`) and the cover photo fetch (`fetchDestinationImage`) concurrently. Rather than waiting for Gemini to finish before starting the photo search, both run in parallel, cutting total response latency in half.

Q53: How does the weather service handle coordinates?

Answer: In `weatherService.js`, the service first checks an in-memory geocode cache. If missing, it checks a dictionary of known coordinates (`KNOWN_COORDINATES`). If still missing, it queries OpenStreetMap Nominatim with an `AbortController` timeout before calling Open-Meteo.

Q54: What is an `AbortController`?

Answer: It is a standard JavaScript interface that allows an ongoing fetch request to be canceled. We set a 2.8s - 4.0s timeout timer that calls `controller.abort()`, preventing external geocoding delays from holding backend request threads open.

Q55: Why must you send a custom `User-Agent` header to Nominatim?

Answer: The OpenStreetMap Foundation's Acceptable Use Policy strictly requires all API clients to provide a unique, descriptive `User-Agent`. Requests with generic or missing headers are automatically rejected with an HTTP 403 Forbidden.

Q56: How does the Wikipedia image fallback work?

Answer: If no Unsplash key is configured, `unsplashService.js` calls Wikipedia's REST API (`/api/rest_v1/page/summary/<city>`). Wikipedia provides high-resolution, copyright-cleared photos of major landmarks for virtually every city in the world without requiring an API key.

Q57: What is the purpose of `ErrorBoundary.jsx`?

Answer: `ErrorBoundary` is a React class component that implements `componentDidCatch` to catch JavaScript errors anywhere in child component trees, log the error, and display a friendly fallback UI instead of crashing the entire React app.

Q58: What is the purpose of `lucide-react`?

Answer: `lucide-react` provides lightweight, customizable SVG icon components (e.g. `MapPin`, `Calendar`, `DollarSign`, `CloudSun`, `AlertCircle`) that render crisply at any display resolution with zero layout shift.

Q59: What is the difference between `npm run dev` and `npm run build`?

Answer: `npm run dev` runs Vite in development mode with live ESM serving and HMR. `npm run build` compiles, transpiles, and minifies the code into static HTML, CSS, and JS bundles inside the `/dist` directory for production deployment.

Q60: How does the backend serve the production frontend build?

Answer: In `server/src/index.js`, the server checks if `dist/` exists via `fs.existsSync()`. If present, it serves static files via `express.static()` and handles all unmatched non-API routes by returning `dist/index.html` for client-side routing.

Category 3: Advanced Questions (Q61 – Q90)

Q61: What is the multi-model cascade implemented in `geminiService.js`?

Answer: Instead of hardcoding a single AI model, `generateTripPlan()` loops through `["gemini-1.5-flash", "gemini-2.0-flash", "gemini-1.5-pro"]`. If the primary flash model encounters a transient error, timeout, or rate-limiting (HTTP 429), the service automatically retries with the next candidate before engaging the fallback engine.

Q62: How does the AI prompt enforce JSON-only output?

Answer: The prompt explicitly states: *"Return ONLY a valid, parseable JSON object. DO NOT include any markdown fences (like ```json), DO NOT include any explanatory text before or after the JSON."* It also specifies the exact JSON schema structure required.

Q63: Why does `geminiService.js` still strip markdown fences even after asking the model not to include them?

Answer: LLMs are non-deterministic and have strong training priors to wrap code blocks in markdown fences (```` ```json ... ``` ````). Defensive programming dictates sanitizing the raw response with `.replace(/```json/gi, "").replace(/```/g, "").trim()` before calling `JSON.parse()`.

Q64: How does TripWise AI detect and handle AI hallucinations like duplicate activities?

Answer: After parsing the JSON, `geminiService.js` iterates through all daily activities and tracks their titles using an ES6 `Set` (`seenTitles`). If `seenTitles.has(act.title)` evaluates to true, it detects that the AI generated repetitive schedule items, discards the hallucinated output, and invokes `generateFallbackItinerary()`.

Q65: How does the curated fallback engine guarantee zero failures?

Answer: `CURATED_DESTINATION_ITINERARIES` contains handcrafted, 100% verified multi-day itineraries for major destinations (Seoul, Busan, Tokyo, Kyoto, Paris, London, Athens, etc.) with real landmarks, descriptions, costs, and timings. For uncurated cities, it procedurally synthesizes realistic themed activities and calculates proportional budgets mathematically.

Q66: Why did you choose native `node:sqlite` over an ORM like Prisma or TypeORM?

Answer: Native `node:sqlite` has zero dependencies, compiles zero native C++ code on installation, and introduces zero abstraction overhead. ORMs introduce complex schema migrations, hidden N+1 query problems, and heavy bundle sizes. Native prepared statements provide maximum transparency, predictability, and performance.

Q67: What is the difference between SQLite rollback journaling and WAL mode?

Answer: In traditional rollback journaling, changes are written directly to the database file while the original unchanged pages are kept in a rollback journal. Writers require an exclusive lock, blocking all readers. In WAL mode, changes are appended to a separate write-ahead log file, allowing readers to read uncommitted snapshots concurrently without lock contention.

Q68: How do B-tree indexes optimize queries in SQLite?

Answer: B-trees maintain sorted keys with pointers to row IDs. In our schema, indexes like `idx_activities_trip_id` allow SQLite to locate all activities for a given trip in **O(log N)** time via binary search, rather than scanning thousands of activity rows in **O(N)** time.

Q69: Explain the ACID guarantees in your `saveTrip()` transaction.

Answer:

- **Atomicity:** Wrapped in `BEGIN TRANSACTION;` and `COMMIT;`. If writing Day 4 fails, `ROLLBACK;` cancels everything.
- **Consistency:** Enforces foreign keys (`PRAGMA foreign_keys = ON;`) so activity records cannot exist without a valid day and trip.
- **Isolation:** Concurrent readers cannot observe intermediate states where a trip has been created but its activities haven't been inserted.
- **Durability:** Committed transactions are flushed to disk via SQLite's write-ahead log.

Q70: How does `crypto.scryptSync()` protect against brute-force attacks?

Answer: Unlike fast hashing algorithms like MD5 or SHA-256 (which can be calculated billions of times per second on GPUs), `scrypt` is intentionally designed to be **memory-hard**. It requires significant RAM and CPU cycles to compute each hash, making hardware-accelerated ASIC and GPU brute-force attacks computationally and financially unfeasible.

Q71: Explain how `crypto.timingSafeEqual()` works internally.

Answer: It compares two memory buffers in constant time. Instead of terminating early on the first differing byte (`if (a[i] !== b[i]) return false`), it performs a bitwise OR across all XORed bytes (`result |= a[i] ^ b[i]`) across the entire buffer length, ensuring execution time is identical regardless of password correctness.

Q72: What is the anatomy of your custom JWT token?

Answer: It consists of three Base64URL-encoded strings separated by dots (`header.payload.signature`):

1. **Header:** `{"alg": "HS256", "typ": "JWT"}`
2. **Payload:** User claims (`id` , `email` , `name` , `iat` , `exp`)

3. **Signature:** `HMAC-SHA256(header + "." + payload, JWT_SECRET)`

Q73: Why is your JWT authentication stateless?

Answer: The server does not store active session tokens in the database or in memory. When a request arrives, `verifyToken()` validates the HMAC signature and expiration timestamp mathematically using the server's secret key. If valid, the user identity is trusted, reducing database read overhead to zero.

Q74: How do you handle lazy state initialization in `Form.jsx`?

Answer: `useState(() => { ... })` accepts a function rather than a direct value. This ensures the URL search param parsing logic (`new URLSearchParams(...)`) only executes once during initial mount, rather than on every single render.

Q75: What is the purpose of `React.StrictMode` in development?

Answer: `React.StrictMode` intentionally mounts, unmounts, and re-mounts components in development to detect uncleaned side-effects (such as orphaned timers or unremoved event listeners), memory leaks, and legacy lifecycle usage.

Q76: How does Leaflet handle dynamic map centering when the destination changes?

Answer: In `InteractiveMap.jsx`, the `<MapContainer>` accepts `center={[lat, lon]}` and `zoom={12}`. When coordinates update, a helper component using `useMap()` calls `map.flyTo([lat, lon], 12)` to smoothly pan the camera to the new destination.

Q77: How are SVG budget charts rendered with Recharts?

Answer: Recharts takes the structured `budgetBreakdown` object from SQLite, transforms it into an array of `{ name: "Hotels", value: 600, fill: "#3B82F6" }`, and renders responsive SVG `<PieChart>` and `<Pie>` components with tooltip hover interactions.

Q78: How do you prevent cross-site scripting (XSS) in React?

Answer: React escapes all strings rendered in JSX by default before inserting them into the DOM. Unless a developer explicitly uses `dangerouslySetInnerHTML`, injected HTML or script tags like `<script>alert(1)</script>` are treated as plain text literals.

Q79: How do you prevent SQL Injection in your backend?

Answer: We never concatenate raw user input into SQL queries. We use parameterized prepared statements (`db.prepare("INSERT ... VALUES (?, ?)")`). The SQLite engine compiles the query structure first, and parameters are inserted strictly as data literals.

Q80: How does the application handle rate-limiting from OpenStreetMap Nominatim?

Answer:

1. Queries are debounced by 320ms on the frontend.
2. The backend caches geocode results in an in-memory `Map`.
3. If Nominatim fails or times out, the backend falls back to its internal `KNOWN_COORDINATES` dictionary.

Q81: What is the difference between `dependencies` and `devDependencies` in `package.json`?

Answer: `dependencies` are packages required for the application to run in production (e.g. `react`, `express`, `leaflet`). `devDependencies` are packages only required during local development and building (e.g. `vite`, `eslint`, `@vitejs/plugin-react`).

Q82: How does `crypto.randomBytes(4).toString("hex")` work in ID generation?

Answer: `Date.now()` provides millisecond uniqueness, and `crypto.randomBytes(4).toString("hex")` appends 8 cryptographically secure random hexadecimal characters (e.g., `trip_1741498800000_a3f8c12e`), eliminating collision risks even under simultaneous user requests.

Q83: What is the role of `server/src/db/seed.js`?

Answer: It is an automated seeder script that clears and populates the SQLite database with realistic sample trips, days, activities, and budget breakdowns so developers or examiners can test the application immediately without manually generating trips.

Q84: How does the AI Concierge chatbot maintain context in `POST /api/chat`?

Answer: The request payload passes `destination` and a truncated `tripContext` (up to 1,000 characters) containing duration, budget, and travel styles. The prompt injects this context so Gemini answers specifically about that user's trip rather than giving generic advice.

Q85: What happens if an unauthenticated user generates a trip?

Answer: Guest trip creation is fully supported. The backend accepts `userId: null`, sets `trips.user_id = NULL`, and saves the trip normally. The user can view the full itinerary, map, and weather. If the user later signs up or logs in, their user ID can be linked.

Q86: How does the frontend handle network errors during trip generation?

Answer: `api.js` wraps `fetch()` in a `try...catch` block. If the server is unreachable, it throws an error that `Form.jsx` catches, resetting `loading` to `false` and rendering an inline error alert explaining that the backend could not be reached.

Q87: What is tree-shaking and how does Vite use it?

Answer: Tree-shaking is a dead-code elimination technique used by Rollup during `vite build`. It analyzes ES module `import` and `export` statements and excludes unused functions from the final production bundle, keeping download sizes small.

Q88: Why are coordinates stored as JSON strings in SQLite?

Answer: SQLite does not have a native Point/Geometry data type like PostGIS. Storing `{ lat: 37.98, lon: 23.72 }` as a JSON string (`coordinates TEXT`) allows simple serialization and deserialization while preserving full floating-point precision.

Q89: How does the application prevent duplicate cities in the search dropdown?

Answer: In `CityDropdown.jsx`, `displayedCities` iterates through local matches and remote matches, inserting items into a `Map` keyed by `${item.city.toLowerCase()}_${item.country.toLowerCase()}`. If a remote result matches a local city, the local verified entry is preserved.

Q90: What is the difference between synchronous and asynchronous operations in Node.js?

Answer: Synchronous operations block the Node.js event loop until completion, preventing other tasks from executing. Asynchronous operations (like `fetch`, timers, or promises) delegate work to background threads or the operating system, allowing the single-threaded event loop to continue processing other requests.

Category 4: Code-Specific Questions (Q91 – Q110)

Q91: Where is the form validation logic written?

Answer: Inside the `handleGenerateTrip` function in `src/components/Tripgenerator/Form.jsx`, lines 118–160.

Q92: Which file initializes the SQLite schema?

Answer: `server/src/db/schema.js` inside the `initSchema()` function.

Q93: Where are the database foreign keys and WAL mode configured?

Answer: In `server/src/db/database.js` on lines 21–22: `db.exec("PRAGMA foreign_keys = ON;");` and `db.exec("PRAGMA journal_mode = WAL;");`.

Q94: Where is the atomic transaction executed when saving a trip?

Answer: In `server/src/db/repositories.js` inside `tripsRepo.saveTrip()` on lines 128–246.

Q95: Where are candidate Gemini models declared?

Answer: In `server/src/services/geminiService.js` on line 580: `const candidateModels = ["gemini-1.5-flash", "gemini-2.0-flash", "gemini-1.5-pro"];`.

Q96: Where is markdown fence stripping implemented?

Answer: In `server/src/services/geminiService.js` on lines 589–592 using `.replace(/```json/gi, "").replace(/```/g, "").trim()`.

Q97: Where is duplicate activity detection implemented?

Answer: In `server/src/services/geminiService.js` on lines 598–612 using `const seenTitles = new Set();`.

Q98: What is the exact debounce delay in `CityDropdown.jsx`?

Answer: 320 milliseconds (`setTimeout(..., 320)` on line 93 of `CityDropdown.jsx`).

Q99: Where is password hashing defined?

Answer: In `server/src/routes/auth.js` in `hashPassword()` on lines 67–71 using `crypto.scryptSync(password, salt, 64)`.

Q100: Where is constant-time password verification defined?

Answer: In `server/src/routes/auth.js` in `verifyPassword()` on line 79 using `crypto.timingSafeEqual(keyBuffer, derivedKey)`.

Q101: How many destinations are in `src/data/destinations.js`?

Answer: Exactly 178 cities across 56 countries.

Q102: Which component renders the interactive Leaflet map?

Answer: `src/components/Details/Explore/InteractiveMap.jsx`.

Q103: Which component renders the visual budget breakdown chart?

Answer: `src/components/Details/RSection/BudgetCard.jsx`.

Q104: Where is the centralized frontend API client located?

Answer: In `src/services/api.js`.

Q105: How does `src/services/api.js` know which backend URL to call?

Answer: Line 7: `const API_BASE = import.meta.env.VITE_API_BASE_URL || "/api";`.

Q106: Where is the Open-Meteo weather endpoint called?

Answer: In `server/src/services/weatherService.js` on line 201.

Q107: Where is the Wikipedia image fallback implemented?

Answer: In `server/src/services/unsplashService.js` on lines 145–188.

Q108: Which function retrieves user-specific saved trips?

Answer: `tripsRepo.getAllTrips(userId)` in `server/src/db/repositories.js`.

Q109: What is the purpose of `src/components/Details/ChatButton.jsx`?

Answer: It renders a floating AI Travel Concierge chat modal that queries `POST /api/chat`.

Q110: Where is the global error handler middleware located?

Answer: In `server/src/index.js` on lines 81–88: `app.use((err, req, res, next) => { ... })`.

Category 5: Bugs & Problem-Solving Questions (Q111 – Q130)

Q111: What was the bug with hardcoded form defaults and how did you resolve it?

Answer: Initially, the form pre-loaded with "Seoul", "San Francisco", `$2500`, and `5 days`, causing users to accidentally create unintended trips. We resolved it by initializing all form states to empty strings and empty arrays, requiring explicit manual entry.

Q112: Why did `CityDropdown` originally have a duplicate key warning for Athens?

Answer: Both Athens, Greece and Athens, Georgia (USA) existed in the dataset with overlapping identifiers. We resolved it by generating unique keys combining `${city}_${country}` and disambiguating city records.

Q113: How did you fix the ESLint warning regarding missing hook dependencies?

Answer: React Hooks ESLint rules flagged `useEffect` calls where variables were used inside effects without being listed in the dependency array. We audited all dependencies, wrapping stable calculations in `useCallback` or `useMemo` and adding appropriate dependencies.

Q114: Why did OpenStreetMap Nominatim occasionally return HTTP 403 Forbidden?

Answer: Nominatim requires an explicit, identifiable `User-Agent`. When requests lacked this header, Nominatim blocked them. We added `"User-Agent": "TripWiseAI/1.0"` to the fetch headers.

Q115: What caused database locking errors before enabling WAL mode?

Answer: Under default SQLite journaling, writing to the database placed an exclusive lock on the file, causing concurrent read requests from other routes to fail with `SQLITE_BUSY`. Enabling `PRAGMA journal_mode = WAL;` solved this by isolating writes to the write-ahead log.

Q116: How did you fix Gemini returning markdown code fences that broke `JSON.parse()`?

Answer: Even when prompted for raw JSON, Gemini occasionally wrapped responses in ```` ```json ... ``` ````. We added regex sanitization `.replace(/```json/gi, "").replace(/```/g, "").trim()` before parsing.

Q117: What was the issue with AI repeating the same attraction across multiple days?

Answer: LLMs sometimes lack cross-day context memory and repeat famous landmarks (e.g. recommending the Louvre on Day 1 and Day 3). We implemented a duplicate activity detector using an ES6 `Set()`. If duplicate titles occur, the response is rejected in favor of the curated non-repeating engine.

Q118: How did you fix the dropdown staying open after clicking outside?

Answer: We added an event listener to `document` on `mousedown` inside a `useEffect`. If `!dropdownRef.current.contains(event.target)`, it calls `setIsOpen(false)`.

Q119: What happened when the destination had no GPS coordinates?

Answer: The Leaflet map would crash with invalid center coordinates. We resolved this by adding automatic fallback geocoding in `weatherService.js` and defaulting to center coordinates if geocoding fails.

Q120: How did you solve the problem of slow city searches over poor internet?

Answer: By implementing a dual-tier search: 178 popular cities are filtered instantly from in-memory RAM in `<1ms`, providing immediate feedback while the debounced remote search runs asynchronously.

Q121: How did you fix the issue where guest users couldn't generate trips?

Answer: The database schema was updated so `trips.user_id` allows `NULL` values and has `ON DELETE SET NULL`. If a user is not logged in, `userId` is passed as `null` and the trip saves successfully.

Q122: Why did trip duration need an upper limit of 30 days?

Answer: Without a limit, a user could request a 100-day trip, which would exceed Gemini's maximum output token limit (8192 tokens) and cause the response to terminate mid-JSON.

Q123: How did you prevent negative or decimal budgets from breaking calculations?

Answer: We added validation checks: `isNaN(parsedBudget) || parsedBudget <= 0`. If invalid, an error banner is displayed and submission is halted.

Q124: How did you resolve the issue where clicking a curated trip on the landing page didn't populate the form?

Answer: We added a query parameter to landing page links (`/generate-trip?destination=Kyoto`). In `Form.jsx`, we used lazy initialization with `new URLSearchParams(location.search).get("destination")` to pre-populate the destination.

Q125: What was the issue with Unsplash API rate limits and how was it solved?

Answer: Unsplash free tier allows only 50 requests per hour. When exceeded, images failed. We implemented a 5-tier fallback cascade that falls back to Wikipedia's REST API (unlimited and free) and a curated photo dictionary.

Q126: How did you fix the issue where deleting a trip left orphaned activities in the database?

Answer: We enabled `PRAGMA foreign_keys = ON;` and defined all child tables with `FOREIGN KEY (trip_id) REFERENCES trips(id) ON DELETE CASCADE`.

Q127: Why did the map marker icon initially fail to render in Leaflet?

Answer: Leaflet has a known Webpack/Vite bundler issue where default marker icon image paths are not automatically resolved. We resolved it by importing marker images explicitly and configuring `L.Icon.Default.mergeOptions`.

Q128: How did you resolve password verification vulnerabilities?

Answer: Legacy plain-text or unsalted SHA-256 hashes are vulnerable to rainbow tables and timing attacks. We migrated to salted `crypto.scryptSync()` and `crypto.timingSafeEqual()`.

Q129: How did you prevent duplicate accounts during user registration?

Answer: The `users` table defines `email TEXT UNIQUE NOT NULL`. In `auth.js`, the route executes `usersRepo.findUserByEmail(email)` and returns `HTTP 409 Conflict` if the email is already registered.

Q130: What was the issue with Vite dev server refusing external host connections?

Answer: When testing via tunnels or network IPs, Vite blocked incoming requests with "Blocked host". We configured `server.allowedHosts: true` in `vite.config.js` to allow development access across local network devices.

Category 6: Architectural & Design Decision Questions (Q131 – Q150)

Q131: Why did you build TripWise AI as a full-stack app instead of a frontend-only app calling Gemini directly?

Answer: Calling Gemini directly from the frontend exposes the private API key to anyone inspecting browser network traffic. A full-stack architecture keeps API keys secure on the server, allows multi-table relational persistence, and enables server-side caching and validation.

Q132: Why did you choose React over Vanilla JavaScript?

Answer: A trip planning interface has high state complexity: autocomplete dropdowns, style chips, collapsible itinerary days, interactive maps, weather tabs, and budget charts. Managing this with vanilla DOM manipulation would lead to spaghetti code. React's declarative component model and Virtual DOM ensure maintainable, high-performance UI rendering.

Q133: Why did you choose SQLite over MongoDB?

Answer: Travel itineraries have inherently relational structures: a User owns Trips, a Trip owns Days, a Day owns Activities, and a Trip owns Budget Breakdown. SQLite enforces strict relational schema integrity, foreign keys, and ACID transactions. Additionally, native `node:sqlite` requires zero external daemon setup.

Q134: Why did you choose SQLite over PostgreSQL?

Answer: PostgreSQL requires installing a separate database server, setting up user roles, and managing connection pools. SQLite is embedded directly into the Node process and stored in a single file (`tripwise.db`). Anyone who clones the repo can run it immediately without external database configuration.

Q135: Why did you use native `node:sqlite` instead of `better-sqlite3`?

Answer: `better-sqlite3` is a third-party npm package that compiles C++ code during installation (`node-gyp`). It frequently fails to build on different operating systems or Node versions. Native `node:sqlite` is built directly into Node.js 22+, eliminating all native compilation failures.

Q136: Why did you choose Open-Meteo over OpenWeatherMap?

Answer: OpenWeatherMap requires developer account creation, credit card verification, and API keys with tight call limits. Open-Meteo is completely free, open-source, requires zero API keys, and provides high-accuracy 7-day forecasts.

Q137: Why did you choose Tailwind CSS v4 over Bootstrap?

Answer: Bootstrap imposes rigid, generic component styles that are difficult to customize for modern travel interfaces. Tailwind provides low-level utility classes that allow pixel-perfect custom designs with zero unused CSS in production.

Q138: Why do you have both a 178-city local dataset and an external geocoding API?

Answer: A pure API search introduces network latency (200-500ms) on every keystroke. A pure local dataset cannot cover every small town on Earth. Our hybrid approach delivers instant `<1ms` search for 178 top destinations while providing universal worldwide search via Nominatim as a fallback.

Q139: Why did you implement a curated fallback engine instead of just showing an error when Gemini fails?

Answer: In software engineering, resilience is a critical quality attribute. If an examiner evaluates the project without an internet connection or without a Gemini API key, the app should still demonstrate its full functionality (itinerary rendering, maps, charts, persistence) rather than failing with a blank screen.

Q140: Why do you split the trip details view into a two-column layout?

Answer: Vacation planning requires comparing sequential schedules with spatial and financial constraints. The left column focuses on the chronological narrative (Day 1, Day 2), while the right column provides spatial and analytical context (Map, Weather, Budget).

Q141: Why do you use Lucide icons instead of FontAwesome?

Answer: FontAwesome requires large web fonts or bulky icon bundles. Lucide React provides individual tree-shakeable SVG components, ensuring that only the specific icons used in the app are bundled into production.

Q142: Why do you store dates as ISO strings rather than Unix timestamps?

Answer: ISO strings (`YYYY-MM-DD`) are human-readable directly in database queries and JSON payloads, and map cleanly to HTML `<input type="date">` elements without requiring time-zone conversion libraries.

Q143: Why did you choose `scrypt` over `bcrypt` or plain SHA-256?

Answer: Plain SHA-256 is vulnerable to GPU-based rainbow table attacks. `bcrypt` has a 72-byte password length limitation and requires third-party C++ bindings. `scrypt` is built directly into Node's `node:crypto` standard library and is memory-hard, making it highly resistant to hardware brute-force attacks.

Q144: Why did you implement custom JWT generation instead of using `jsonwebtoken`?

Answer: Using native `node:crypto` to build HMAC-SHA256 tokens demonstrates a deep, first-principles understanding of cryptographic standards (Base64URL encoding, cryptographic hashing, signature verification) and reduces third-party dependency vulnerabilities.

Q145: Why did you normalize the database into 7 tables instead of storing one large JSON blob in `trips.itinerary`?

Answer: Normalization enables granular querying, updating, and analytics. For example, we can count total activities across all trips, update an individual activity's cost, or delete a single day without reading, parsing, and re-writing an entire monolithic JSON document.

Q146: Why do you use React Router DOM v7?

Answer: Version 7 is the latest, most stable release of React Router, offering enhanced type-safety, efficient nested routing, and seamless declarative navigation hooks.

Q147: What is the benefit of keeping the backend stateless?

Answer: Stateless backends do not maintain user session state in memory. Any instance of the server can handle any user request, making horizontal scaling behind a load balancer trivial.

Q148: How does the application maintain responsive design across mobile and desktop?

Answer: Tailwind's responsive breakpoint modifiers (`md:` , `lg:`) are used throughout. For example, the trip details page renders a single stacked column on mobile (`flex flex-col`) and automatically transforms into a dual-column split on desktop (`lg:grid lg:grid-cols-12`).

Q149: If you had 6 more months to work on this project, what would you add?

Answer:

1. Flight and hotel booking API integrations (Amadeus or Skyscanner).
2. Multi-user collaboration for group trip planning via WebSockets.
3. Offline PWA (Progressive Web App) support with service workers.
4. Export itinerary to PDF and Google Calendar synchronization.

Q150: What was the most important engineering lesson you learned while building TripWise AI?

Answer: The importance of designing for failure. Real-world systems depend on external services (AI models, geocoders, weather APIs) that can fail, timeout, or rate-limit at any time. Building defensive architectures—with debouncing, timeouts, input sanitization, multi-model cascades, and deterministic fallback engines—is what separates fragile demos from robust production-grade software.

PART 15

TripWise AI — Bugs Encountered & Engineering Solutions

This document details 12 real-world software engineering bugs encountered during the development of TripWise AI. For each bug, you will find the symptom, root cause, code comparison (Before vs. After), and a script on how to explain it during an interview.

Bug 1: Hardcoded Default Form Values

- **Severity:** High (User Experience & Data Quality)
- **Symptom:** When a user opened `/generate-trip`, the inputs automatically populated with *"Seoul"*, *"San Francisco, USA"*, `$2500`, and `5 days`. Users frequently submitted the form without changing these values, cluttering the database with accidental duplicate trips.
- **Root Cause:** The `useState` hooks in `Form.jsx` had static fallback strings hardcoded into their initial declarations.
- **Before (Buggy Code):**

```
const [city, setCity] = useState("San Francisco, USA");
const [destination, setDestination] = useState("Seoul, South Korea");
const [budget, setBudget] = useState("2500");
const [duration, setDuration] = useState("5");
const [selectedStyles, setSelectedStyles] = useState(["Culture", "City Exploration"]);
```

- **After (Fixed Code):**

```
const [city, setCity] = useState("");
const [destination, setDestination] = useState(() => {
  const params = new URLSearchParams(location.search || window.location.search);
  return params.get("destination") || "";
});
const [budget, setBudget] = useState("");
const [duration, setDuration] = useState("");
const [selectedStyles, setSelectedStyles] = useState([]);
```

- **Interview Pitch:**

> *"Initially, the form contained hardcoded defaults for demonstration purposes. During user testing, we noticed users accidentally generated Seoul trips without realizing the form wasn't blank. We refactored all form state to initialize completely empty, using a lazy initializer only if a destination query parameter was explicitly passed from the landing page cards."*

Bug 2: Duplicate Key Warning in City Dropdown

- **Severity:** Medium (React Rendering Warning)
- **Symptom:** React console logged: `Warning: Encountered two children with the same key, 'athens'`.
- **Root Cause:** The dataset contained both Athens, Greece and Athens, Ohio/Georgia (USA), and keys were generated solely from `item.city.toLowerCase()`.
- **Before (Buggy Code):**

```
{displayedCities.map((item) => (
  <div key={item.city.toLowerCase()} onClick={() => handleSelect(item)}>
    {item.city}, {item.country}
  </div>
))}
```

- **After (Fixed Code):**

```
{displayedCities.map((item) => (
  <div
    key={`${item.city.toLowerCase()}_${item.country.toLowerCase()}_${item.id || ""}`}
    onClick={() => handleSelect(item)}
  >
    {item.city}, {item.country}
  </div>
))}
```

- **Interview Pitch:**

> *"We encountered a key collision because multiple cities across different countries share the same name (e.g. Athens, Greece vs Athens, USA). We resolved this by composing compound unique keys from both the city name and country name, disambiguating the React Virtual DOM list reconciliation."*

Bug 3: Missing Hook Dependencies in ESLint (`react-hooks/exhaustive-deps`)

- **Severity:** Medium (Code Quality & Potential Stale State)
- **Symptom:** Running `npm run lint` flagged warnings: `React Hook useEffect has a missing dependency: 'activeCountry'`.
- **Root Cause:** Variables referenced inside `useEffect` were omitted from the dependency array, causing effects to use stale closures when country filters changed.
- **Before (Buggy Code):**

```
useEffect(() => {
  if (searchQuery.length >= 2) {
    searchCities(searchQuery, activeCountry);
  }
}, [searchQuery]); // activeCountry is missing!
```

- **After (Fixed Code):**

```
useEffect(() => {
  if (searchQuery.length >= 2) {
    searchCities(searchQuery, activeCountry);
  }
}, [searchQuery, activeCountry]); // Properly tracks both variables
```

- **Interview Pitch:**

> *"We audited our components with ESLint's `react-hooks/exhaustive-deps` rule. In `CityDropdown.jsx`, we ensured that any state variable read inside `useEffect` or `useMemo` is strictly declared in the dependency array, preventing stale closure bugs when users change filter tabs."*

Bug 4: OpenStreetMap Nominatim 403 Forbidden & Rate-Limiting

- **Severity:** High (External API Failure)
- **Symptom:** City search requests outside the 178-catalog occasionally returned HTTP 403 Forbidden or failed to load.
- **Root Cause:**
  1. OpenStreetMap Nominatim requires an explicit, identifiable `User-Agent` header. Requests sent with generic Node/fetch headers are automatically blocked.
  2. Typing quickly sent multiple requests within milliseconds, violating Nominatim's 1-request-per-second usage policy.
- **Before (Buggy Code):**

```
// Server route calling Nominatim with no headers
const res = await fetch(`https://nominatim.openstreetmap.org/search?q=${query}&format=json`);
```

- **After (Fixed Code):**

```
// 1. Frontend: 320ms Debounce
const timer = setTimeout(() => searchCities(query), 320);

// 2. Backend: Custom User-Agent & AbortController
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 2800);

const response = await fetch(nominatimUrl, {
  headers: {
    "User-Agent": "TripWiseAI/1.0 (city-search-service)",
    Accept: "application/json",
  },
  signal: controller.signal,
});
clearTimeout(timeoutId);
```

- **Interview Pitch:**

> *"Nominatim has strict usage policies. We solved rate-limiting by introducing a 320ms debounce on the frontend and an in-memory cache on the backend. We also added a custom User-Agent header and an AbortController with a 2.8-second timeout to prevent sluggish external responses from blocking our server."*

Bug 5: SQLite Database Locking (`SQLITE_BUSY`)

- **Severity:** High (Backend Concurrency)
- **Symptom:** When a user requested a new trip while another request was reading from `/api/trips`, the server threw: `SqliteError: database is locked`.
- **Root Cause:** SQLite's default rollback journal places an exclusive file lock on the entire database file during writes, blocking concurrent readers.
- **Before (Buggy Code):**

```
export const db = new DatabaseSync(dbPath);
// Default journal mode: DELETE
```

- **After (Fixed Code):**

```
export const db = new DatabaseSync(dbPath);
db.exec("PRAGMA foreign_keys = ON;");
db.exec("PRAGMA journal_mode = WAL;");
```

- **Interview Pitch:**

> *"Under concurrent read and write operations, SQLite's default journaling caused `SQLITE_BUSY` locking errors. We resolved this by enabling Write-Ahead Logging via `PRAGMA journal_mode = WAL;`. WAL isolates writes to a dedicated log file, allowing concurrent read operations without blocking."*

Bug 6: Gemini AI Markdown Code Fences Breaking `JSON.parse()`

- **Severity:** High (AI Pipeline Failure)
- **Symptom:** Trip generation crashed with `SyntaxError: Unexpected token '`', "```json..." is not valid JSON`.

- **Root Cause:** LLMs often wrap JSON output in markdown fences (`` ```json\n{...}\n``` ``) despite prompt instructions to return raw JSON only.
- **Before (Buggy Code):**

```
const text = await result.response.text();
const parsed = JSON.parse(text); // Crashes if fences are present!
```

- **After (Fixed Code):**

```
const text = await result.response.text();
const cleaned = text
  .replace(/```json/gi, "")
  .replace(/```/g, "")
  .trim();
const parsed = JSON.parse(cleaned);
```

- **Interview Pitch:**

> *"Generative AI models non-deterministically wrap JSON responses in markdown syntax blocks. To prevent `JSON.parse` syntax exceptions, we implemented defensive regex sanitization that strips markdown fences and trims whitespace before parsing."*

Bug 7: AI Hallucinating Duplicate Activities Across Different Days

- **Severity:** Medium (Itinerary Realism)
- **Symptom:** For longer trips (e.g. 5 days in Paris), Gemini recommended the *Eiffel Tower* on Day 1, Day 3, and Day 5.
- **Root Cause:** LLMs generate text token-by-token and can lose context across large prompt outputs, repeating famous landmarks.
- **Before (Buggy Code):**

```
// Returned parsed JSON directly without inspecting activities
return parsed;
```

- **After (Fixed Code):**

```
const seenTitles = new Set();
let hasDuplicates = false;

for (const day of parsed.days) {
  if (!day.activities) continue;
  for (const act of day.activities) {
    if (seenTitles.has(act.title)) {
      hasDuplicates = true;
      break;
    }
    seenTitles.add(act.title);
  }
  if (hasDuplicates) break;
}

if (!hasDuplicates) {
  return parsed;
} else {
  // Graceful fallback to verified non-repeating engine
  return generateFallbackItinerary(destination, duration, budget, travelStyle, preferences);
}
```

- **Interview Pitch:**

> *"LLMs frequently suffer from repetition hallucinations. We engineered an automated quality check using an ES6 Set to ensure all activity titles across the entire trip are unique. If duplicates are detected, the response is discarded and our non-repeating curated engine steps in."*

Bug 8: City Dropdown Remaining Open on Outside Click

- **Severity:** Low (UI Annoyance)
- **Symptom:** Clicking anywhere outside the city search menu left the dropdown menu visible over other form elements.
- **Root Cause:** React components do not listen to clicks outside their DOM boundary by default.
- **Before (Buggy Code):**

```
// Only toggled isOpen when clicking the input itself
<input onClick={() => setIsOpen(!isOpen)} />
```

- **After (Fixed Code):**

```
const dropdownRef = useRef(null);

useEffect(() => {
  const handleClickOutside = (event) => {
    if (dropdownRef.current && !dropdownRef.current.contains(event.target)) {
      setIsOpen(false);
    }
  };
  document.addEventListener("mousedown", handleClickOutside);
  return () => document.removeEventListener("mousedown", handleClickOutside);
}, []);
```

- **Interview Pitch:**

> *"To provide a polished user experience, we bound an outside-click listener to `document.addEventListener('mousedown')`. Using a React `useRef` attached to the container, we check if the clicked element is within the dropdown boundaries; if not, we automatically close it."*

Bug 9: Orphaned Database Records on Trip Deletion

- **Severity:** High (Database Integrity)
- **Symptom:** When a trip was deleted from `trips`, rows in `itinerary_days`, `activities`, and `budget_breakdown` remained in the database, causing storage bloat and broken foreign keys.
- **Root Cause:** Foreign keys were not cascading deletions because `PRAGMA foreign_keys = ON;` was missing and tables lacked `ON DELETE CASCADE`.
- **Before (Buggy Code):**

```
CREATE TABLE activities (
  id TEXT PRIMARY KEY,
  day_id TEXT,
  trip_id TEXT
);
```

- **After (Fixed Code):**

```
CREATE TABLE activities (
  id TEXT PRIMARY KEY,
  day_id TEXT NOT NULL,
  trip_id TEXT NOT NULL,
  FOREIGN KEY (day_id) REFERENCES itinerary_days(id) ON DELETE CASCADE,
  FOREIGN KEY (trip_id) REFERENCES trips(id) ON DELETE CASCADE
);
```

- **Interview Pitch:**

> *"We enforced strict relational integrity by declaring `FOREIGN KEY ... ON DELETE CASCADE` on all child tables and enabling SQLite foreign key enforcement. Deleting a trip now cleanly and automatically purges all related days, activities, and budget records in a single operation."*

Bug 10: Timing Attack Vulnerability in Password Verification

- **Severity:** High (Security)
- **Symptom:** Password hash comparison used standard equality `storedHash === inputHash`.
- **Root Cause:** Standard string comparisons terminate immediately upon encountering the first mismatch, allowing attackers to measure execution time down to nanoseconds to deduce password hashes.
- **Before (Buggy Code):**

```
const derivedKey = crypto.scryptSync(password, salt, 64).toString("hex");
return key === derivedKey; // Vulnerable to timing attacks!
```

- **After (Fixed Code):**

```
const keyBuffer = Buffer.from(key, "hex");
const derivedKey = crypto.scryptSync(password, salt, 64);
return crypto.timingSafeEqual(keyBuffer, derivedKey); // Constant-time comparison
```

- **Interview Pitch:**

> *"We identified a cryptographic timing vulnerability where standard string equality leaks information about byte matches. We replaced string comparison with Node's native `crypto.timingSafeEqual()`, ensuring password verification executes in constant time regardless of correctness."*

PART 16

TripWise AI — Testing Checklist & Quality Verification Guide

This document outlines manual testing procedures, automated API curl scripts, edge-case test cases, and fallback verification instructions for TripWise AI.

1. Manual UI Verification Checklist

Execute these manual tests in a web browser (`http://localhost:5173`):

Test ID	Feature / Component	Test Action	Expected Result	Pass / Fail
TC-01	**Form Initial State**	Open `/generate-trip` directly.	Every input is completely blank. No hardcoded default cities (no Seoul, no San Francisco, no default budget).	[ ]
TC-02	**Validation: Empty City**	Leave starting city empty and click "Generate Trip".	Form halts submission. Red error banner displays: *"Please enter a Starting City."*	[ ]
TC-03	**Validation: Empty Destination**	Enter city, leave destination empty, click "Generate Trip".	Form halts submission. Error banner displays: *"Please enter or select a Destination."*	[ ]
TC-04	**Validation: Zero/Negative Budget**	Enter budget `0` or `-500`, click "Generate Trip".	Error banner displays: *"Budget must be a valid positive number."*	[ ]
TC-05	**Validation: Excessive Duration**	Enter duration `45`, click "Generate Trip".	Error banner displays: *"Trip duration must be between 1 and 30 days."*	[ ]
TC-06	**Validation: No Style Selected**	Fill city, dest, budget, duration, but uncheck all styles.	Error banner displays: *"Please select at least one Travel Style."*	[ ]
TC-07	**Instant Local Search**	Type *"Chandigarh"* in CityDropdown.	Displays within `<1ms` from local catalog with `India` and pre-filled coordinates.	[ ]
TC-08	**Debounced Remote Search**	Type an uncurated city like *"Gwalior"*.	Loader spins after 320ms; OpenStreetMap Nominatim results appear in dropdown list.	[ ]
TC-09	**Outside Click Dropdown**	Open CityDropdown, click outside the menu.	Dropdown closes automatically.	[ ]
TC-10	**Trip Generation Flow**	Fill valid inputs, select 2 styles, click "Generate Trip".	Button displays spinner. Navigates to `/trip-details?id=trip_...` within 3–8 seconds.	[ ]
TC-11	**Itinerary Display**	Inspect left column on `/trip-details`.	Correct number of days render. Each day has activities with time, title, cost, and tips.	[ ]
TC-12	**Interactive Leaflet Map**	Inspect right column on `/trip-details`.	Map centers over destination coordinates with custom marker pin and zoom controls.	[ ]
TC-13	**Live Weather Widget**	Check the 7-day weather card.	Displays current temperature and 7-day high/low forecast with appropriate weather icons.	[ ]
TC-14	**Visual Budget Chart**	Check the Budget Breakdown card.	Recharts pie chart renders proportional slices for flights, hotels, food, transport, etc.	[ ]
TC-15	**AI Concierge Chat**	Click floating chat button, ask a travel question.	AI responds with friendly, contextual travel recommendations for that specific destination.	[ ]
TC-16	**User Registration**	Go to `/signup`, register a new user.	Returns token, stores user in `localStorage`, updates Navbar with user's name.	[ ]
TC-17	**Saved Trips Dashboard**	Log in, visit `/dashboard`.	Displays all trips generated by this user. Clicking a trip card opens its full details.	[ ]

2. API Endpoint Verification (curl Commands)

You can run these curl commands in your terminal to verify backend REST endpoints independently of the frontend:

1. Backend Health Check

```
curl -X GET http://localhost:5001/api/health
```

Expected Response:

```
{
  "status": "ok",
  "service": "TripWise AI Backend",
  "uptime": 12.45,
  "timestamp": "2026-09-08T..."
}
```

---

2. City Geocoding Search

```
curl -X GET "http://localhost:5001/api/places/search-cities?q=Athens&country=Greece"
```

Expected Response:

```
{
  "success": true,
  "source": "local",
  "data": [
    {
      "id": "athens_gr",
      "city": "Athens",
      "country": "Greece",
      "coordinates": { "lat": 37.9838, "lon": 23.7275 }
    }
  ]
}
```

---

3. Weather Forecast

```
curl -X GET "http://localhost:5001/api/weather?destination=Paris&lat=48.8566&lon=2.3522"
```

Expected Response:

```
{
  "destination": "Paris",
  "coordinates": { "lat": 48.8566, "lon": 2.3522 },
  "current": { "temperature_2m": 22.4, ... },
  "forecast": [
    { "day": "Mon", "high": 24, "low": 15, "weatherCode": 1 },
    ...
  ]
}
```

---

4. User Signup & JWT Generation

```
curl -X POST http://localhost:5001/api/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"name":"Test Traveler","email":"traveler@tripwise.ai","password":"securepassword123"}'
```

Expected Response:

```
{
  "success": true,
  "message": "Account created successfully.",
  "token": "eyJhbGciOiJIUzI1Ni...",
  "data": {
    "id": "usr_...",
    "name": "Test Traveler",
    "email": "traveler@tripwise.ai"
  }
}
```

---

5. Generate Trip & Persist to SQLite

```
curl -X POST http://localhost:5001/api/trips/generate \
  -H "Content-Type: application/json" \
  -d '{
    "city": "London",
    "destination": "Athens, Greece",
    "country": "Greece",
    "coordinates": {"lat": 37.9838, "lon": 23.7275},
    "budget": 1800,
    "duration": 5,
    "travelStyle": "Culture, City Exploration"
  }'
```

Expected Response: `HTTP 201 Created` with full joined trip record containing `id`, `days`, `activities`, and `budgetBreakdown`.

---

3. Fallback & Offline Engine Verification

To demonstrate that TripWise AI is immune to API quota exhaustion or network downtime:

1. **Simulate Missing Gemini API Key:**
   - In `server/.env`, temporarily comment out the key: `# GEMINI_API_KEY=...`
   - Restart backend: `npm run server:dev`
2. **Generate a Trip for a Curated Hub:**
   - Go to `/generate-trip`, set destination to *"Seoul"*, duration `5`, budget `2000`.
   - Click "Generate Trip".
   - **Verification:** The trip generates in under 1 second. Inspect Day 1: it renders *"Gyeongbokgung Palace Royal Guard Ceremony"* with 100% authentic timings and costs from `CURATED_DESTINATION_ITINERARIES`.
3. **Generate a Trip for an Uncurated City:**
   - Set destination to *"Chandigarh, India"*, duration `3`, budget `800`.
   - Click "Generate Trip".
   - **Verification:** Procedural engine synthesizes unique, non-repeating daily themes with proportional budget allocations.
4. **Restore Key:** Uncomment `GEMINI_API_KEY` in `server/.env`.

---

4. Security & Edge-Case Verification

Test Scenario	Input Data	Expected Defense
SQL Injection Attempt	Destination: `' OR 1=1; DROP TABLE trips; --`	Treated strictly as a literal string. Prepared statement safely searches for that literal city name without executing SQL commands.
XSS Script Injection	City: `<script>alert('xss')</script>`	React automatically escapes JSX string literals. Rendered safely as harmless text on screen.
Password Timing Attack	Submitting password with correct first 10 characters vs wrong first character	`crypto.timingSafeEqual()` processes both in identical execution time.
Duplicate Account Creation	Re-registering `traveler@tripwise.ai`	Server responds with `HTTP 409 Conflict` ("Account with this email already exists").
Tampered JWT Token	Modifying any payload character in the JWT token string	Server rejects with `HTTP 401 Unauthorized` ("Invalid authorization token") because HMAC signature verification fails.

PART 17

TripWise AI — 8-Day Mastery Study Plan

This study plan guides you step-by-step through the TripWise AI codebase, preparing you for college project vivas, technical presentations, and developer interviews. Spend 60 to 90 minutes each day following the structured tasks and self-tests.

Day 1: Project Vision, Architecture & The 3 Pitches

Focus:

Understand what problem the project solves, how user journeys flow, and memorize your elevator pitches.

Tasks:

1. Read `01_PROJECT_OVERVIEW.md`.
2. Inspect the live application in your browser (`http://localhost:5173`):
   - Explore the Landing page (`/`).
   - Open `/generate-trip` and observe the form.
   - Click "Generate Trip" and review the resulting `/trip-details` page.
3. Practice reciting the **30-Second Elevator Pitch** aloud 3 times until fluent.
4. Review the **1-Minute** and **3-Minute** comprehensive pitches.

Self-Test:

- [ ] Can I explain why manual trip planning is frustrating without looking at notes?
- [ ] Can I name the 4 key layers of the system (React, Express, SQLite, Gemini)?
- [ ] Can I deliver the 30-second elevator pitch without hesitation?

Day 2: Tech Stack Deep Dive & Dependency Auditing

Focus:

Master the exact technologies and version numbers, and justify why each tool was chosen over industry alternatives.

Tasks:

1. Read `02_TECH_STACK.md`.
2. Open `package.json` in the project root:
   - Identify `react` (`^19.2.6`), `vite` (`^8.0.12`), `tailwindcss` (`^4.3.0`), `leaflet` (`^1.9.4`), `recharts` (`^3.8.1`).
3. Open `server/package.json`:
   - Identify `@google/generative-ai` (`^0.24.1`), `express` (`^4.21.2`), `cors` (`^2.8.5`).
   - Note that `node:sqlite` and `node:crypto` are **built directly into Node.js 22+** and require zero npm packages!
4. Learn the answers to: *"Why Vite over Create React App?"* and *"Why native SQLite over MongoDB?"*

Self-Test:

- [ ] What version of React is used in this project? (React 19)
- [ ] Why does SQLite have no npm package in `server/package.json`? (Built into Node.js 22)
- [ ] What port does each service run on? (Frontend: 5173, Backend: 5001)

Day 3: Frontend Architecture, Component Hierarchy & React Hooks

Focus:

Understand how React components communicate, the difference between props and state, and how React hooks power the UI.

Tasks:

1. Read `04_FRONTEND_GUIDE.md`.
2. Open `src/App.jsx` and review the React Router DOM route declarations.
3. Inspect `src/components/Tripgenerator/Form.jsx`:
   - Identify every `useState` call (lines 11–30).
   - Identify the lazy initializer reading `location.search`.
4. Inspect `src/components/Tripgenerator/CityDropdown.jsx`:
   - Identify `useMemo` for local search.
   - Identify `useEffect` for the 320ms debounce and outside-click listener.
   - Identify `useRef` for input auto-focus.

Self-Test:

- [ ] Can I explain the difference between State and Props with a code example?
- [ ] Why is `useMemo` used in `CityDropdown.jsx`?
- [ ] How does `useRef` auto-focus the search input without causing a re-render?

Day 4: Trip Form, Empty-State Principles & Dual-Tier Geocoding

Focus:

Understand form validation, zero-default UX principles, and how the hybrid local/remote geocoding engine operates.

Tasks:

1. Read `05_FORM_AND_CITY_SEARCH.md`.
2. Open `src/data/destinations.js`:
   - Verify that there are exactly **178 cities across 56 countries**.
   - Review the `filterDestinations()` helper function.
3. Open `src/components/Tripgenerator/CityDropdown.jsx`:
   - Trace the 320ms `setTimeout` and `clearTimeout` cleanup.
   - Trace how the ES6 `Map` deduplicates cities using `${city}_${country}`.
4. Review the validation constraints in `Form.jsx` (positive budget, duration between 1 and 30 days, at least one style chip).

Self-Test:

- [ ] Why was having hardcoded defaults (like Seoul) a bad UX design?
- [ ] What happens when a user types a city with 1 character vs 2+ characters?
- [ ] How does `clearTimeout` prevent unnecessary HTTP requests during rapid typing?

Day 5: Backend Architecture, Express Middleware & REST APIs

Focus:

Understand the Express request-response cycle, middleware mechanics, HTTP status codes, and endpoint routing.

Tasks:

1. Read `06_BACKEND_GUIDE.md`.

2. Open `server/src/index.js`:
   - Trace the middleware pipeline: `cors()`, `express.json()`, request logger, routers, static handler, global error handler.
3. Open `server/src/routes/trips.js`:
   - Trace `POST /generate`: input validation, `Promise.all` calling Gemini and Unsplash, saving to SQLite, returning `HTTP 201`.
4. Study the HTTP status codes: `200`, `201`, `400`, `401`, `404`, `409`, `500`.

Self-Test:

- [ ] Why is `cors()` required?
- [ ] What is the purpose of `express.json()`?
- [ ] What HTTP status code is returned when a new trip is generated? (`201 Created`)

Day 6: Google Gemini AI, Prompt Engineering & Fallback Engine

Focus:

Understand how the LLM is prompted, how JSON schemas are strictly enforced, how hallucinations are caught, and how the fallback engine guarantees 100% uptime.

Tasks:

1. Read `07_GEMINI_INTEGRATION.md`.
2. Open `server/src/services/geminiService.js`:
   - Review the system prompt and JSON schema constraints.
   - Review the candidate models loop: `["gemini-1.5-flash", "gemini-2.0-flash", "gemini-1.5-pro"]`.
   - Review the regex sanitization stripping markdown backticks (`replace(/```json/gi, "")`).
   - Review the `seenTitles` Set duplicate activity detection.
   - Review `CURATED_DESTINATION_ITINERARIES` for Seoul, Tokyo, Paris, and Athens.
3. Test running the app with the Gemini API key commented out in `server/.env` to see the fallback engine in action.

Self-Test:

- [ ] Why call Gemini from the backend instead of the frontend? (API key security)
- [ ] How does TripWise AI detect if the AI recommended the same landmark twice?
- [ ] What happens if the user has no Gemini API key or runs out of quota?

Day 7: Database Architecture, SQLite Schema & Security

Focus:

Master the 7 relational tables, foreign key constraints, atomic transactions, WAL mode, and password cryptography.

Tasks:

1. Read `08_DATABASE_GUIDE.md` and `11_CODE_EXPLANATION.md`.
2. Open `server/src/db/schema.js`:
   - Trace the 7 tables: `users`, `trips`, `itinerary_days`, `activities`, `budget_breakdown`, `trip_highlights`, `saved_places`.
   - Identify the B-Tree indexes and `ON DELETE CASCADE`.
3. Open `server/src/db/repositories.js`:
   - Trace `tripsRepo.saveTrip()`: `BEGIN TRANSACTION`, parameter bindings, `COMMIT`, and `ROLLBACK`.
4. Open `server/src/routes/auth.js`:
   - Review `crypto.scryptSync()` and `crypto.timingSafeEqual()`.
   - Review the HMAC-SHA256 JWT generation.

Self-Test:

- [ ] What are the ACID properties and how does `saveTrip()` enforce them?
- [ ] What does `PRAGMA journal_mode = WAL;` do?
- [ ] Why use `crypto.timingSafeEqual()` instead of `===` for passwords?

Day 8: Viva Rehearsal, Bug Explanations & Final Polish

Focus:

Practice answering real examiner questions and confidently explaining the bugs you solved.

Tasks:

1. Read `14_BUGS_AND_FIXES.md` and pick your favorite 3 bugs to explain in an interview.
2. Read `15_TESTING_GUIDE.md` and execute the 5 core curl commands in your terminal.
3. Randomly select and answer 15 questions from `13_VIVA_QUESTIONS.md`.
4. Keep `17_FINAL_CHEAT_SHEET.md` handy for morning-of-presentation review.

Self-Test:

- [ ] Can I explain 2 major bugs I encountered and how I fixed them?
- [ ] Can I demonstrate the app live, create a trip, and show the database entry?
- [ ] Am I confident and ready to ace the viva? **Yes!**

PART 18

TripWise AI — Final Viva & Presentation Morning Cheat Sheet

Keep this document open on the morning of your viva, project evaluation, or technical interview. It contains the most vital facts, commands, metrics, and instant answers in a condensed, easy-to-read format.

1. Quick Commands & Runtime URLs

Purpose	Terminal Command	Working Directory	Active URL
Start Frontend	`npm run dev`	`tripwise-ai/`	`http://localhost:5173`
Start Backend	`npm run server:dev` *(with auto-restart)*	`tripwise-ai/`	`http://localhost:5001`
Backend Health	`curl http://localhost:5001/api/health`	Anywhere	`status: "ok"`
Database File	`ls -la server/src/data/tripwise.db*`	`tripwise-ai/`	Binary SQLite file + `-wal` + `-shm`

2. Key Architecture Metrics (Memorize These Numbers!)

- **Frontend Framework:** React `19.2.6` (React 19!) + Vite `8.0.12`
- **Styling:** Tailwind CSS `4.3.0`
- **Backend Runtime & Server:** Node.js `v22.5.0+` + Express `4.21.2`
- **Database Engine:** Native `node:sqlite` (`DatabaseSync`) with zero external npm dependencies
- **Database Tables:** Exactly **7 relational tables** (`users` , `trips` , `itinerary_days` , `activities` , `budget_breakdown` , `trip_highlights` , `saved_places`)
- **City Catalog:** Exactly **178 curated cities across 56 countries** with pre-verified GPS coordinates
- **Search Debounce Delay: 320 milliseconds**
- **AI Models:** Google Gemini (`gemini-1.5-flash` , `gemini-2.0-flash` , `gemini-1.5-pro`)
- **Weather Provider:** Open-Meteo API (**0 API keys required**, 100% free)
- **Geocoding Provider:** OpenStreetMap Nominatim with custom `User-Agent: TripWiseAI/1.0`
- **Password Security:** Salted `crypto.scryptSync(password, salt, 64)` + `crypto.timingSafeEqual()`
- **Token Security:** Custom signed stateless HMAC-SHA256 JWT tokens

3. Top 10 Viva Questions & Instant Punchy Answers

1. What is TripWise AI?

> *"TripWise AI is a full-stack automated travel planning application that transforms user travel preferences—origin, destination, budget, duration, and style—into a day-by-day, hour-by-hour realistic itinerary enriched with live weather forecasts, interactive Leaflet maps, visual budget charts, and persistent SQLite storage."*

2. Why did you choose native `node:sqlite` over MongoDB or PostgreSQL?

> *"Because travel itineraries are inherently relational (Users own Trips, Trips own Days, Days own Activities, Trips own Budget Breakdowns). SQLite provides strict foreign keys, cascade deletes, and ACID transactions in an embedded, serverless single file (`tripwise.db`) with zero npm dependencies and zero external database server setup required."*

3. How does the city search dropdown work without lagging?

> *"We built a dual-tier hybrid engine: Tier 1 filters 178 popular cities in memory in under 1 millisecond. Tier 2 uses a 320ms debounced asynchronous call to OpenStreetMap Nominatim to cover small towns worldwide. Both result sets are merged and deduplicated using an ES6 Map."*

4. What happens if your Gemini API key expires or the internet goes down during the demo?

> *"The app never crashes. We engineered a zero-failure fallback engine with pre-computed, verified multi-day itineraries for major destinations (Seoul, Paris, Tokyo, Athens, etc.) and procedural schedule generators for custom cities, ensuring the app generates a complete, authentic trip under any conditions."*

5. How do you prevent the AI from repeating the same landmark across different days?

> *"In `geminiService.js`, we iterate over all generated activities using an ES6 `Set` (`seenTitles`). If any activity title appears more than once across different days, we catch the hallucination, discard the repetitive schedule, and activate our curated non-repeating engine."*

6. How are passwords stored and verified securely?

> *"Passwords are never stored in plain text. We generate a 16-byte random salt and derive a 64-byte key using `crypto.scryptSync`. During login, we extract the salt, recompute the hash, and compare buffers using `crypto.timingSafeEqual()` to eliminate cryptographic timing attacks."*

7. What is Write-Ahead Logging (WAL) in SQLite?

> *"In traditional SQLite journaling, writes place an exclusive file lock, blocking concurrent readers. Under `PRAGMA journal_mode = WAL;`, writes are appended to a separate log file, allowing concurrent read operations without database lock errors (`SQLITE_BUSY`)."*

8. How does `tripsRepo.saveTrip()` ensure data consistency?

> *"Saving a trip touches 5 different tables. We wrap all inserts in an atomic ACID transaction (`BEGIN TRANSACTION ... COMMIT`). If an error occurs on any day or activity, `ROLLBACK` undoes all writes completely, preventing partial or orphaned records."*

9. Why did you use Vite instead of Create React App?

> *"Create React App is officially deprecated and relies on slow Webpack bundling. Vite leverages native ES Modules (ESM) in the browser, starting dev servers in under 300 milliseconds and updating components instantly via Hot Module Replacement (HMR)."*

10. How do the frontend and backend communicate?

> *"Via a RESTful JSON API. The frontend uses a centralized `api.js` client using the browser's native `fetch()` API. The Express backend uses CORS and `express.json()` middleware to parse requests and return standardized JSON responses with appropriate HTTP status codes (200, 201, 400, 401, 404, 500)."*

---

4. Live Demo Golden Path (Step-by-Step Examiner Presentation)

Follow this exact sequence during your live presentation for a flawless demonstration:

1. **Start on Landing Page (`/`):**
   - Point out the modern hero section, clean responsive navbar, and curated trip cards.
   - Click the **"Create Trip"** button in the navbar.
2. **Show the Form (`/generate-trip`):**
   - **Point out that the form is completely blank by default** (no hardcoded defaults!).
   - In **Starting City**, type *"Chandigarh"* (demonstrate instant `<1ms` search from local catalog).
   - In **Destination City**, type *"Athens, Greece"* (show coordinates resolving automatically).
   - Enter **Budget**: `1800`.
   - Enter **Duration**: `5` days.
   - Select **Travel Styles**: Click *Culture* 🏛️ and *City Exploration* 🏙️.
3. **Submit the Trip:**
   - Click **"Generate Trip"**.

- Point out the loading state spinner.
- The app navigates to `/trip-details?id=trip_...` within 3–6 seconds.

4. **Walk Through the Trip Details Page (`/trip-details`):**
   - **Hero Section:** Highlight the dynamic destination cover photo, date range, and budget tags.
   - **Left Column:** Expand Day 1 and Day 2. Show the detailed timeline, activity tags, costs, durations, and AI travel recommendations.
   - **Right Column:**
     - Point out the **Interactive Leaflet Map** centered over Athens.
     - Show the **Live 7-day Weather Forecast** from Open-Meteo.
     - Show the **Recharts Budget Breakdown** pie chart.
     - Show the **Travel Tips** and **Trip Highlights** cards.
   - **AI Travel Concierge:** Click the floating chat button in the bottom right corner. Type: *"What is the best way to travel from Athens airport to the city center?"*. Show the prompt and friendly answer.
5. **Demonstrate Persistence on Dashboard (`/dashboard`):**
   - Click **"Dashboard"** in the navbar.
   - Show that the newly generated Athens trip is permanently stored in SQLite and displays in the saved trips grid with its cover photo and metadata.
   - Click the card to reopen the trip details instantly!